Azure

For Developers

Azure for Developers

01
Introduction to Azure

02
Running your apps on Azure

03
Storage in the cloud

04
Building a communication strategy for your applications

05
Security & Identity

06
Monitoring & Tooling

Four-day programme

Day 1

Introduction to Azure and services

- Cloud concepts and IaaS, PaaS, SaaS
- Azure infrastructure
- App Service, Functions, Logic Apps

Day 2

Container-based applications

- Containers on Azure
- Container Apps and AKS
- Registries
- Hosting choices compared

Application settings

- App Configuration
- KeyVaults

Day 3

Storage, databases and communication patterns

- Azure storage
- Databases on Azure
- Service Bus and Event Hubs

Day 4

Monitoring and infrastructure as code

- Monitoring with Azure Monitor
- Application Insights
- Azure Resource Manager and Bicep

Introduction to Azure

Introduction to Azure

01

Core Cloud computing concepts

02

What is IaaS, PaaS & SaaS?

03

Overview of Azure Service

04

Exploring the Azure Portal

What is Microsoft Azure?

- Azure is a cloud computing platform from Microsoft
  - Provides a hosted environment for applications, apps, and data storage:
  - Always available

- Benefits of cloud computing for software vendors:
  - Lower costs potentially
  - Ability to cope with elastic demand (e.g. peak demand, fast growth in demand, periodic bursts in demand)
  - Better availability

Introduction to Cloud Computing

- Owning data centers is not for everyone
  - Costly
  - Complicated
  - High maintenance
  - Very low utilizations of hardware, network and power
- Cloud computing
  - Central data centers for providing hosting services
  - On-demand, scalable, unlimited computation and storage

Benefits of Cloud Computing

Cloud computing introduces many benefits to software vendors

Fast

On-demand computing and storage resources

Lower costs

Pay for what you use instead of owning data centres

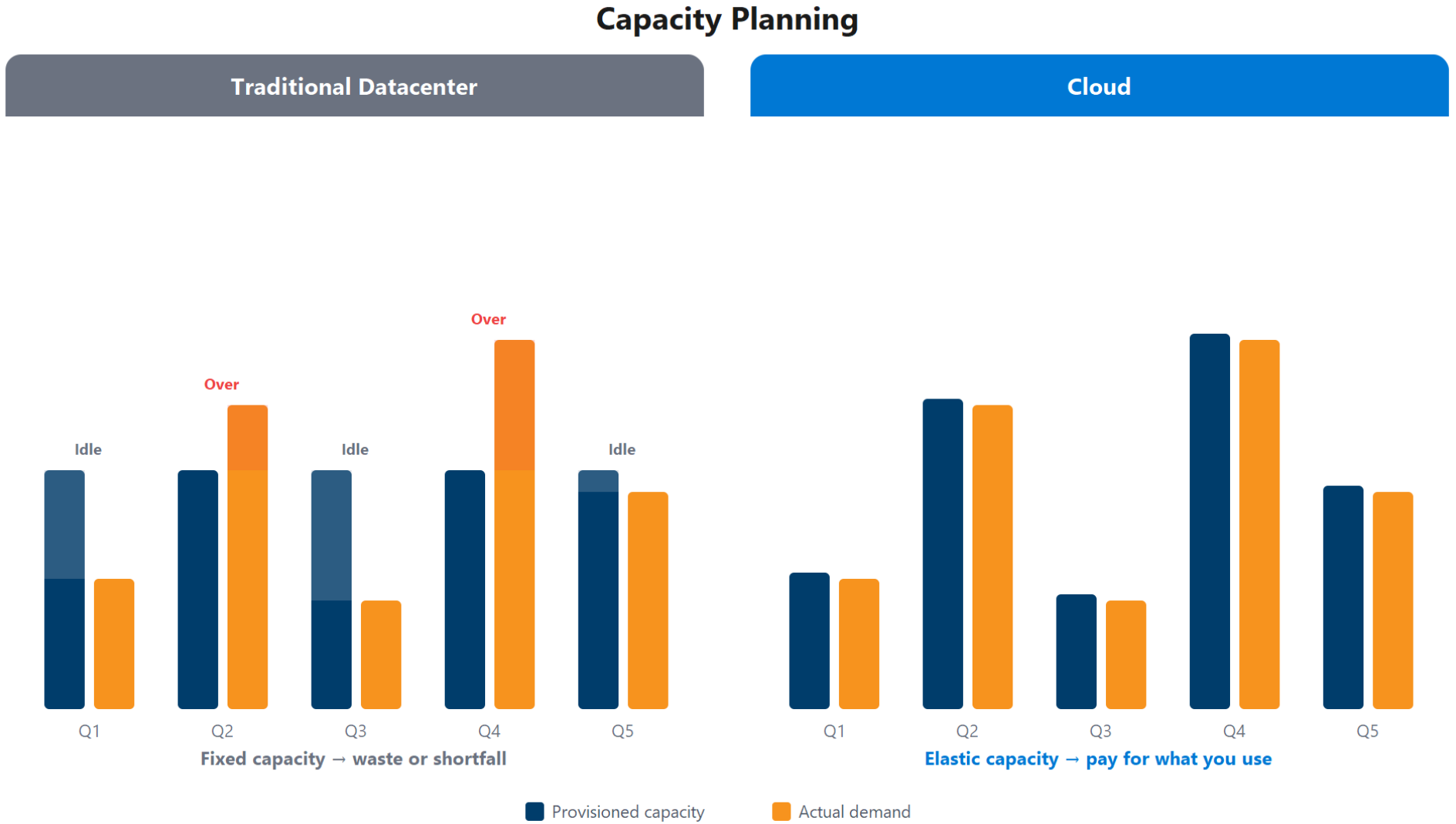
Better availability

Redundant infrastructure across regions and zones

Elasticity

Scale out and in as demand changes

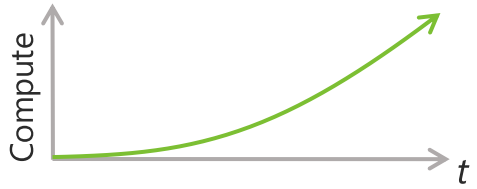
The consumption-based model



© Copyright Microsoft Corporation. All rights reserved.

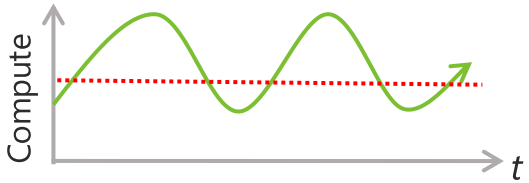
Cloud Elasticity

- Cloud elasticity deals with different growth patterns:



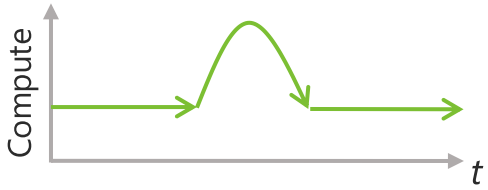
Fast Growth

Successful services needs to scale to answer the demand



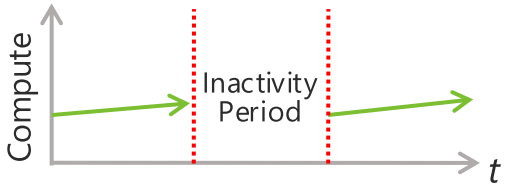
Predictable Bursts

Services with seasonality trends, peaks due to periodic increased demand.



Unpredictable Bursting

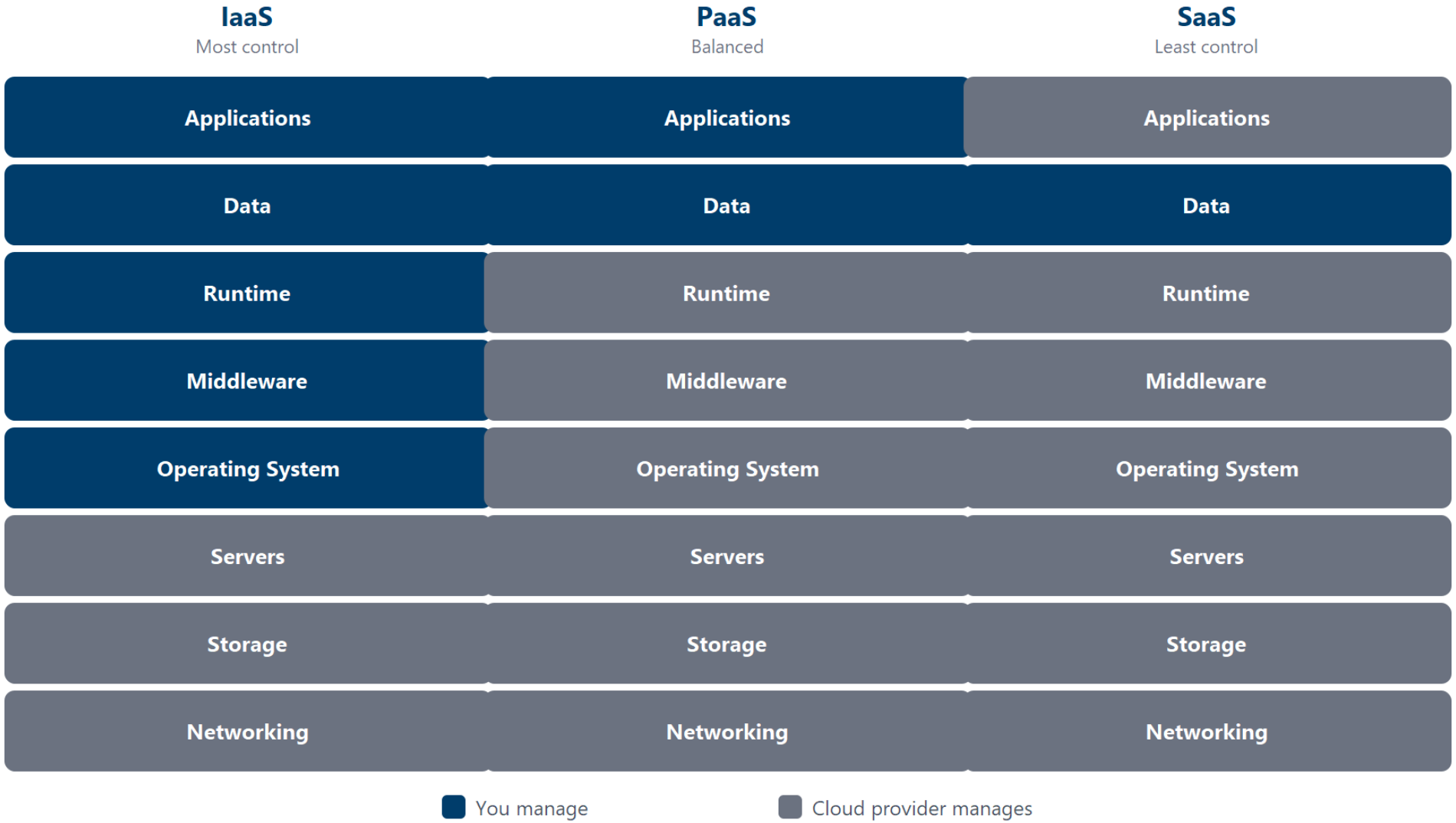
Unexpected or unplanned peak in demand



Periodical Processing

On & off workloads (long idle times) or batch processing

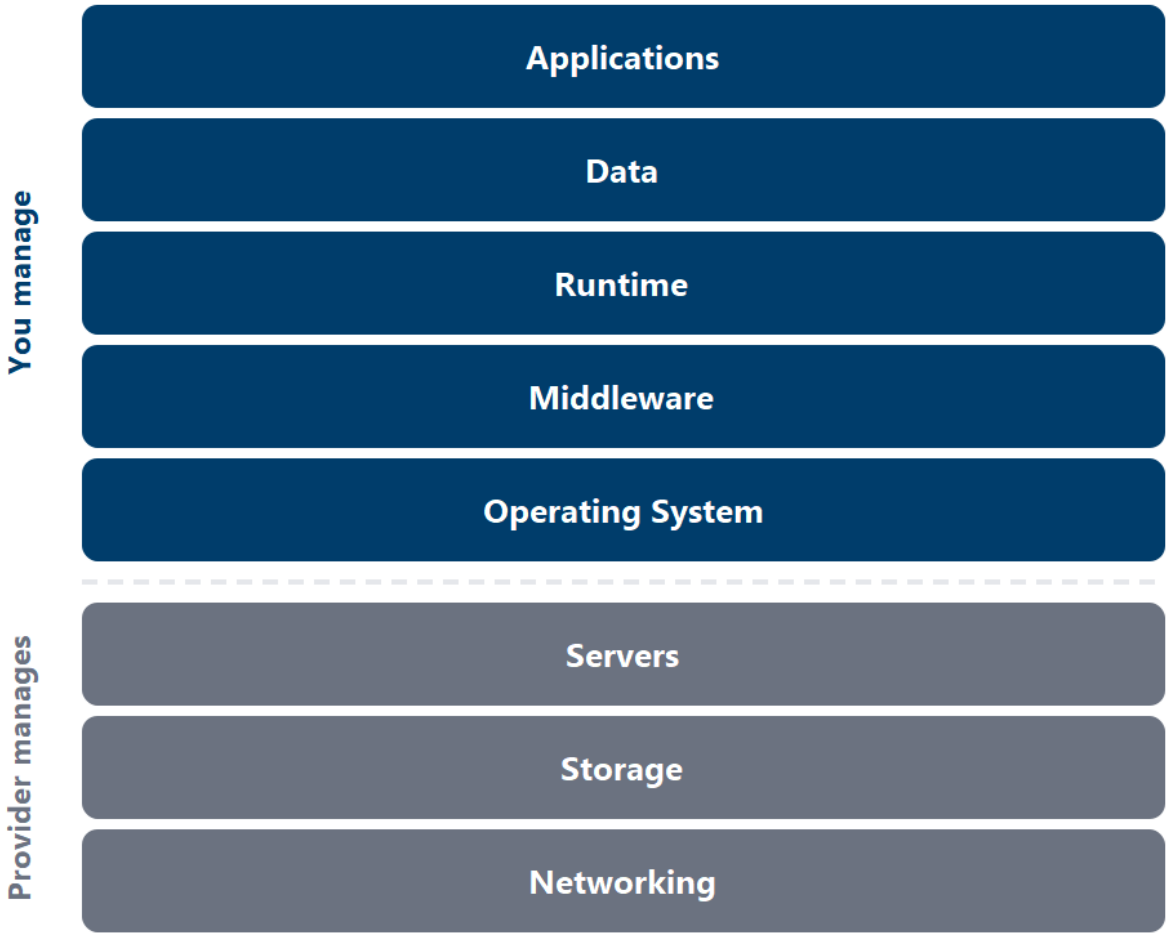
Cloud service types comparison



© Copyright Microsoft Corporation. All rights reserved.

Infrastructure as a service (IaaS)

Maximum control — you manage the workload stack



Common Scenarios

Lift-and-shift migration

Set up cloud resources similar to on-premises, then move workloads to IaaS infrastructure.

Testing and development

Rapidly replicate dev/test environments.
Start up or shut down with complete control.

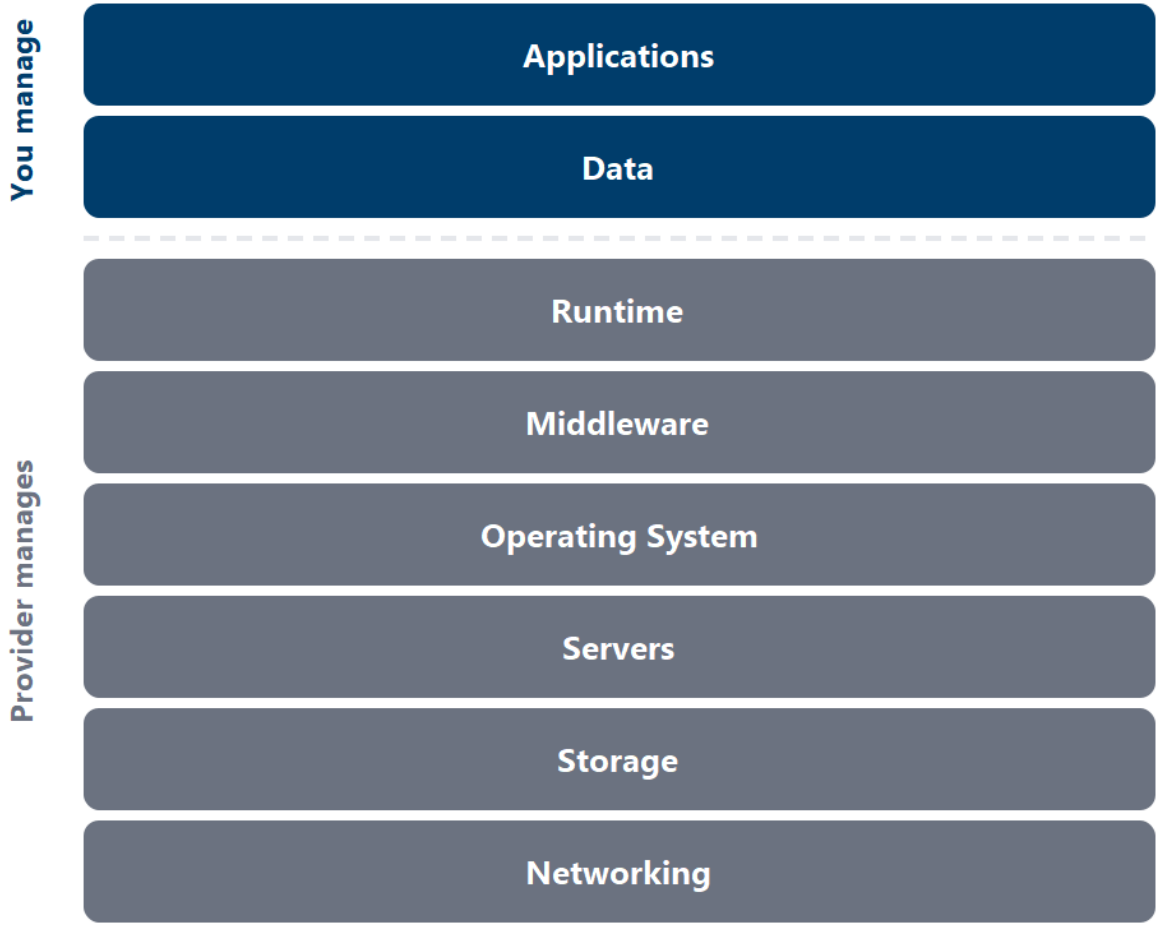
Key takeaway

You rent hardware — what you do with it is up to you.

© Copyright Microsoft Corporation. All rights reserved.

Platform as a service (PaaS)

Balanced — you focus on your application and data



Common Scenarios

Development framework

Build cloud-based applications using built-in components with scalability and high availability.

Analytics and business intelligence

Analyze and mine data, find insights, and predict outcomes using platform tools.

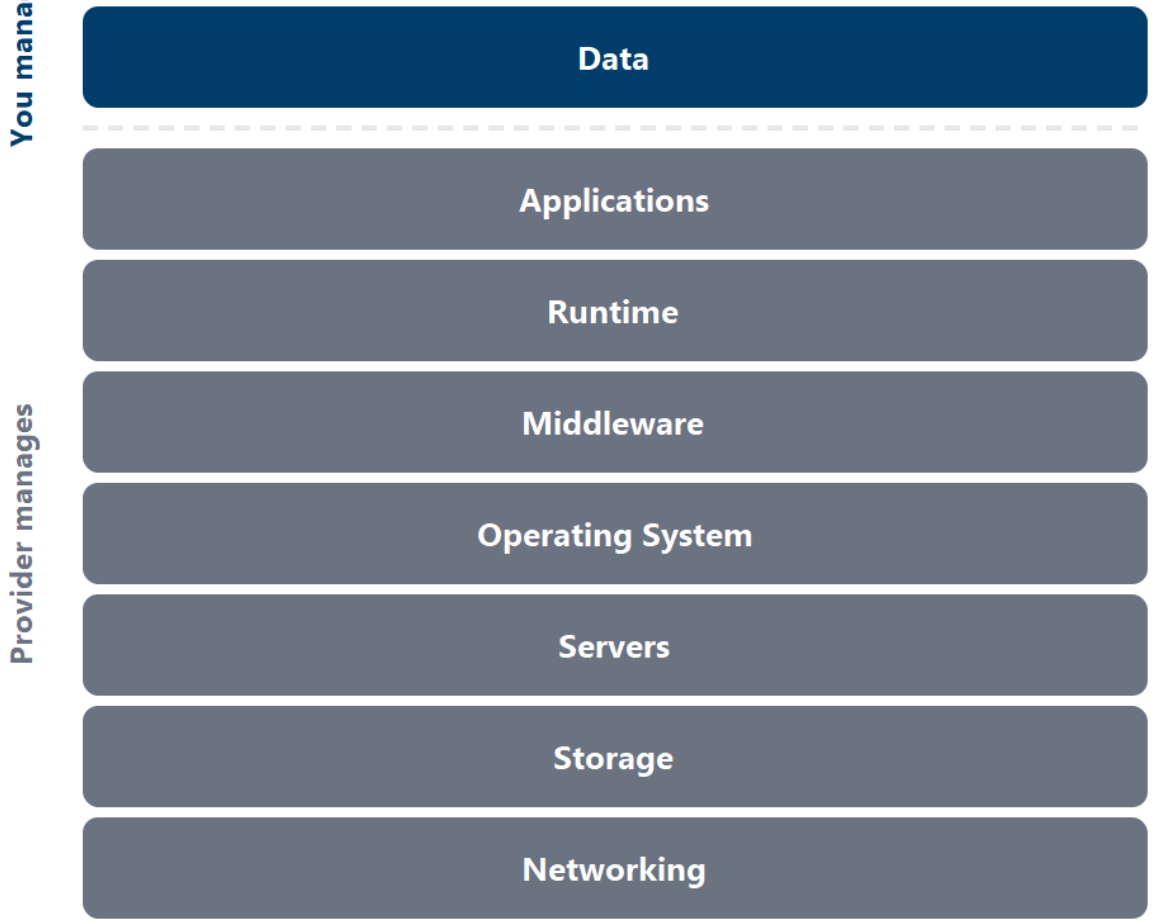
Key takeaway

You build on the platform — no infrastructure headaches.

© Copyright Microsoft Corporation. All rights reserved.

Software as a service (SaaS)

Least control — you use the fully managed application



Common Scenarios

Email and messaging
Microsoft 365, Teams, Outlook

Productivity applications
Word, Excel, PowerPoint online

Finance and expense tracking
Cloud-based financial management tools

Key takeaway
You use the app — the provider handles everything else.

© Copyright Microsoft Corporation. All rights reserved.

What is Microsoft Azure?

Microsoft Azure Service Categories

Compute	Networking	Storage	Databases	AI + ML
Virtual Machines	Virtual Network	Blob Storage	Azure SQL	Azure OpenAI
App Service	Load Balancer	Azure Files	Azure Cosmos DB	AI Services
Azure Functions	VPN Gateway	Queue Storage	PostgreSQL Flex	Machine Learning
Container Apps	ExpressRoute	Table Storage	MySQL Flex	Bot Service
Azure Kubernetes Service	Azure DNS	Managed Disks	Redis Cache	AI Search

Identity + Security	DevOps + Management	IoT	Analytics	Integration
Microsoft Entra ID	Azure DevOps	IoT Hub	Synapse Analytics	Logic Apps
Key Vault	Azure Monitor	IoT Central	Data Factory	API Management
Defender for Cloud	ARM Templates	Digital Twins	Databricks	Service Bus
DDoS Protection	Azure Policy	Stream Analytics	HDInsight	Event Grid
Firewall	Cost Management	Event Hubs	Azure Data Explorer	Notification Hubs

Hyper scale Infrastructure is the enabler

80+ announced regions and 500+ datacenters in 34 countries

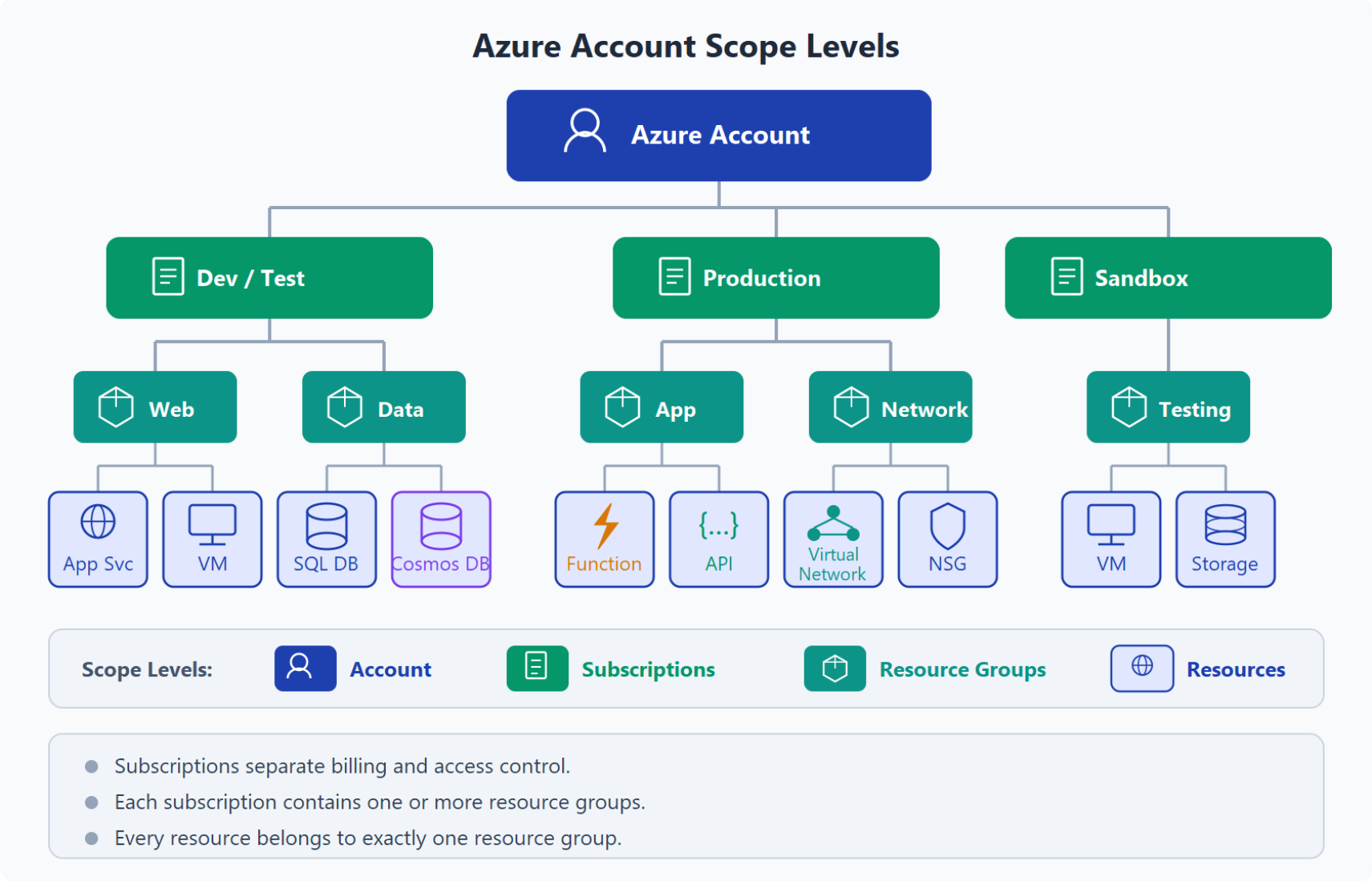


- Global private fiber network linking all regions
- More regions than any other cloud provider
- Availability zones in most regions

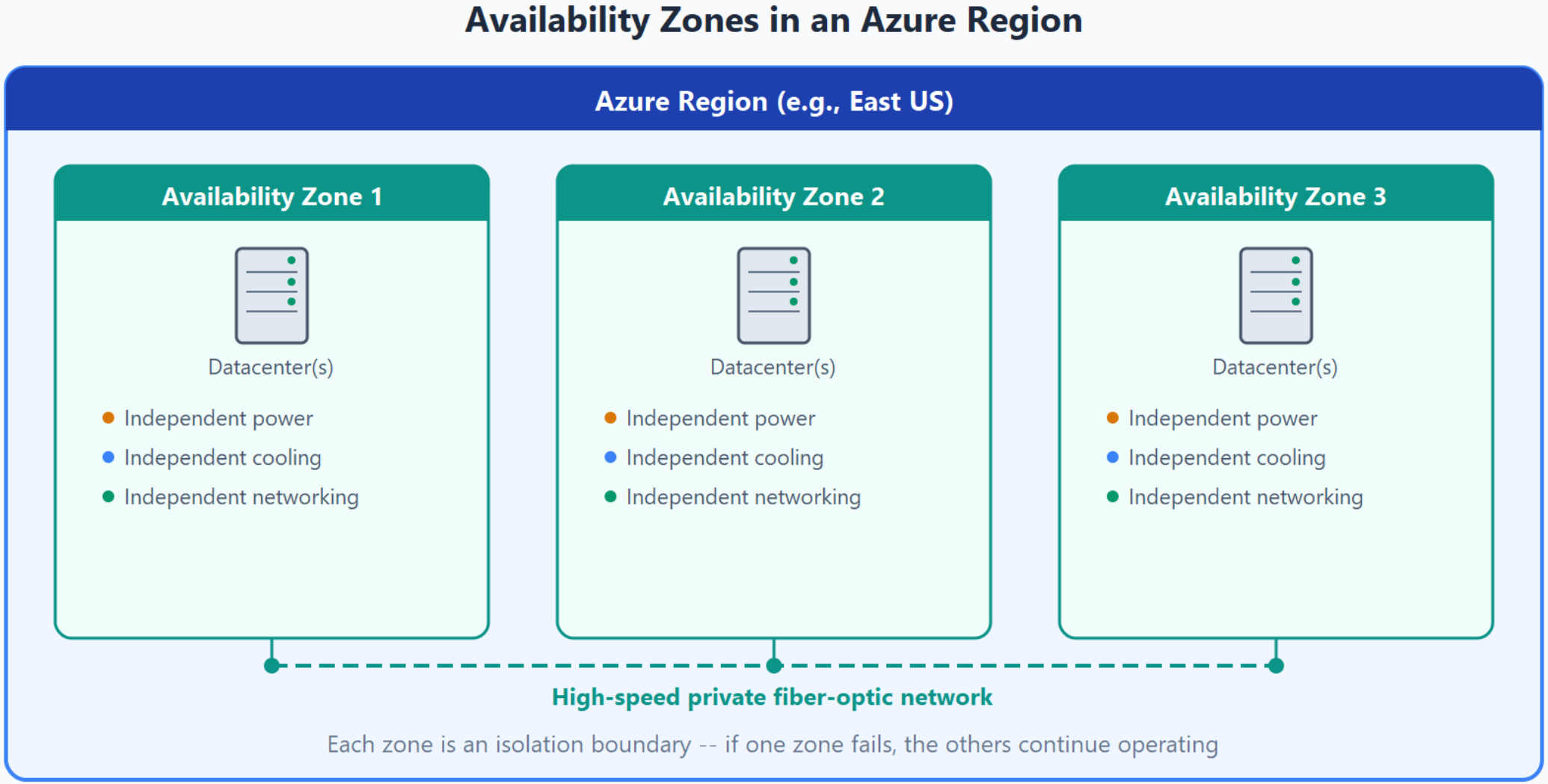
Operational

* Operated by 21Vianet Map shows a selection of regions; see datacenters.microsoft.com for all

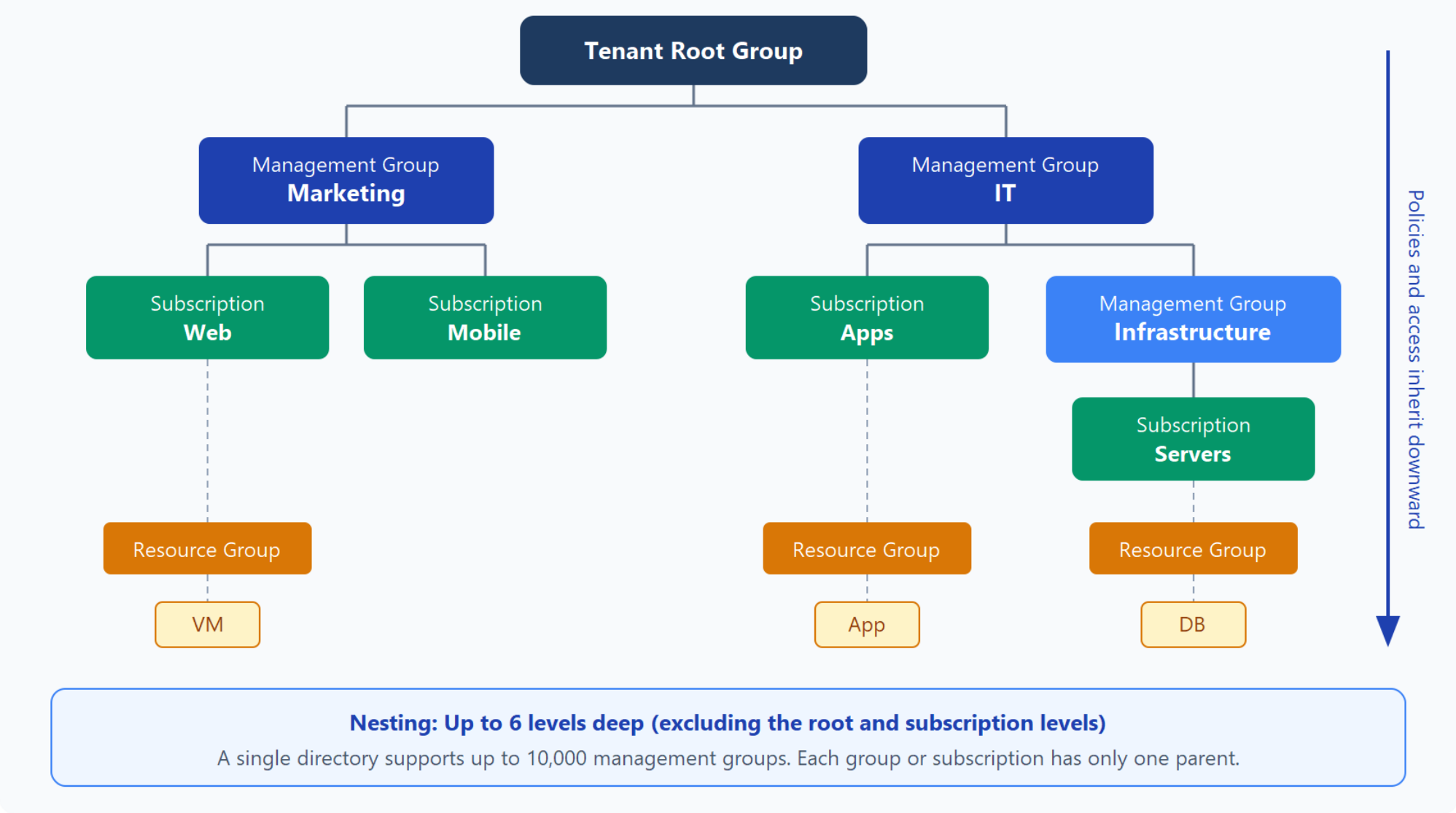
Get started with Azure accounts



Azure physical infrastructure



Azure management infrastructure



Nesting: Up to 6 levels deep (excluding the root and subscription levels)

A single directory supports up to 10,000 management groups. Each group or subscription has only one parent.

Azure Compliance
The largest compliance portfolio in the industry



ISO 27001



SOC 1 Type 2



SOC 2 Type 2



PCI DSS Level 1



Cloud Controls Matrix



ISO 27018



Content Delivery and Security Association



Shared Assessments



FedRAMP JAB P-ATO



HIPAA / HITECH



FIPS 140-2



21 CFR Part 11



FERPA



DISA Level 2



CJIS



IRS 1075



ITAR-ready



Section 508 VPAT



European Union Model Clauses



EU Safe Harbor



United Kingdom G-Cloud



China Multi Layer Protection Scheme



China GB 18030



China CCCPPF



Singapore MTCS Level 3



Australian Signals Directorate



New Zealand GCIO



Japan Financial Services



ENISA IAF

Azure is Open Cloud

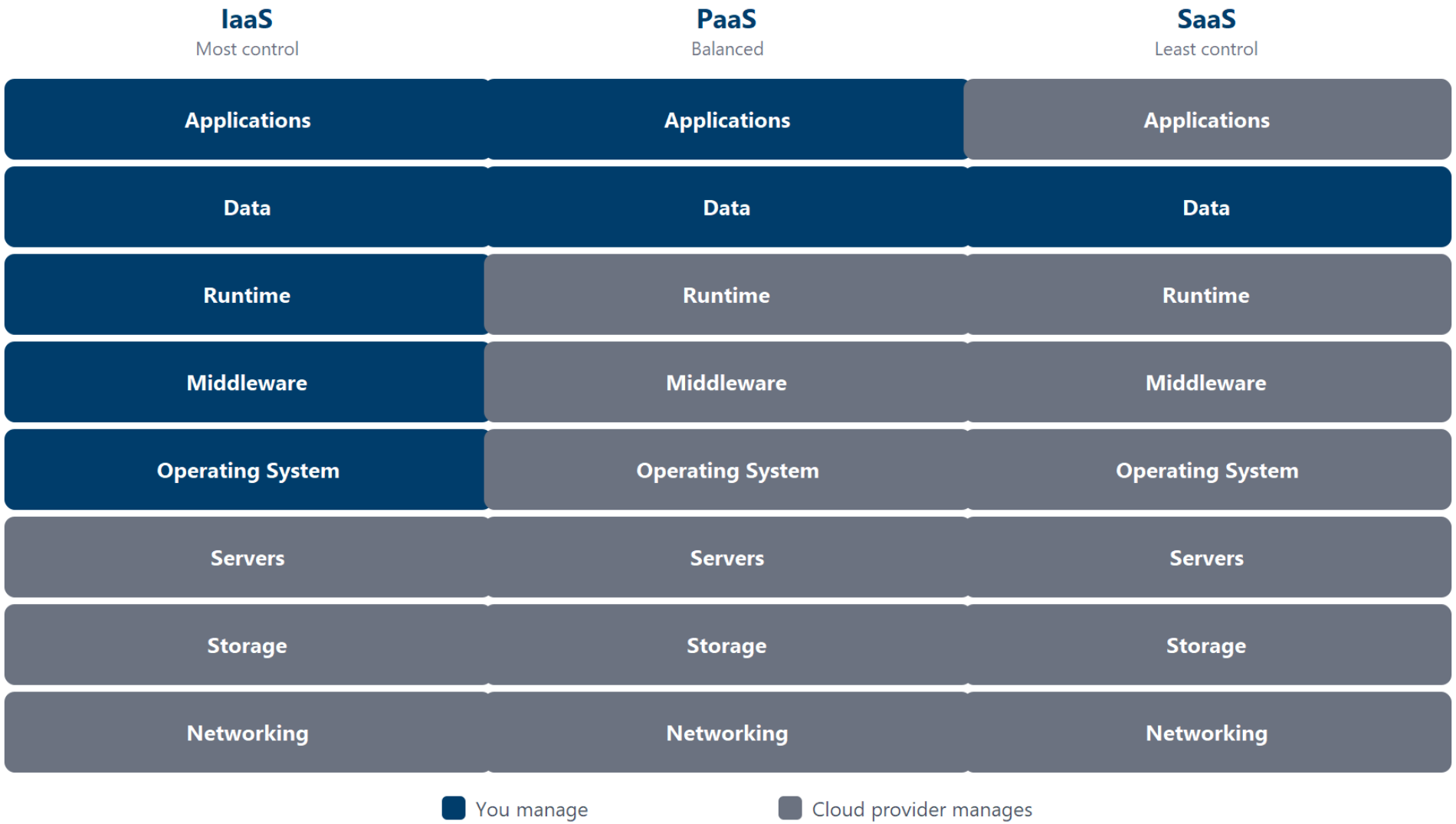


Application Hosting

Characteristics of Distributed Applications

- Modern applications consume data from distributed data sources to fulfill users' expectations
- Distributed applications are designed for:
  - Scalability
  - Low latency
  - Availability
  - Reliability
  - Security and privacy

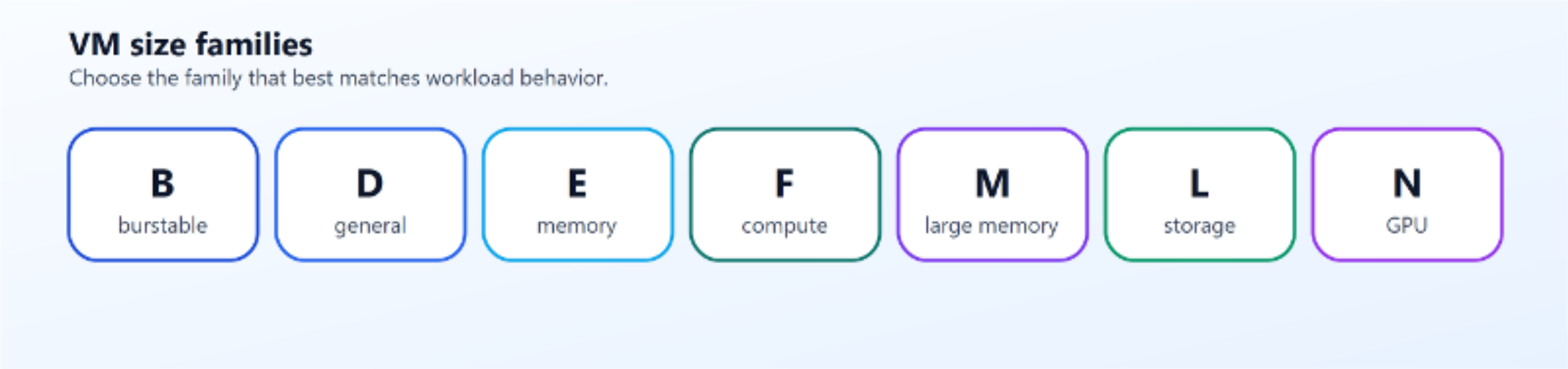
Cloud service types comparison



Microsoft Azure IaaS

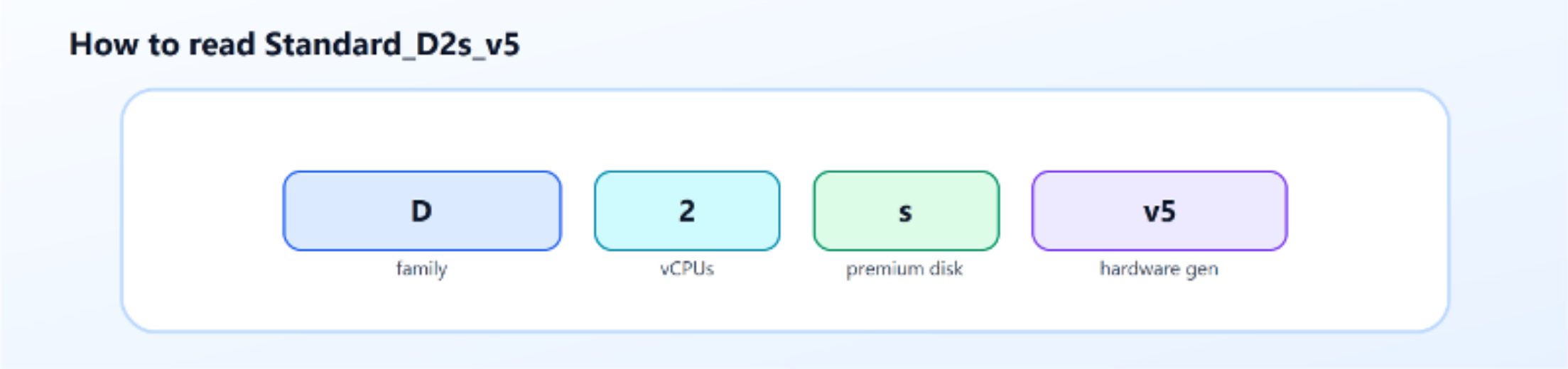
- Microsoft Azure IaaS provide virtual machines you can create and control
  - Windows and Linux operating systems
  - Persistent OS and data disks that are backed-up and cloned for durability
  - Fine-grained control over deployed software
  - Support for hosting stateful services and applications such as SQL Server, SharePoint, and Active Directory
  - Easy migration to and from on-premises
  - Support for custom virtual networks configuration and software pre-requirements

Azure Virtual Machines families



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Decoding Azure Virtual Machine names



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Azure SaaS

- Some examples of SaaS services powered by Microsoft Azure
  - Microsoft 365 (Exchange Online, SharePoint Online)
  - Microsoft Teams
  - Dynamics 365 (CRM and ERP)
  - GitHub and Microsoft Copilot

Azure PaaS compute options

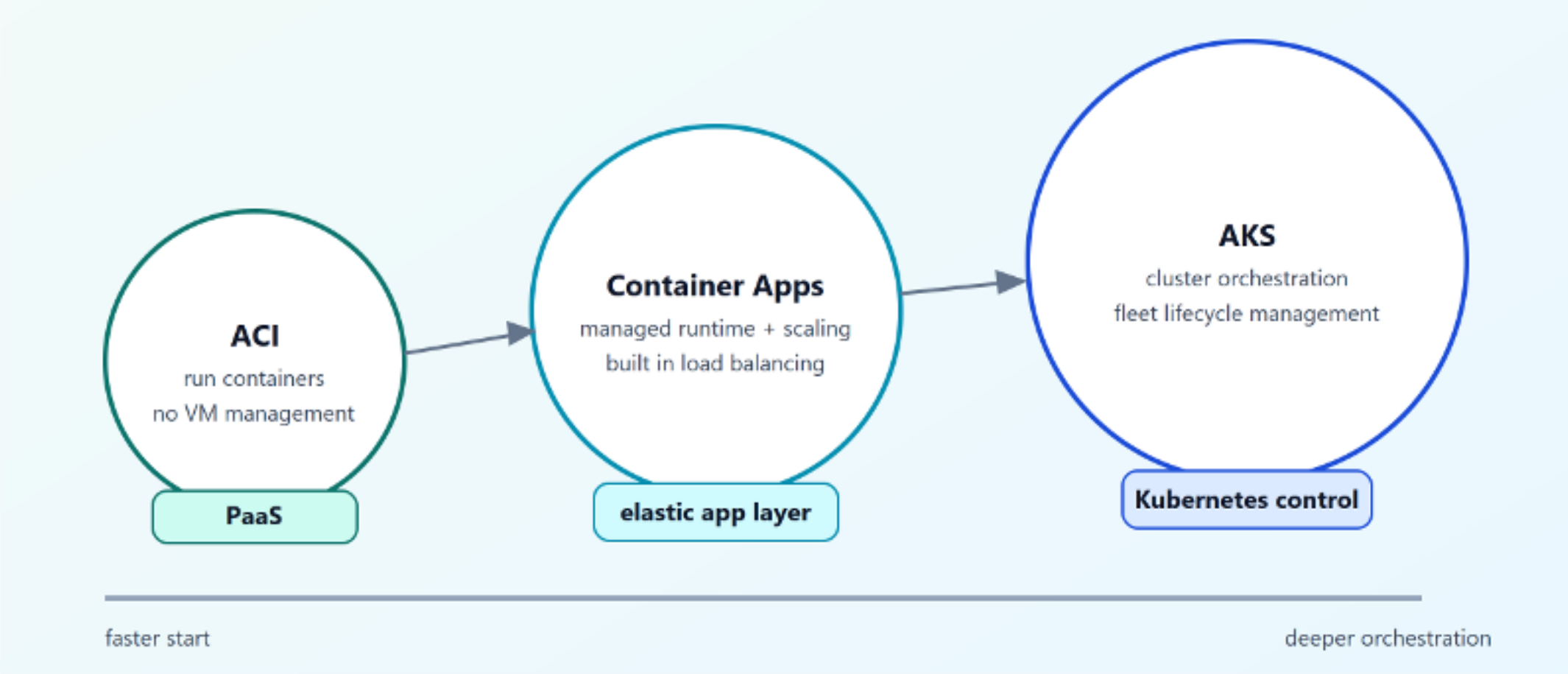
- Azure offers managed hosting for every type of workload
  - App Service: web apps, APIs and WebJobs
  - Azure Functions: event-driven, serverless code
  - Azure Container Apps: serverless containers and microservices
  - Azure Kubernetes Service (AKS): managed Kubernetes
  - Service Fabric managed clusters: stateful microservices
- Cloud Services (Web & Worker roles) are retired
  - Classic retired 31 Aug 2024; extended support retires 31 Mar 2027

Application hosting options



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

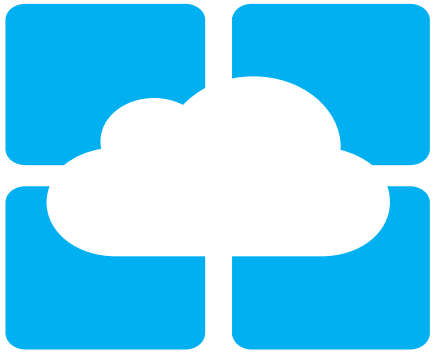
Azure container services



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Azure Application Components

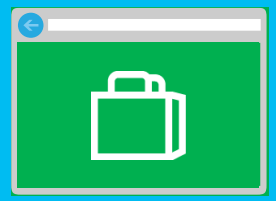
- Azure provides a broad range of supporting services, including:
  - Storage services (Blob, Files, Queues, Tables)
  - Messaging: Service Bus, Event Hubs, Event Grid
  - Identity: Microsoft Entra ID (replaces Access Control Service)
  - Caching: Azure Managed Redis
  - Content delivery: Azure Front Door
  - Databases as a service: Azure SQL, Azure Cosmos DB



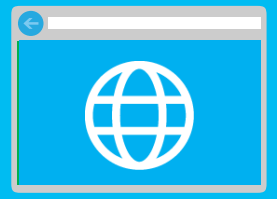
Azure App Service

“PaaS that developers love and businesses can trust”

Apps



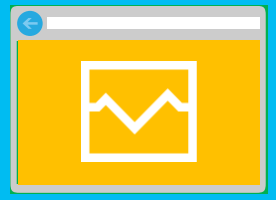
eCommerce



Digital Global Presence



Custom Apps



LOB



API / Services / ISV

Services



APIM



Containers



Logic Apps



API Apps



Notification Hubs



Web Apps



Azure Functions

Platform



App Service

Fully Managed Platform **Enterprise Grade**

Developer Experience

- Auto scale
- OS and Framework patching
- Load balancing
- Languages and Frameworks
- Superior DevOps
- Self service supportability
- Enterprise grade SLA
- Security and Compliance
- On-Premise Connectivity



Web Apps



Create and deploy mission-critical web apps that scale with your business.

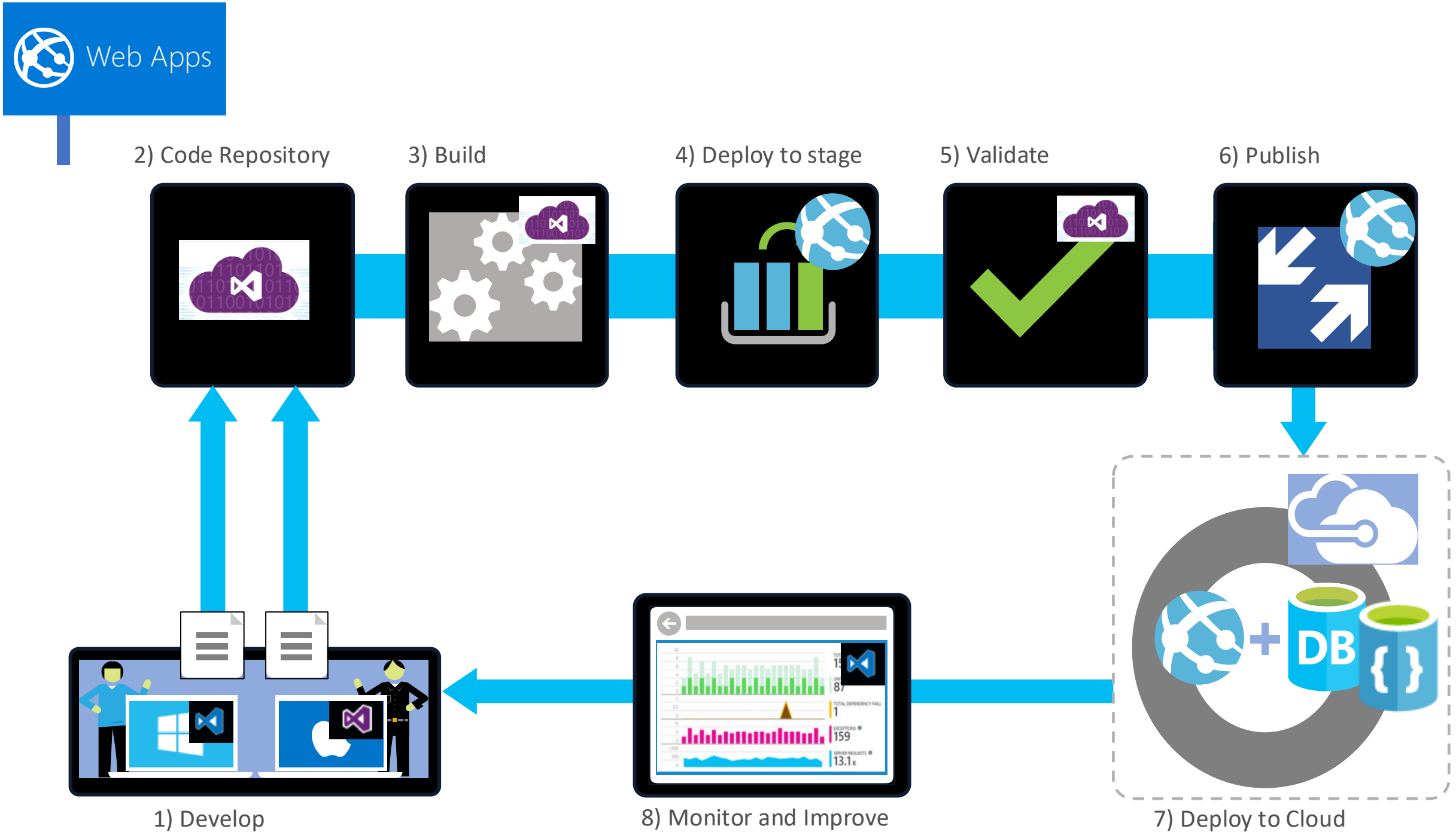
Supports .NET, Java, PHP, Node.js, Python and containers

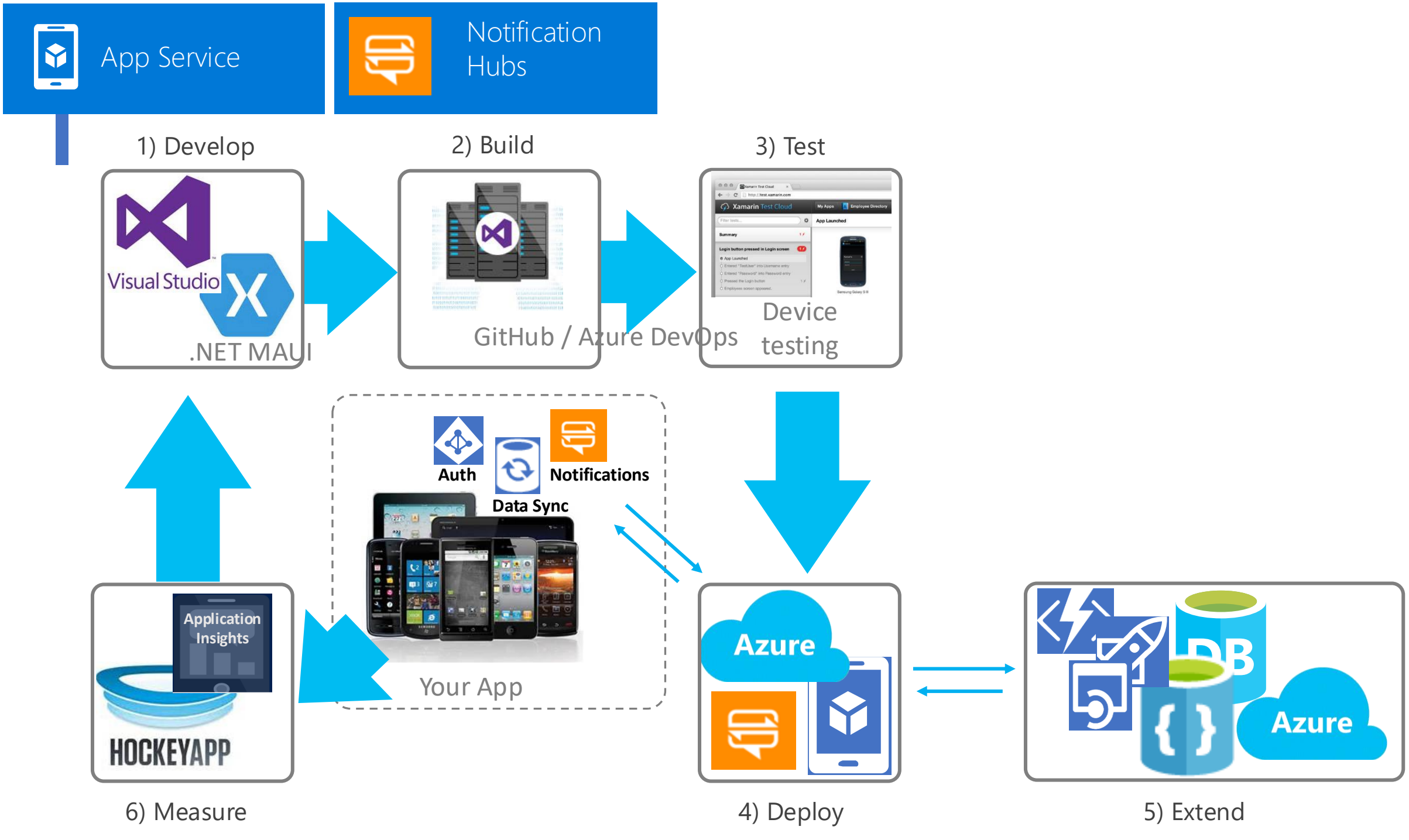
Built-in auto-scale and load balancing

High availability with auto-patching

Continuous deployment with GitHub Actions, Azure DevOps and Git

Supports WordPress, Umbraco, Joomla, and Drupal


Web Apps
2) Code Repository
3) Build
4) Deploy to stage
5) Validate
6) Publish
DB
1) Develop
8) Monitor and Improve
7) Deploy to Cloud


App Service
Notification Hubs
1) Develop
2) Build
3) Test
Visual Studio
.NET MAUI
GitHub / Azure DevOps
Xamarin Test Cloud
Device testing
Auth
Notifications
Data Sync
Your App
Application Insights
HOCKEYAPP
Azure
DB
Azure
6) Measure
4) Deploy
5) Extend

What are WebJobs?

A flexible way to run background jobs: .cmd, .bat, .exe, .ps1, .sh, .php, .py, .js, .jar

Continuous

Runs constantly in the background

On demand

Started manually or via the API

Scheduled

Runs on a CRON schedule

WebJobs scale with your App Service plan

How are they stored and started?

WebJobs are stored within the website

- site\wwwroot\app_data\jobs\{job_type}\{job_name}

Will decide what to run by a first one found wins rule:

1. First look for a file called run.{supported extension}
2. If not found, look for any file with supported extension
3. If not, then not a runnable WebJob

Paths & Files

File/Path	
disable.job	Existence of file stops the job. Deleting it starts the job.
settings.job	Allows you to indicate: is_singleton to scale with website. { “is_singleton”: true } or { “is_singleton”: false }
site\wwwroot\App_Data\jobs\continuous	Path for continuous web jobs
site\wwwroot\App_Data\jobs\triggered	Path for on demand and scheduled web jobs
data\jobs	Data directories for jobs

WebJob SDK

- A method can be “triggered” by some event
- Route parameters and bindings help simplify inputs and outputs
- A few options for logging that surface information in the Azure Portal

WebJob SDK – Triggers and Bindings

- Queues (Storage and ServiceBus)
- Blobs
- Tables

- Project Site:
- https://github.com/Azure/azure-webjobs-sdk

WebJob SDK Extensions

- TimerTrigger
- FileTrigger
- SendGrid
- ErrorTrigger
- WebHooks

- Project Site:
- https://github.com/Azure/azure-webjobs-sdk-extensions

Many Options for Deploying

Azure portal

Zip deploy

Visual Studio and VS Code

Right-click publish

Azure CLI

az webapp deploy

GitHub Actions

CI/CD from your repo

Azure DevOps Pipelines

CI/CD with approvals

FTPS

File-based upload

Kudu (SCM site)

Drag-and-drop deploy

Scheduling with Logic Apps

Actions

- HTTP, queues & connectors
- Run-after error handling
- Retry policies

Cloud-ilities

- High availability
- Execution reliability

Scheduling

- Recurrence trigger
- Sliding window
- Time zones & schedules

Monitoring

- Run history & Azure Monitor

Auth

- Managed identity
- OAuth (Entra ID)
- Client certificates

Management

- Portal
- Bicep / ARM templates
- Azure CLI
- VS Code

Logic Apps vs. WebJobs

- Logic Apps
  - Serverless workflow service with a Recurrence trigger
  - Schedules and orchestrates calls; your code runs elsewhere
  - Great if you call an external endpoint or API
  - Run history, retries and alerts built in
- Azure WebJobs
  - Runs in the context of an App Service web app -> same compute resources
  - Hosts your code, not only schedules it

Container Apps vs. Continuous WebJobs

- Azure Container Apps
- Dedicated, independently scaled compute
- Event-driven scaling (KEDA), including scale to zero
- Scales independently of your web app
- Full control over the image (install any dependency)

- Requires a container build and registry
- More moving parts to deploy

- Azure WebJobs
- Same compute resources as your web app
- Scales with your App Service plan
- WebJobs SDK simplifies code
- Simple and quick deployment model
- Simple out-of-the-box execution history and monitoring

Related Azure Services

Logic Apps

- Recurring or event-driven workflows that call services
- 1,000+ connectors, run history and retries built in

Azure Functions

- Serverless, event-driven code
- Timer trigger with CRON expressions
- Pay per execution (Flex Consumption)

Batch

- Large-scale application execution – 1000's of VMs
- Existing or new applications
- Invoke via command line
- On-demand or recurring work

App Service WebJobs

- Small-scale execution of work within the context of a web app
- WebJobs SDK simplifies app development
- CRON schedules built in

Automation

- Hosted execution of PowerShell and Python runbooks
- On-demand or recurring invocation

Container Apps, AKS or Virtual Machines

- Full control, but most work

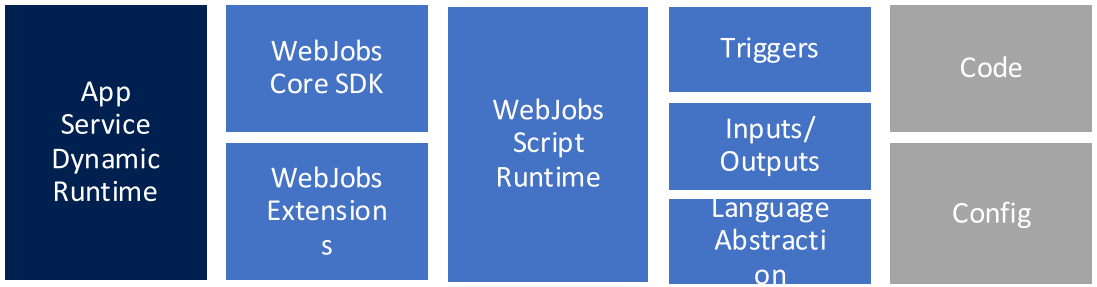
Function Apps

- Built on App Service and the WebJobs SDK
- Serverless
- Pay per use (Flex Consumption) or reserved (Premium, Dedicated)

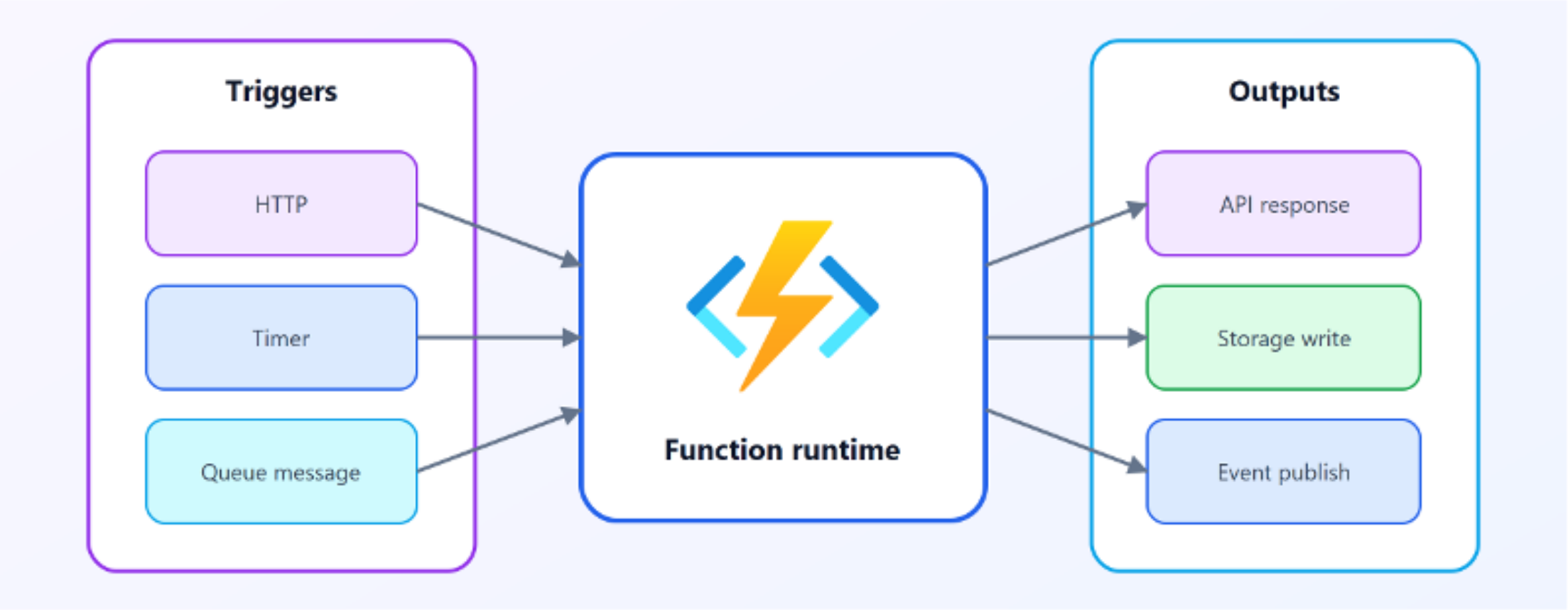


Azure Functions architecture

- Azure Functions is built around the WebJobs SDK runtime. The WebJobs SDK makes it easy to react to events and work with data in a consistent abstracted fashion.
- Your code runs in a separate language worker process. For .NET, use the isolated worker model: the in-process model reaches end of support on 10 November 2026.

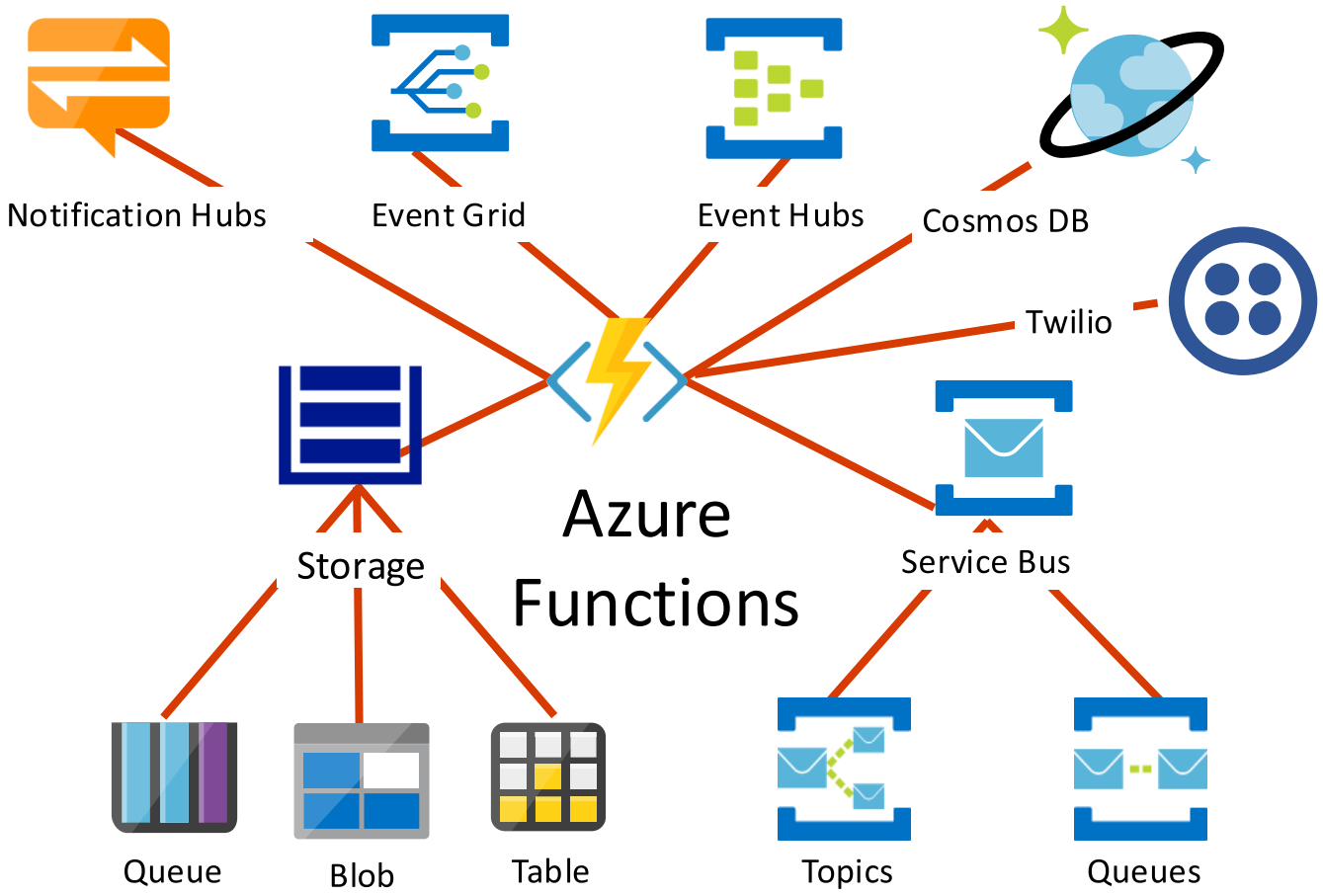


Azure Functions



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Discover Azure Functions



Discover Azure Functions

Compare Azure Functions and Azure Logic Apps

	Durable Functions	Logic Apps
Development	Code-first (imperative)	Designer-first (declarative)
Connectivity	About a dozen built-in binding types, write code for custom bindings	Large collection of connectors, Enterprise Integration Pack for B2B scenarios, build custom connectors
Actions	Each activity is an Azure function; write code for activity functions	Large collection of ready-made actions
Monitoring	Azure Application Insights	Azure portal, Azure Monitor logs
Management	REST API, Visual Studio	Azure portal, REST API, PowerShell, Visual Studio
Execution context	Can run locally or in the cloud	Supports run-anywhere scenarios

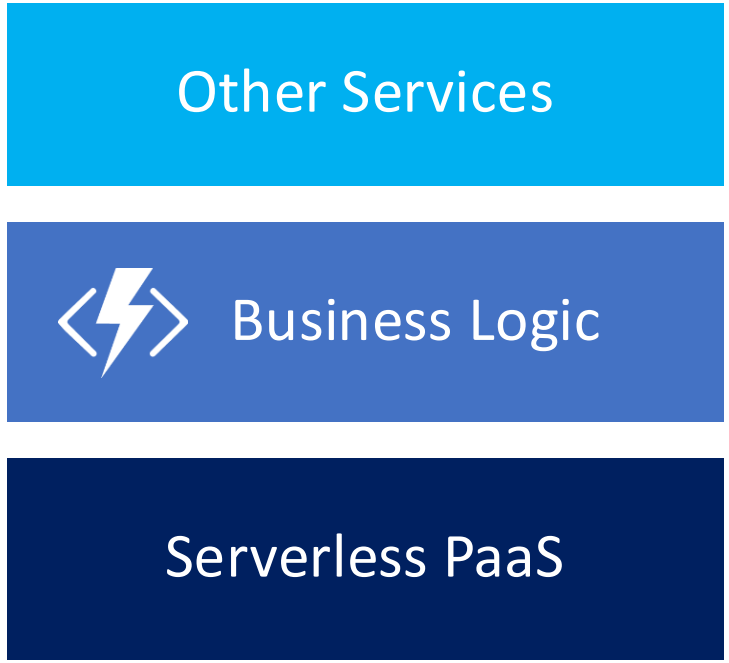
Discover Azure Functions

Compare Functions and WebJobs

	Functions	WebJobs with WebJobs SDK
Serverless app model with automatic scaling	Yes	No
Develop and test in browser	Yes	No
Pay-per-use pricing	Yes	No
Integration with Logic Apps	Yes	No
Trigger events	Timer Azure Storage queues and blobs Azure Service Bus queues and topics Azure Cosmos DB Azure Event Hubs HTTP/WebHook (GitHub Slack) Azure Event Grid	Timer Azure Storage queues and blobs Azure Service Bus queues and topics Azure Cosmos DB Azure Event Hubs File system

Dual abstraction

- Serverless compute abstracts away the compute
- Bindings abstract away the services you interact with



Hosting plans and scaling

- Azure Functions offers several hosting plans
- Flex Consumption (recommended for serverless)
  - Pay per execution; scales to zero; VNet integration; always-ready instances
- Consumption (legacy)
  - Linux Consumption retires 30 September 2028
- Premium and Dedicated (App Service plan)
  - Pay for reserved instances; no cold start
  - Dedicated: you're responsible for scale

Flex Consumption pricing

Pay-per-execution model with two meters

Executions

Number of function executions

Execution time × memory

Instance memory of 512 MB, 2 GB or 4 GB

Optional always-ready instances are billed separately, even when idle. A monthly free grant applies to on-demand usage.

Develop Azure Functions



Explore Azure Functions development

Overview

A function contains two important pieces:

- **Your code**, which can be written in a variety of languages
- **Some config**, the *function.json* file.

The *function.json* file defines the function's trigger, bindings, and other configuration settings. Every function has one and only one trigger. The runtime uses this config file to determine the events to monitor and how to pass data into and return data from a function execution.

```
{
  "disabled":false,
  "bindings":[
      // ... bindings here
      {
          "type": "bindingType",
          "direction": "in",
          "name": "myParamName",
          // ... more depending on binding
      }
  ]
}
```

Explore Azure Functions development

Function app

A function app provides an execution context in Azure in which your functions run.

- It is the unit of deployment and management for your functions.
- A function app is comprised of one or more individual functions that are managed, deployed, and scaled together.
- All of the functions in a function app share the same pricing plan, deployment method, and runtime version.
- Think of a function app as a way to organize and collectively manage your functions.

Explore Azure Functions development

- Folder structure
- All code for a function app is located in a root project folder that contains a host configuration file.
- The *host.json* file contains runtime-specific configurations and is in the root folder of the function app.
- Specific folder structures required by the function app depend on language. To the right is an example of the folder structure of a JavaScript project.

```
FunctionsProject
  | - MyFirstFunction
  | | - index.js
  | | - function.json
  | - MySecondFunction
  | | - index.js
  | | - function.json
  | - SharedCode
  | | - myFirstHelperFunction.js
  | | - mySecondHelperFunction.js
  | - node_modules
  | - host.json
  | - package.json
  | - extensions.csproj
```

Explore Azure Functions development

Local development environments

Functions makes it easy to use your favorite code editor and development tools to create and test functions on your local computer.

- Your local functions can connect to live Azure services, and you can debug them on your local computer using the full Functions runtime.
- The way in which you develop functions on your local computer depends on your language and tooling preferences.

Create triggers and bindings

Overview

- Triggers are what cause a function to run. A trigger defines how a function is invoked and a function must have exactly one trigger.
- Binding to a function is a way of declaratively connecting another resource to the function; bindings may be connected as input bindings, output bindings, or both.
- You can mix and match different bindings to suit your needs.
- Triggers and bindings let you avoid hardcoding access to other services.

Create triggers and bindings

Trigger and binding definitions

Triggers and bindings are defined differently depending on the development language.

C# class library - decorating methods and parameters with C# attributes

Java - decorating methods and parameters with Java annotations

JavaScript/PowerShell/Python/TypeScript - updating function.json schema

```
{
  "dataType": "binary",
  "type": "httpTrigger",
  "name": "req",
  "direction": "in"
}
```

Create triggers and bindings

Binding direction

All triggers and bindings have a direction property in the *function.json* file:

- For triggers, the direction is always `in`
- Input and output bindings use `in` and `out`
- Some bindings support a special direction `inout`. If you use `inout`, only the **Advanced editor** is available via the **Integrate** tab in the portal.

When you use attributes in a class library to configure triggers and bindings, the direction is provided in an attribute constructor or inferred from the parameter type.

Create triggers and bindings

Azure Functions trigger and binding example

You want to write a new row to Azure Table storage whenever a new message appears in Azure Queue storage.

This scenario can be implemented using an Azure Queue storage trigger and an Azure Table storage output binding.

```
"bindings": [
    {
      "type": "queueTrigger",
      "direction": "in",
      "name": "order",
      "queueName": "myqueue-items",
      "connection": "STORAGE_ACCT_SETTING"
    },
    {
      "type": "table",
      "direction": "out",
      "name": "$return",
      "tableName": "outTable",
      "connection": "STORAGE_ACCT_SETTING"
    }
```

Create triggers and bindings

C# script example

Here's C# script code that works with this trigger and binding.

```
...
public static Person Run(JObject order, ILogger log)
{
    return new Person() {
        PartitionKey = "Orders",
        RowKey = Guid.NewGuid().ToString(),
        Name = order["Name"].ToString(),
        MobileNumber = order["MobileNumber"].ToString() };
}
public class Person
{
    public string PartitionKey { get; set; }
    public string RowKey { get; set; }
    public string Name { get; set; }
    public string MobileNumber { get; set; }
}
```

Create triggers and bindings

JavaScript example

The same *function.json* file can be used with a JavaScript function:

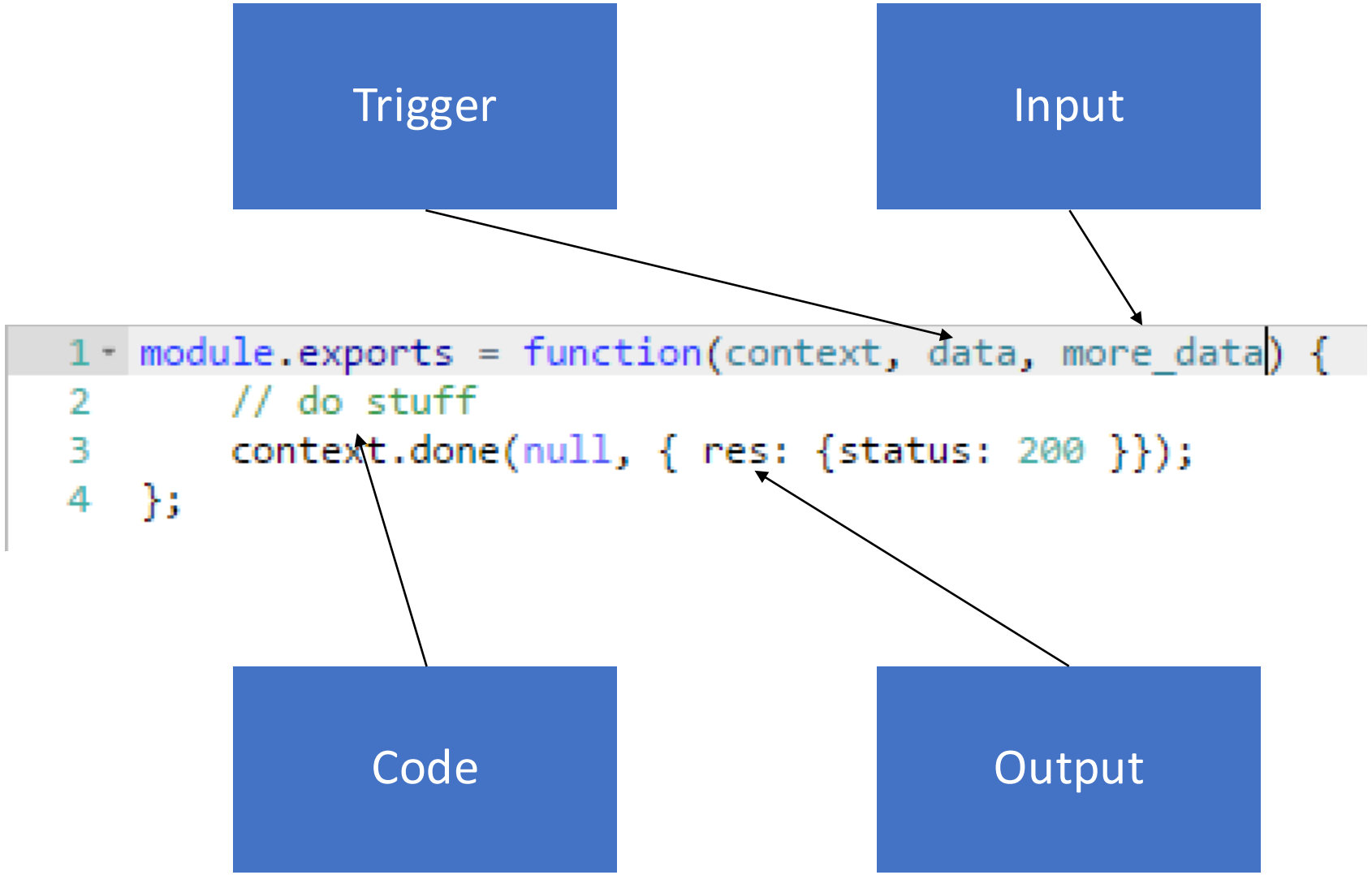
```
module.exports = async function (context, order) {
    order.PartitionKey = "Orders";
    order.RowKey = generateRandomId();
    context.bindings.order = order;
};
function generateRandomId() {
    return Math.random().toString(36).substring(2, 15) +
        Math.random().toString(36).substring(2, 15);
}
```

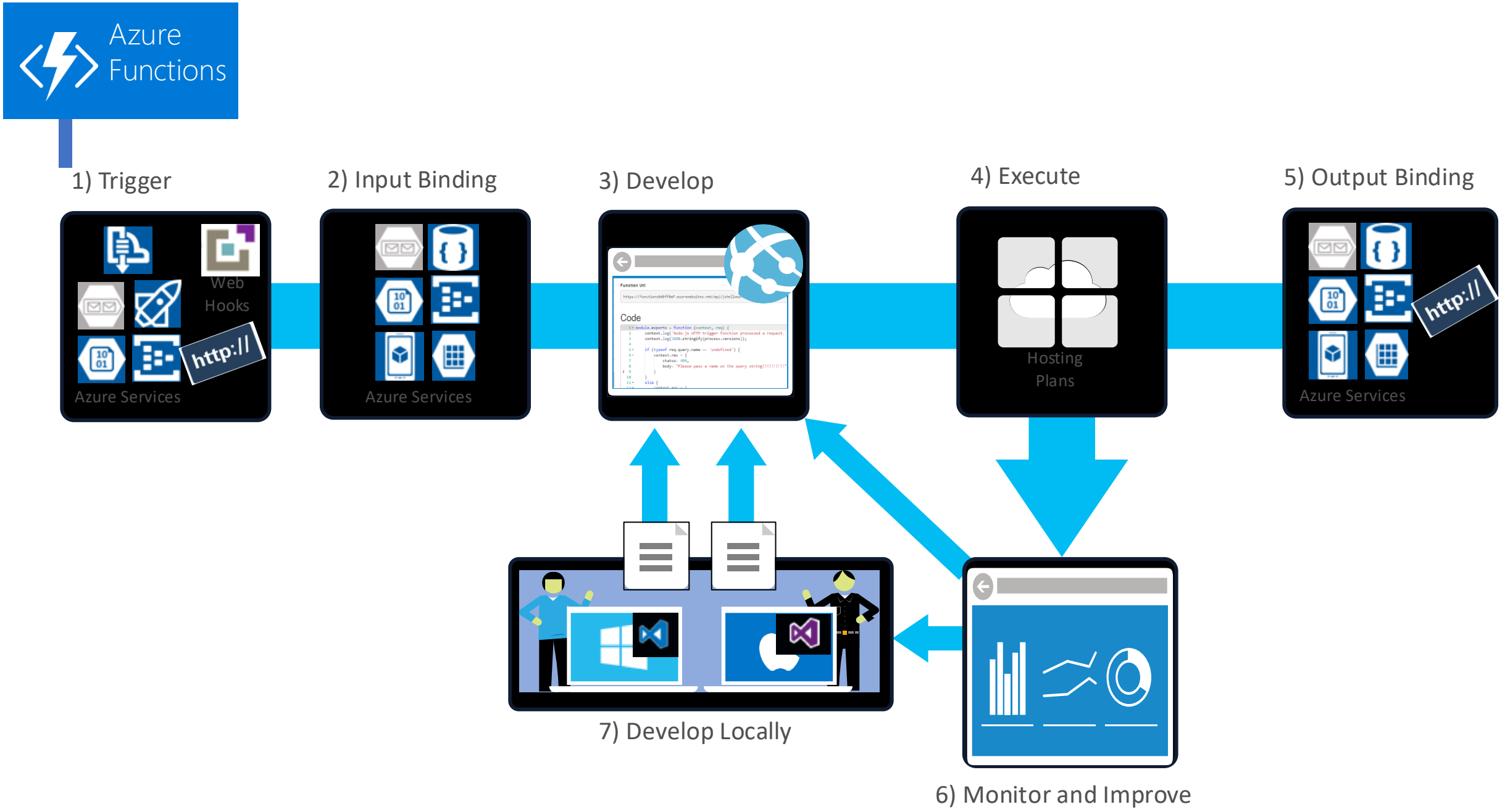
Create triggers and bindings

Class library example

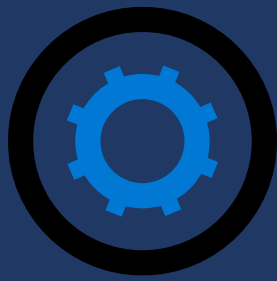
```
public static class QueueTriggerTableOutput
{
    [FunctionName("QueueTriggerTableOutput")]
    [return: Table("outTable", Connection = "CONNECTION")]
    public static Person Run(
        [QueueTrigger("myqueue-items", Connection = "CONNECTION")]JObject order,
        ILogger log)
    {
        return new Person() {
            PartitionKey = "Orders",
            RowKey = Guid.NewGuid().ToString(),
            Name = order["Name"].ToString(),
            MobileNumber = order["MobileNumber"].ToString() };
    }
...
```

Functions programming concepts




Azure Functions
1) Trigger
Web Hooks
http://
Azure Services
2) Input Binding
Azure Services
3) Develop
Code
4) Execute
Hosting Plans
5) Output Binding
http://
Azure Services
7) Develop Locally
6) Monitor and Improve

Implement Durable Functions



Explore Durable Functions app patterns

Overview

- The *durable functions* extension lets you define stateful workflows by writing *orchestrator functions* and stateful entities by writing *entity functions* using the Azure Functions programming model.
- Behind the scenes, the extension manages state, checkpoints, and restarts for you, allowing you to focus on your business logic.

Supported languages

Durable Functions currently supports the following languages:

- C#
- JavaScript
- Python
- F#
- PowerShell

Explore Durable Functions app patterns

Application patterns

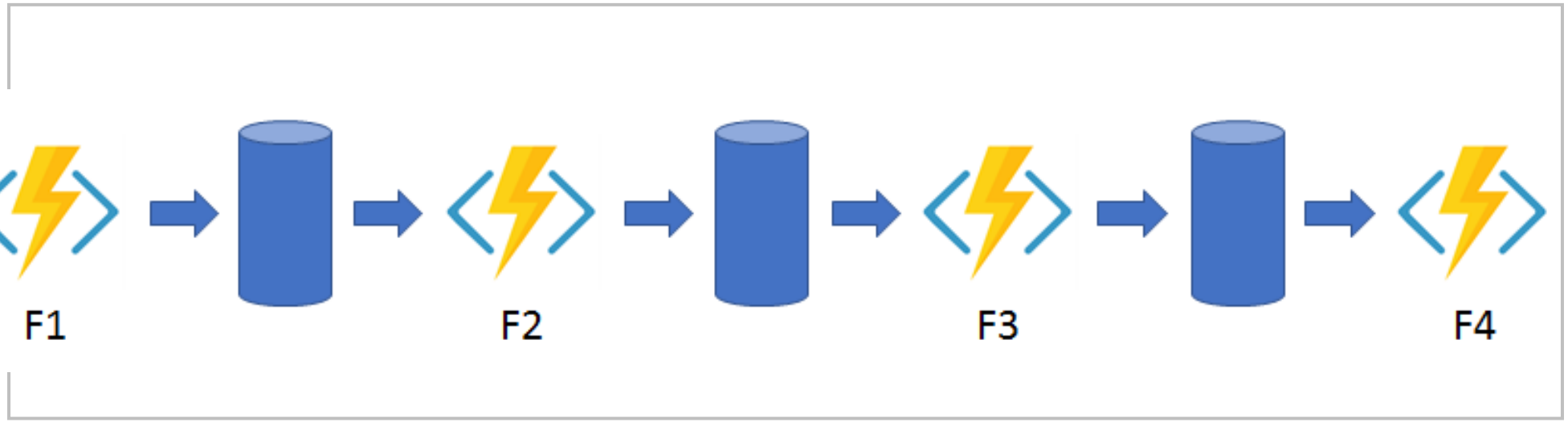
The primary use case for Durable Functions is simplifying complex, stateful coordination requirements in serverless applications.

The following sections describe typical application patterns that can benefit from Durable Functions:

- Function chaining
- Fan-out/fan-in
- Async HTTP APIs
- Monitor
- Human interaction

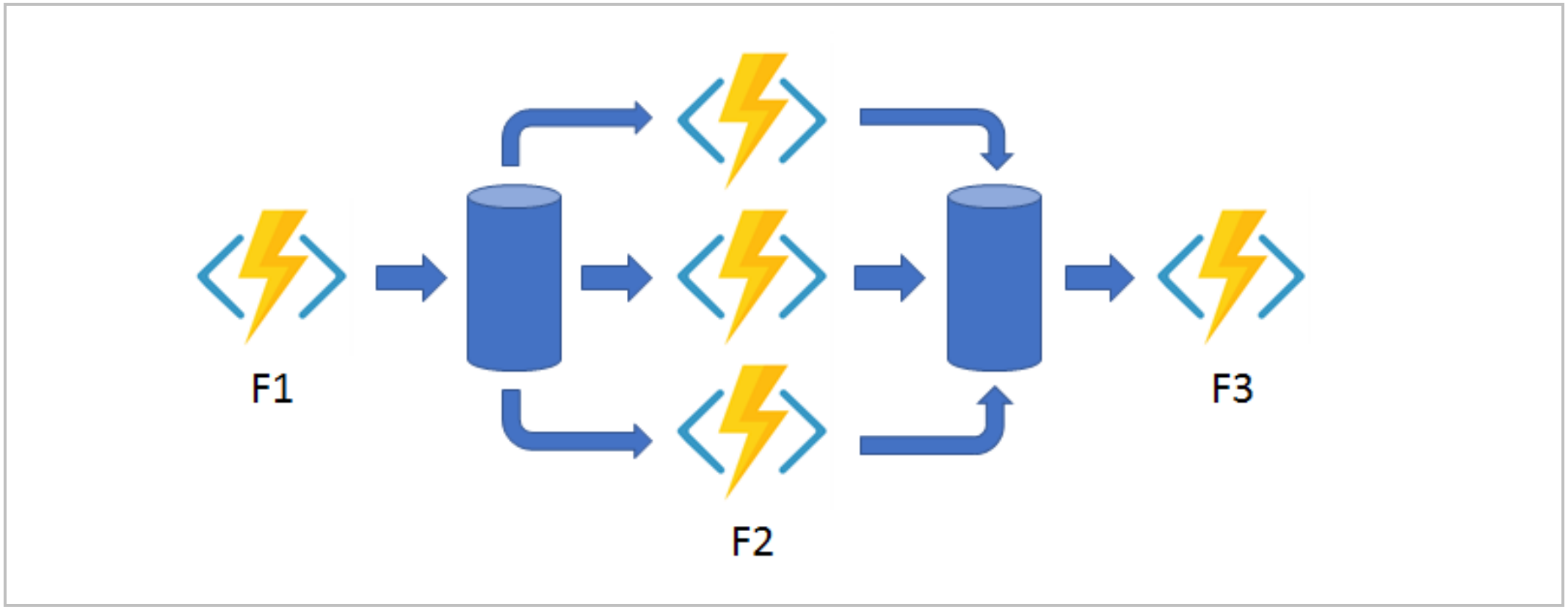
Explore Durable Functions app patterns

Function chaining



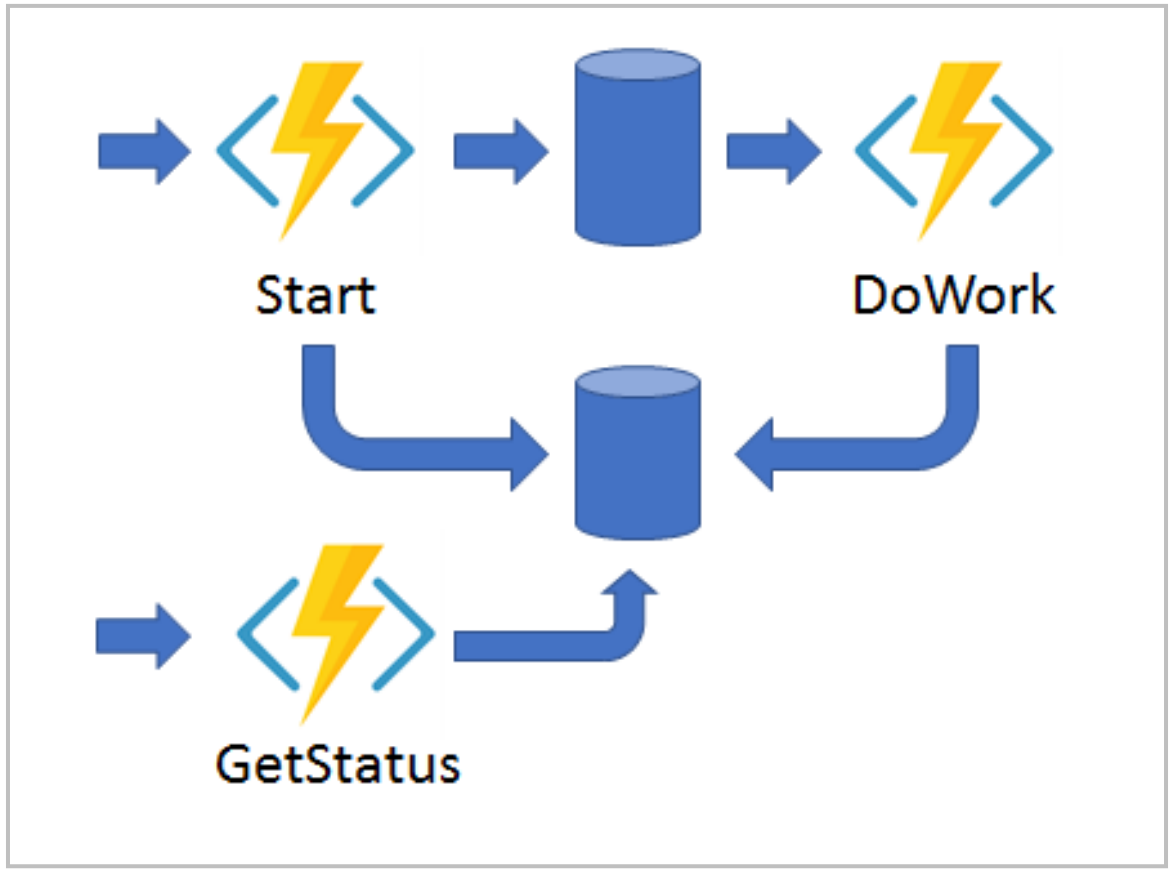
Explore Durable Functions app patterns

Fan out/fan in



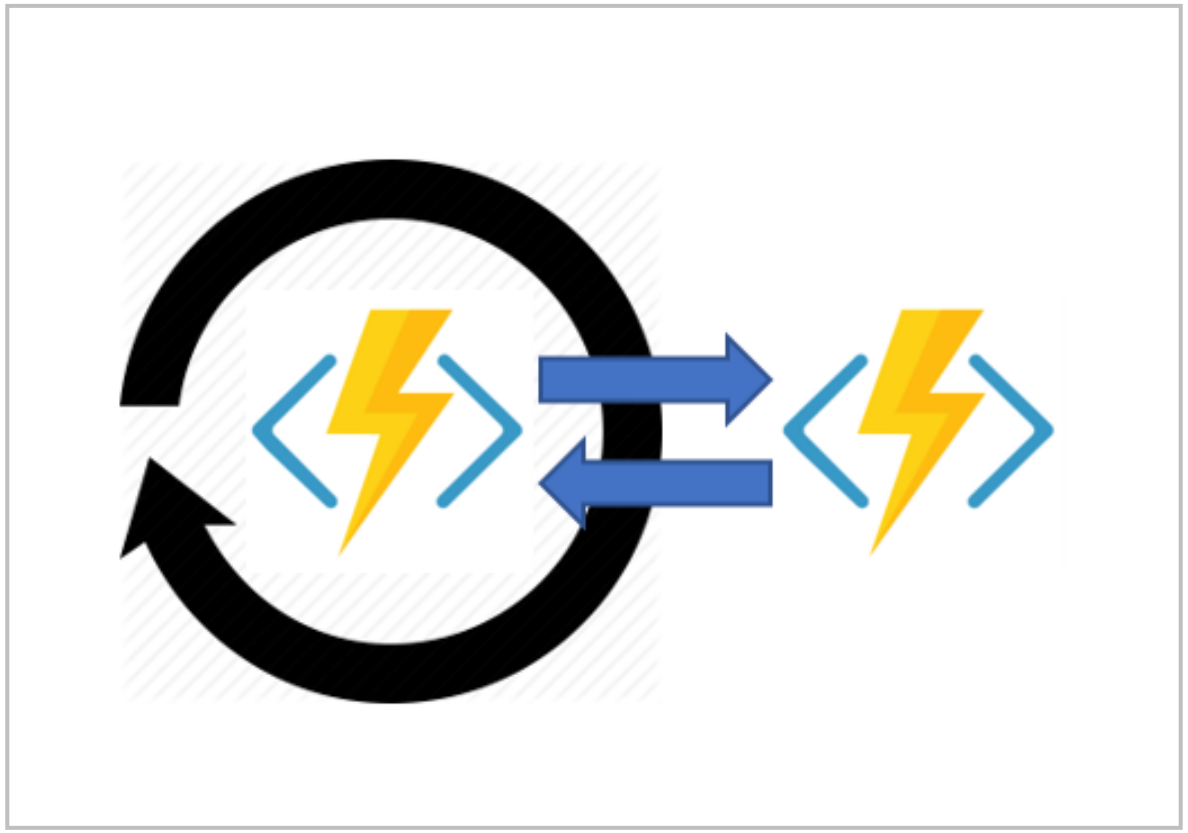
Explore Durable Functions app patterns

Async HTTP APIs



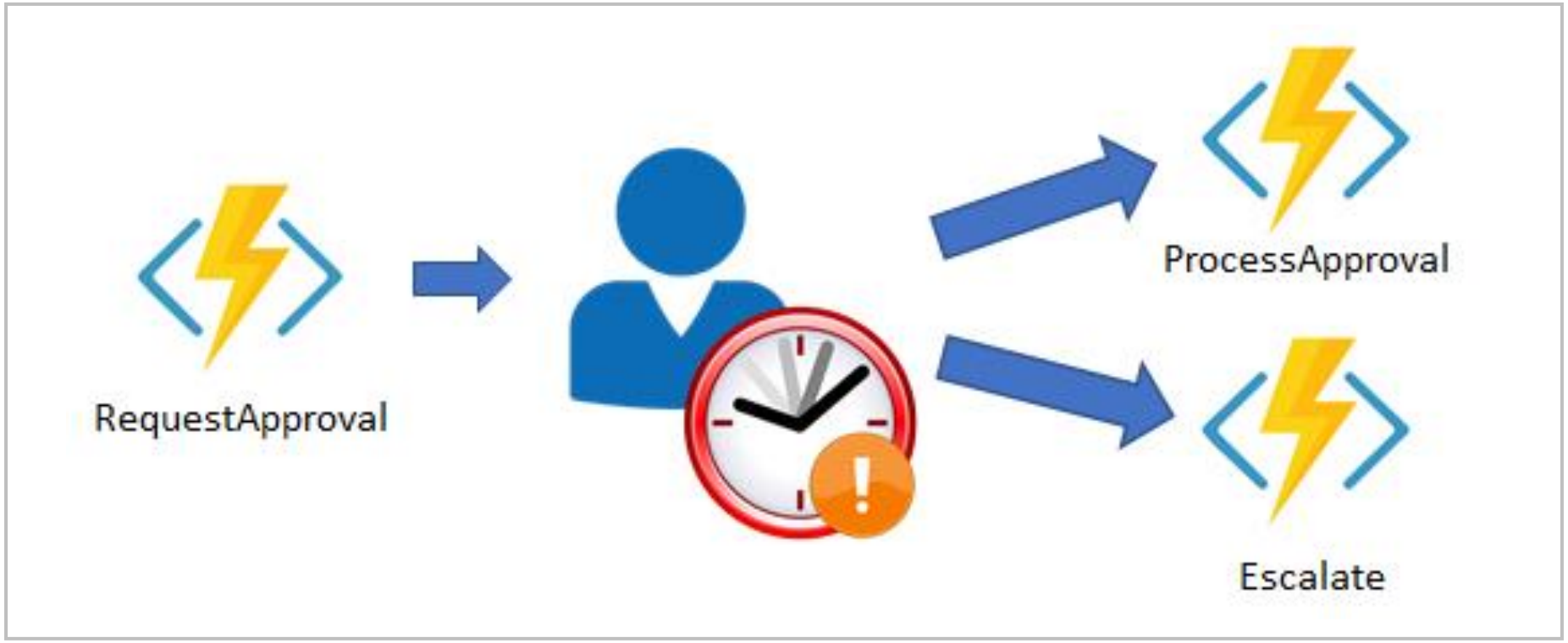
Explore Durable Functions app patterns

Monitor



Explore Durable Functions app patterns

Human interaction



Discover the four function types

Overview

There are currently four durable function types in Azure Functions:

- Orchestrator
- Activity
- Entity
- Client

Orchestrator functions

- Describe how actions are executed and the order in which actions are executed.
- Describe the orchestration in code (C# or JavaScript).
- Can have many different types of actions.
- Can also interact with entity functions.

Discover the four function types

Activity functions

- Activity functions are the basic unit of work in a durable function orchestration.
- For example, you might create an orchestrator function to process an order. The tasks involve checking the inventory, charging the customer, and creating a shipment.
- Each task would be a separate activity function.
- These activity functions may be executed serially, in parallel, or some combination of both.

Entity functions

Entity functions define operations for reading and updating small pieces of state. We often refer to these stateful entities as durable entities. Some things to note:

- Entities are accessed via a unique identifier, the entity ID. An entity ID is simply a pair of strings that uniquely identifies an entity instance.
- Operations on entities require that you specify the **Entity ID** of the target entity, and the **Operation name**, which is a string that specifies the operation to perform.

Discover the four function types

Client functions

- Orchestrator and entity functions are triggered by their bindings and both of these triggers work by reacting to messages that are enqueued in a task hub.
- The primary way to deliver these messages is by using an orchestrator client binding, or an entity client binding, from within a *client function*. Any non-orchestrator function can be a client function.
- Unlike other function types, orchestrator and entity functions cannot be triggered directly using the buttons in the Azure portal. If you want to test an orchestrator or entity function in the Azure portal, you must instead run a client function that starts an orchestrator or entity function as part of its implementation.

Explore task hubs

Overview

- A task hub in Durable Functions is a logical container for durable storage resources that are used for orchestrations and entities.
- Orchestrator, activity, and entity functions can only directly interact with each other when they belong to the same task hub.

Azure Storage resources

A task hub in Azure Storage consists of the following resources:

- One or more control queues.
- One work-item queue.
- One history table.
- One instances table.
- One storage container containing one or more lease blobs.
- A storage container containing large message payloads, if applicable.

Explore task hubs

Task hub names

Task hubs in Azure Storage are identified by a name that conforms to these rules:

- Contains only alphanumeric characters
- Starts with a letter
- Has a minimum length of 3 characters, maximum length of 45 characters

The task hub name is declared in the *host.json* file, as shown in the example.

```
{
  "version": "2.0",
  "extensions": {
    "durableTask": {
      "hubName": "MyTaskHub"
    }
  }
}
```

Explore durable orchestrations

Overview

Orchestrator functions characteristics:

- Define function workflows using procedural code.
- Can call other durable functions synchronously and asynchronously.
- Are durable and reliable.
- Can be long-running.

Orchestration identity

- Each *instance* of an orchestration has an instance identifier (also known as an *instance ID*).
- By default, each instance ID is an autogenerated GUID.
- However, instance IDs can also be any user-generated string value. Each orchestration instance ID must be unique within a task hub.

Explore durable orchestrations

Reliability

- Orchestrator functions reliably maintain their execution state by using the event sourcing design pattern.
- Durable Functions uses event sourcing transparently.
- When an orchestration function is given more work to do, the orchestrator wakes up and re-executes the entire function from the start to rebuild the local state.

Control timing in Durable Functions

Overview

- You can use durable timers in orchestrator functions to implement delays or to set up timeouts on async actions.
- Durable timers should be used in orchestrator functions instead of `Thread.Sleep` and `Task.Delay` (C#), or `setTimeout()` and `setInterval()` (JavaScript), or `time.sleep()` (Python).

Timer limitations

When you create a timer that expires at 4:30 pm, the underlying Durable Task Framework enqueues a message that becomes visible only at 4:30 pm.

When running in the Azure Functions Consumption plan, the newly visible timer message will ensure that the function app gets activated on an appropriate VM.

Control timing in Durable Functions

Usage for delay

```
[FunctionName("BillingIssuer")]
public static async Task Run(
    [OrchestrationTrigger] IDurableOrchestrationContext context)
{
    for (int i = 0; i < 10; i++)
    {
        DateTime deadline =
context.CurrentUtcDateTime.Add(TimeSpan.FromDays(1));
        await context.CreateTimer(deadline,
CancellationToken.None);
        await context.CallActivityAsync("SendBillingEvent");
    }
}
```

Control timing in Durable Functions

Usage for timeouts

```
public static async Task<bool> Run([OrchestrationTrigger]
IDurableOrchestrationContext context)
{
    TimeSpan timeout = TimeSpan.FromSeconds(30);
    DateTime deadline = context.CurrentUtcDateTime.Add(timeout);
    using (var cts = new CancellationTokenSource())
    {
        Task activityTask = context.CallActivityAsync("GetQuote");
        Task timeoutTask = context.CreateTimer(deadline, cts.Token);

        Task winner = await Task.WhenAny(activityTask, timeoutTask);
        if (winner == activityTask)
        {
            // success case and timeout case
        }
    }
}
```

Send and wait for events

Overview

Orchestrator functions have the ability to wait and listen for external events. This feature of Durable Functions is often useful for handling human interaction or other external triggers.

Wait for events

- The `WaitForExternalEvent` methods of the orchestration trigger binding allows an orchestrator function to asynchronously wait and listen for an external event

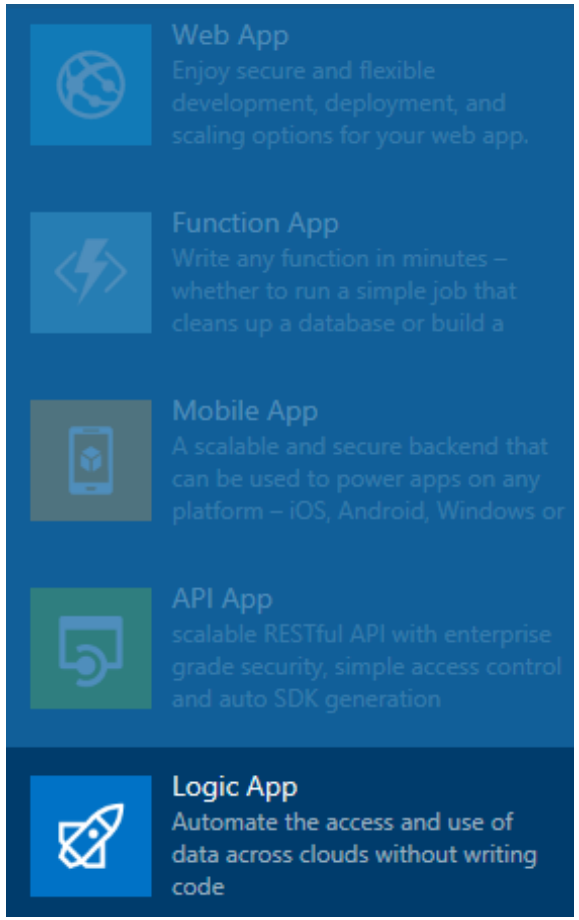
```
[FunctionName("BudgetApproval")]
public static async Task Run(
    [OrchestrationTrigger]
IDurableOrchestrationContext context)
{
    bool approved = await
context.WaitForExternalEvent<bool>("Approval")
;
    if (approved)
    {
        // approval     }
    else
    {
        // approval denied
    }
}
```

Send and wait for events

Send events

- The `RaiseEventAsync` method of the orchestration client binding sends the events that `WaitForExternalEvent` waits for.
- The `RaiseEventAsync` method takes *eventName* and *eventData* as parameters.
- The event data must be JSON-serializable.

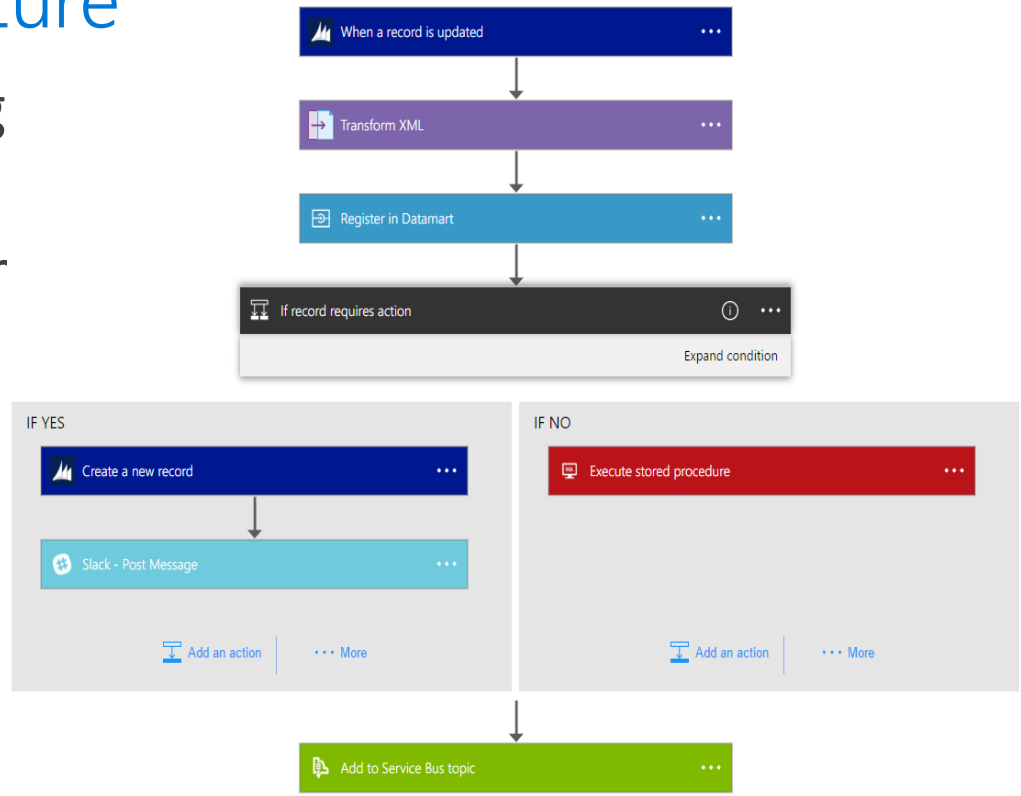
```
[FunctionName("ApprovalQueueProcessor")]
public static async Task Run(
    [QueueTrigger("approval-queue")] string instanceId,
    [DurableClient] IDurableOrchestrationClient client)
{
    await client.RaiseEventAsync(instanceId, "Approval",
true);
}
```


Web App
Enjoy secure and flexible development, deployment, and scaling options for your web app.
Function App
Write any function in minutes – whether to run a simple job that cleans up a database or build a
Mobile App
A scalable and secure backend that can be used to power apps on any platform – iOS, Android, Windows or
API App
scalable RESTful API with enterprise grade security, simple access control and auto SDK generation
Logic App
Automate the access and use of data across clouds without writing code

What can Logic Apps Do?

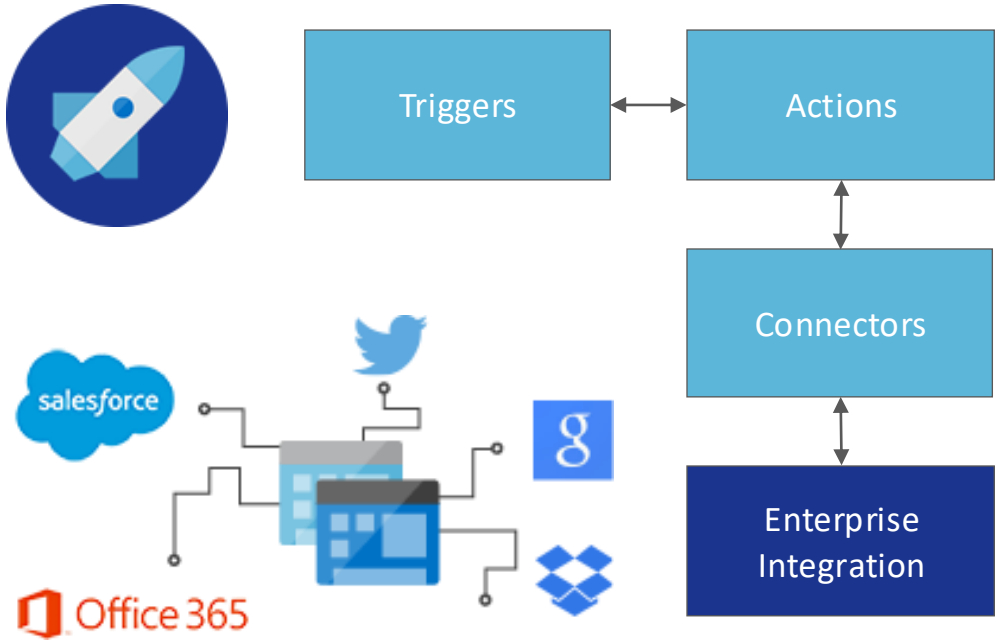
Hyperscale Workflow Engine, born in Azure

- Faster integration using innovative Visual Desig
- Easy workflow creation with triggers and actior
- Mashup applications, data and services
- Built for mission critical Enterprise Integration
- Create, deploy, manage and monitor



Logic Apps

- Workflow
- Managed Connectors
- Triggers
- Actions
- Enterprise Integration



Container based apps

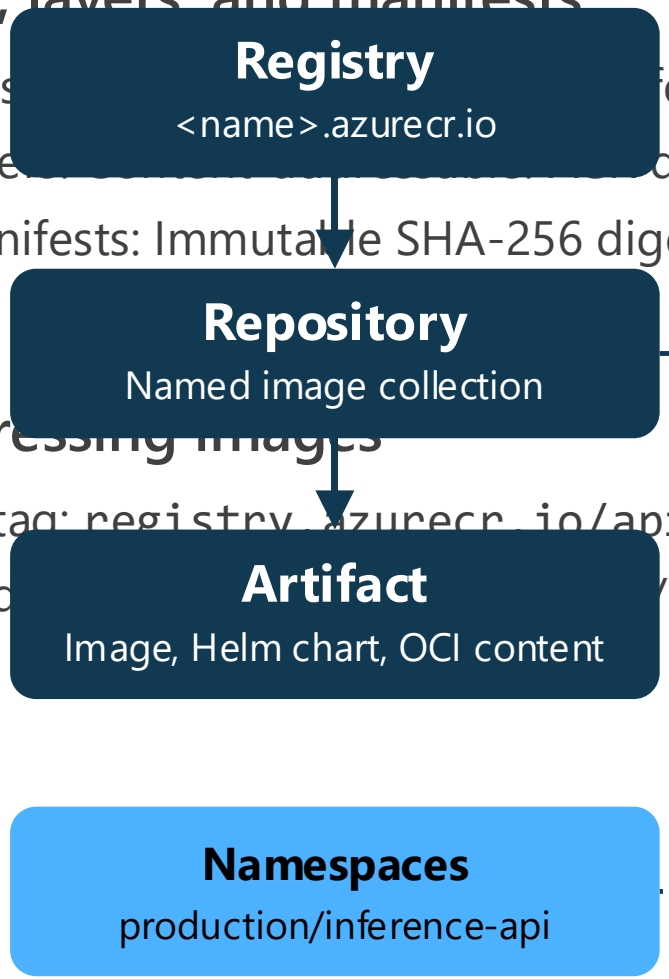
Azure Container registries

- Managed, private container registry service on Azure
- Stores and distributes container images and OCI artifacts
- Direct integration with Azure Kubernetes Service, Container Apps, and App Service
- Supports Docker images, Helm charts, and OCI-compliant content
- Basic, Standard, and Premium service tiers
- Geo-replication for global distribution (Premium tier)

Registries, repositories, and artifacts

Tags, layers, and manifests

- Tags [illegible] references (`repo:tag`). Multiple tags can point to the same artifact.
- Laye[illegible] deduplicates shared layers across images.
- Manifests: Immutable SHA-256 digest (`sha256:abc...`). Pull by digest for guaranteed reproducibility.



Addressing images

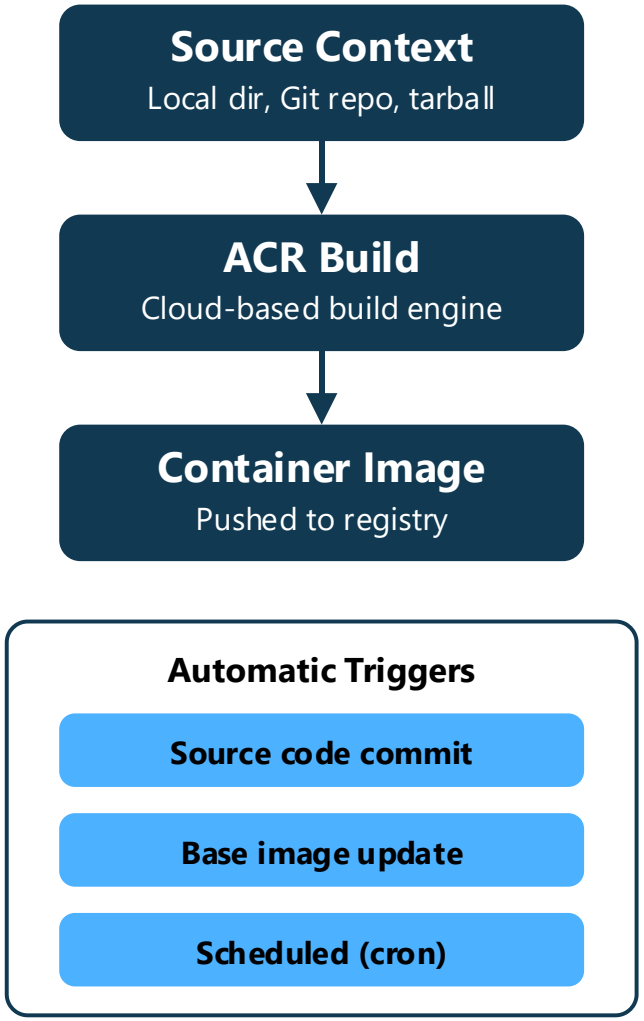
- By tag: `registry.azurecr.io/api:v1.2.0`
- By d[illegible]`/api@sha256:0a2e...`

Build and run images with ACR Tasks

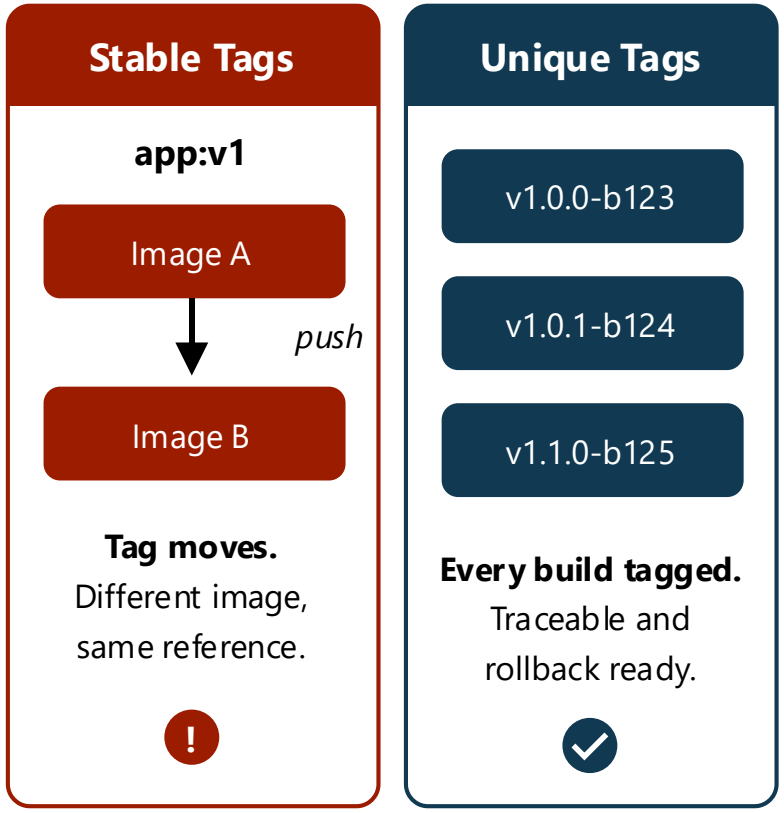
- Quick tasks build and push in one command
- Source code trigger example

```
az acr task create \
  --registry myregistry \
  --name build-inference-api \
  --image inference-api:{{.Run.ID}} \
  --context https://github.com/myorg/
    inference-api.git#main \
  --file Dockerfile \
  --git-access-token $PAT
```

- Multi-step tasks define build-test-push workflows in YAML



Tag and version images



- Lock production images: Prevents deletion, overwrite, and survives retention policies

```
az acr repository update \
  --name myregistry \
  --image inference-api:v1.2.0 \
  --write-enabled false
```

- Clean up untagged images

```
az acr task create \
  --registry myregistry \
  --name cleanup-untagged \
  --cmd "acr purge --filter '.*:.*' \
  --untagged --ago 7d" \
  --schedule "0 0 * * 0" \
  --context /dev/null
```

- Unique tag patterns: build ID, Git SHA, timestamp, or combined

Containers in Azure App Service

- Fully managed platform for custom Linux containers
- Bring your own image from any Docker V2 registry
- Built-in autoscaling, deployment slots, and TLS termination
- Environment configuration via app settings injected as env vars
- Integrated diagnostics: log stream, Kudu console, Azure Monitor
- No infrastructure management. Focus on application code.

Deploy containers to Azure App Service

Image sources

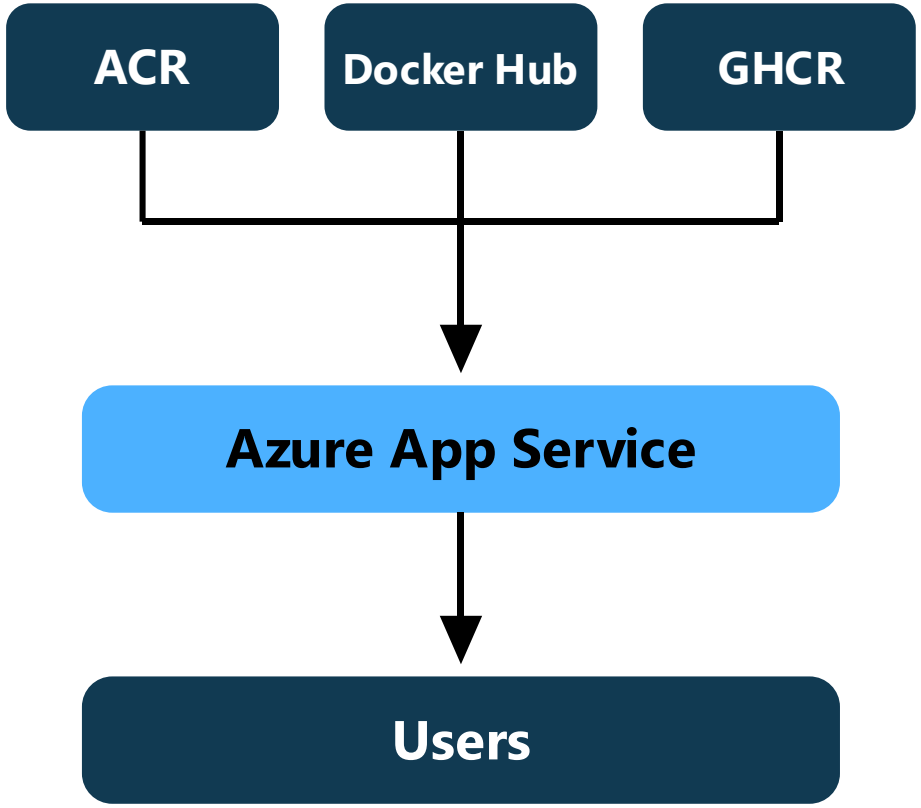
- Azure Container Registry (recommended for production)
- Docker Hub, GitHub Container Registry, self-hosted registries

ACR authentication

- Managed identity (system or user-assigned) with AcrPull role
- Admin credentials for dev/test scenarios

Deployment methods

- Azure portal guided experience
- Azure CLI, VS Code with Docker and App Service extensions
- Continuous deployment via ACR webhook



Configure container runtime behavior

Startup command

- Overrides Dockerfile CMD (ENTRYPOINT unchanged)
- Use for runtime args, migrations, custom configs

Port configuration

- Set `WEBSITES_PORT` if container listens on a port other than 80 or 8080
- App Service handles TLS termination. Container receives HTTP.

Persistent storage

- Survives restarts, shared across scaled instances

Always-on and health checks

- Always-on prevents ~20 min idle unloading (Basic tier+)
- Health check path pinged every minute. Failed instances removed from rotation.



Configure application settings

App settings

- Injected as environment variables at container start
- All settings encrypted at rest
- Access via standard env var methods (`os.environ` in Python)

Connection strings

- Type-prefixed env vars (e.g., `SQLAZURECONNSTR_`)
- Use app settings instead for non-.NET apps

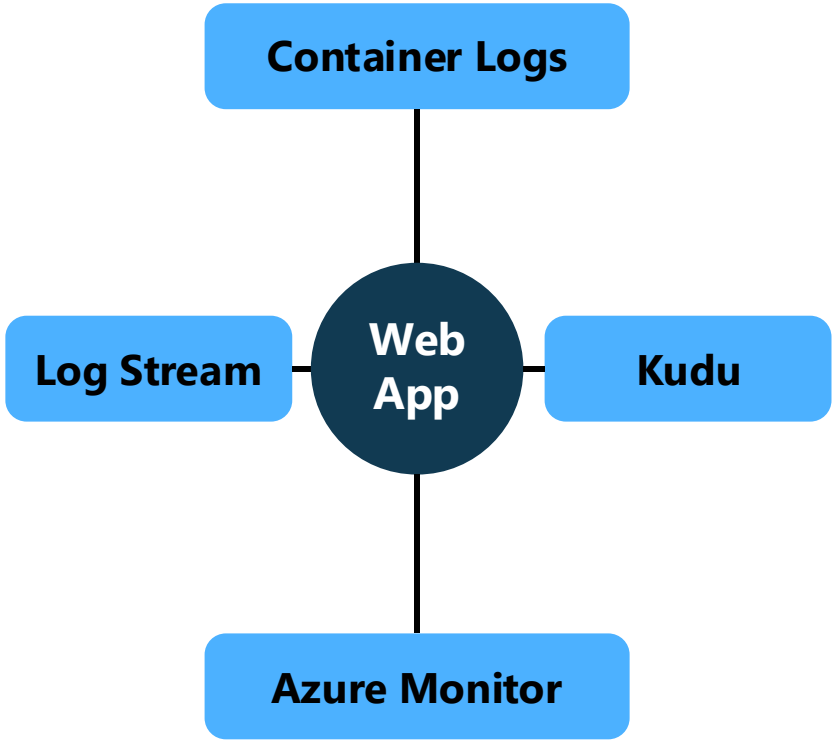
Slot settings

- Stick to the slot during swap operations
- Use for environment identifiers, endpoints, feature flags

Key Vault references

- `@Microsoft.KeyVault(SecretUri=...)` syntax
- Resolved automatically. App reads plain env vars.
- Requires managed identity with Key Vault access

Observe and troubleshoot containerized apps



Container logs

- Captures stdout, stderr, platform lifecycle events

Log stream

- Real-time output via CLI (az webapp log tail) or portal
- Shows all instances in scaled-out apps

Kudu (SCM site)

- Environment variable verification
- File system browser for /home/LogFiles/
- Diagnostic dump download

Azure Monitor integration

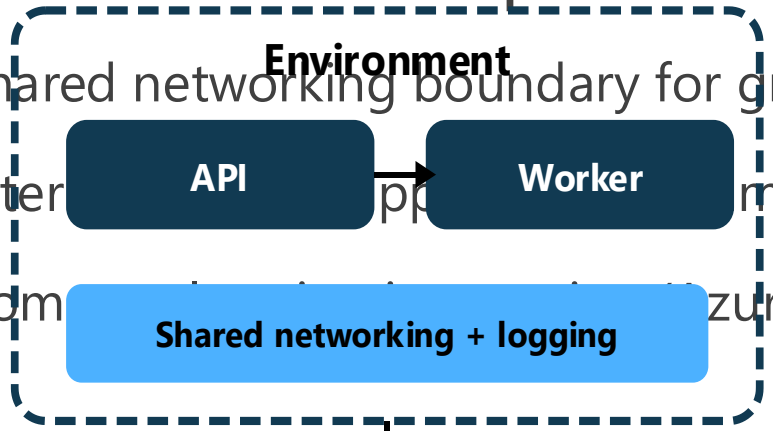
- Send logs to Log Analytics for KQL queries and alerting
- Categories: ConsoleLogs, HTTPLogs, PlatformLogs

Introduction to Azure Container Apps

- Serverless container platform on Azure
- No Kubernetes control plane to manage
- Revision-based deployment model for safe rollouts
- Built-in ingress with external and internal options
- Native secrets and environment variable support
- Integrates with ACR, Azure Monitor, VNET, managed identities

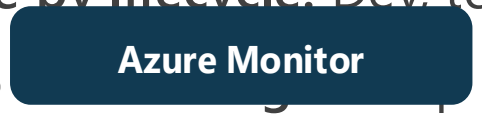
Explore Container Apps environments

What an environment provides



- Shared networking boundary for grouped apps
- Inter pp munication
- Com zure Monitor)

Design decisions



- **Separate by lifecycle:** Dev, test, prod environments
- **Align to** apps that communicate privately
- **Plan observability early:** Choose log destination before production

Deploy a container app using the Azure CLI and YAML

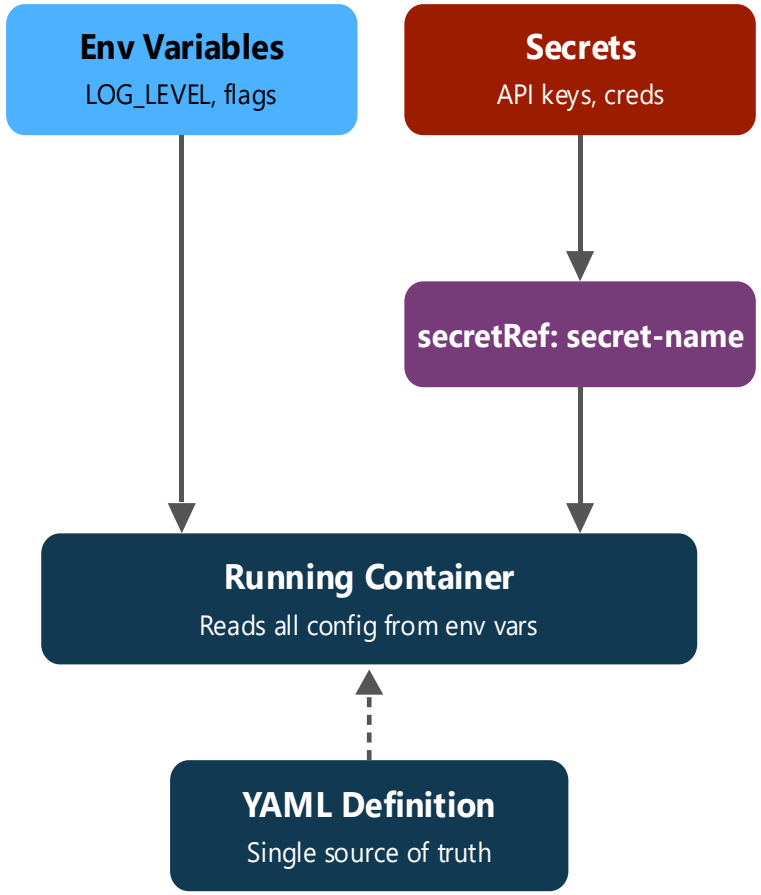
Three deployment paths

- **`containerapp up:`** Fastest path, creates environment automatically
- **`containerapp create:`** Explicit control, existing environment
- **`--yaml:`** Source-controlled config, reviewable via code review

Revisions

- Each update creates a new revision (versioned snapshot)
- Validate new revision before shifting traffic
- Use `containerapp update` for image and config changes

Configure runtime settings with environment variables and secrets



Environment variables (non-sensitive)

- Log levels, feature flags, dependency URLs
- Same image runs in dev, staging, prod with different values
- Set at create time or add incrementally later

Secrets (sensitive values)

- API keys, credentials, signing keys stored separately
- Rotate secrets without rebuilding or redeploying images
- Key Vault references supported for centralized management

Secret references

- Map secrets to env vars so app code reads them transparently
- Keep secret values out of YAML files and shell history

Configure image pull authentication for private registries

Username / password

- Works with any registry (ACR, Docker Hub, GHCR)
- Quick to set up for validation and prototyping
- More secrets to manage and rotate over time

Managed identity (recommended for ACR)

- No credentials to store or rotate
- Grant only AcrPull for least privilege
- Best fit for production AI services pulling many images
- Use `--identity system` for ACR

Key consideration: Registry config is separate from app config. Image pull failures often look like auth errors.

Verify deployments with logs and status

Logs (fastest signal)

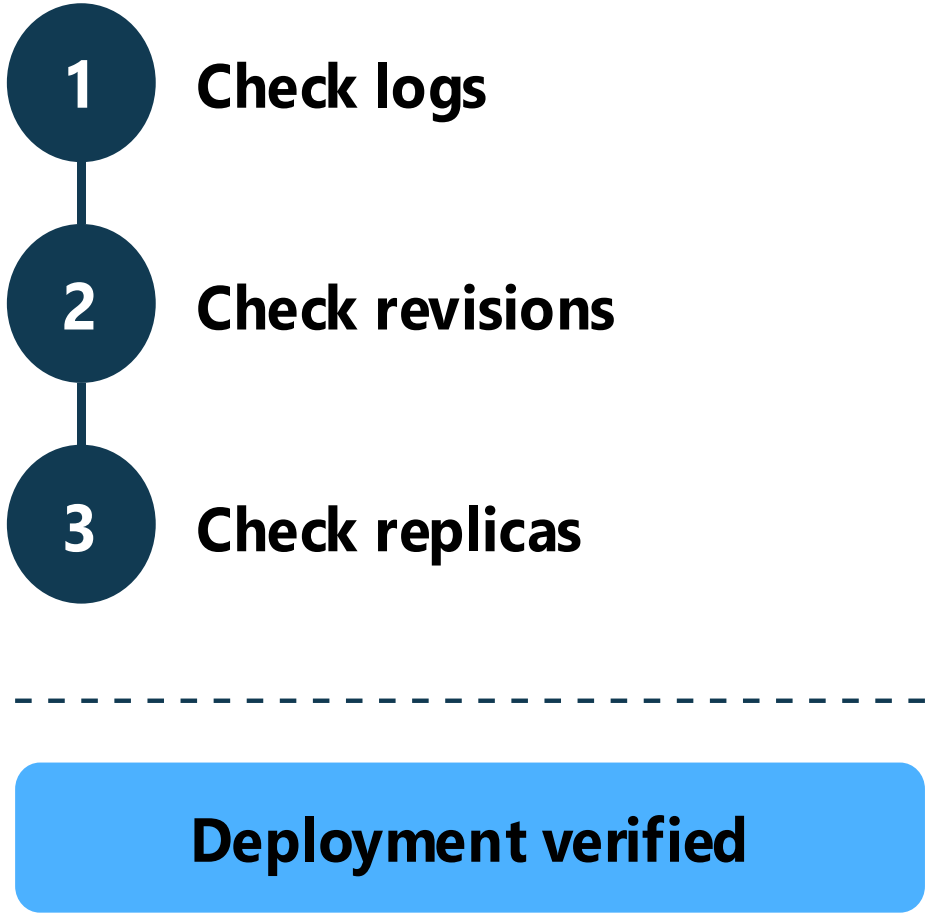
- Console logs surface startup errors, missing env vars, connection failures
- System logs reveal platform-level issues (image pull, scaling)
- Stream logs in real time during active debugging

Revisions (version check)

- Confirm a new revision exists after each update
- Verify the revision reached a healthy, active state
- Include inactive revisions for deeper investigation

Replicas (scaling check)

- Detect scale-to-zero, crash loops, and cold start issues
- Verify expected instance count under load



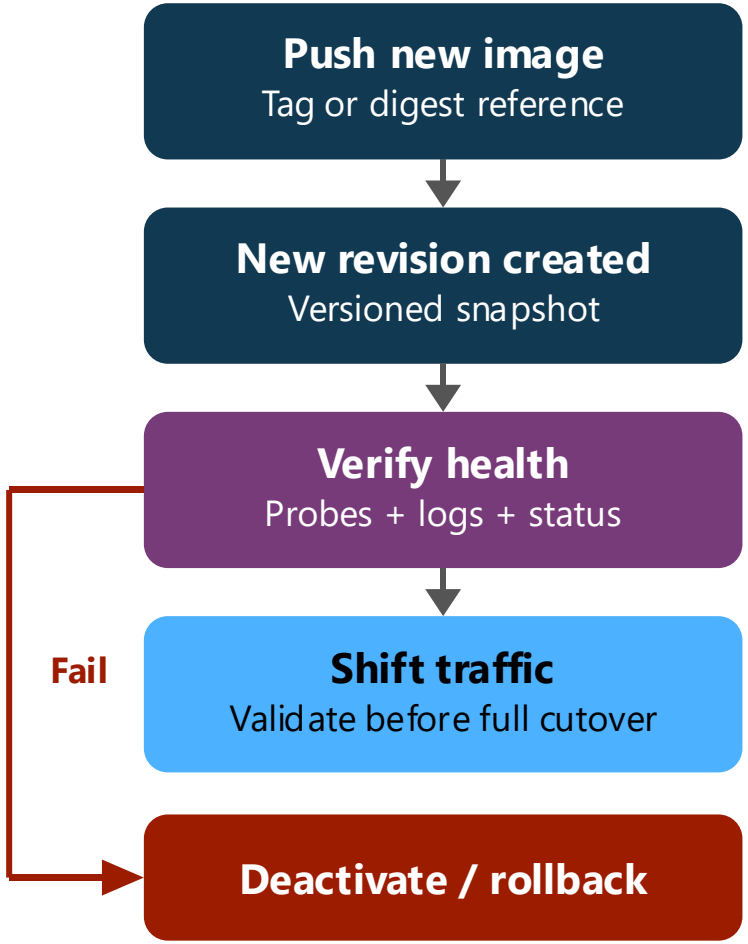
Update images and manage revisions safely

Image identity

- Tags are convenient but mutable. Digests are immutable.
- Use digests in production for traceability during incidents
- Correlate operational behavior with a specific build artifact

Revision workflow

- Each image update creates a new revision
- Verify health before shifting traffic
- Deactivate problematic revisions. Preserve for investigation.
- Define a retention strategy for inactive revisions



Manage the container app lifecycle

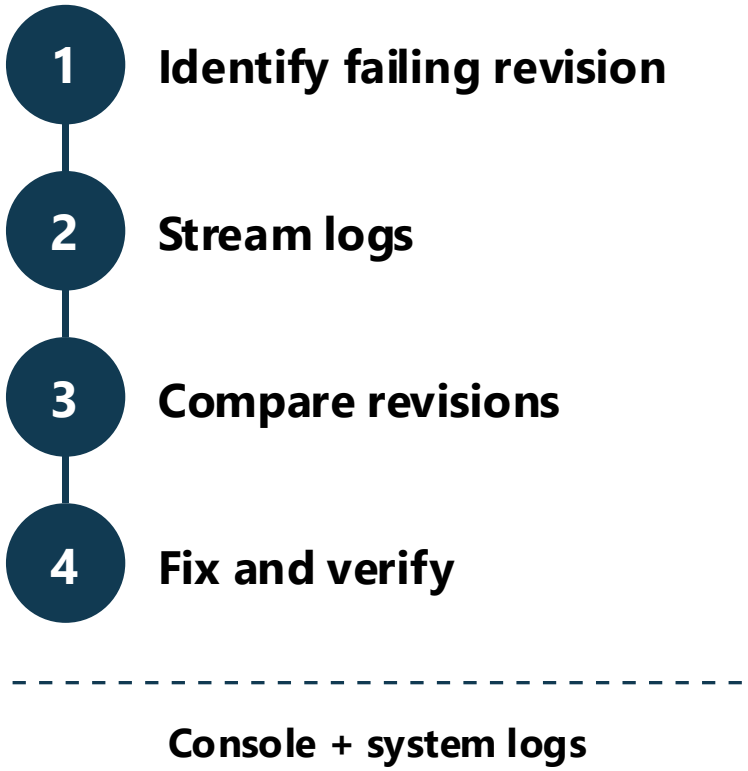
Choose the right action

- **Stop:** Guarantees zero traffic. Broad scope. Use during incidents.
- **Restart:** Clears transient state. Use for deadlocks, stuck processes.
- **Deactivate revision:** Isolates one bad release. Safest first step.

Common failure patterns to validate

- Image pull failures (registry credentials)
- Port mismatches (container vs. ingress config)
- Missing environment variables or secrets
- Probe failures (wrong path, short timeout)
- Resource pressure (CPU throttling, OOM restarts)

Monitor logs and troubleshoot issues



Log types

- Console logs: Application output (errors, startup messages)
- System logs: Platform events (image pull, scaling, probes)

Repeatable troubleshooting workflow

- Identify which revision is failing
- Stream logs while reproducing the issue
- Compare working vs. failing revision config
- Apply targeted fix, verify next revision is healthy

Structured logging tips

- Include request ID, revision ID, model version
- Log latency breakdowns for dependency calls
- Avoid logging sensitive content (prompts, documents)

Configure health probes and troubleshoot failures

Readiness vs. liveness

- Readiness: Guards model loading and warmup.
- Liveness: Catches deadlocks. Triggers restart.

Tuning for AI services

- Set `initialDelaySeconds` to cover model load time
- Use generous `failureThreshold` for slow warmup
- Don’t let liveness fire during startup
- Probe endpoints should not depend on external services

When probes fail immediately after deploy

- Validate port and path match the container’s HTTP server
- Check that timeout allows enough time for response

Optimize container resources and scaling

Per-replica sizing

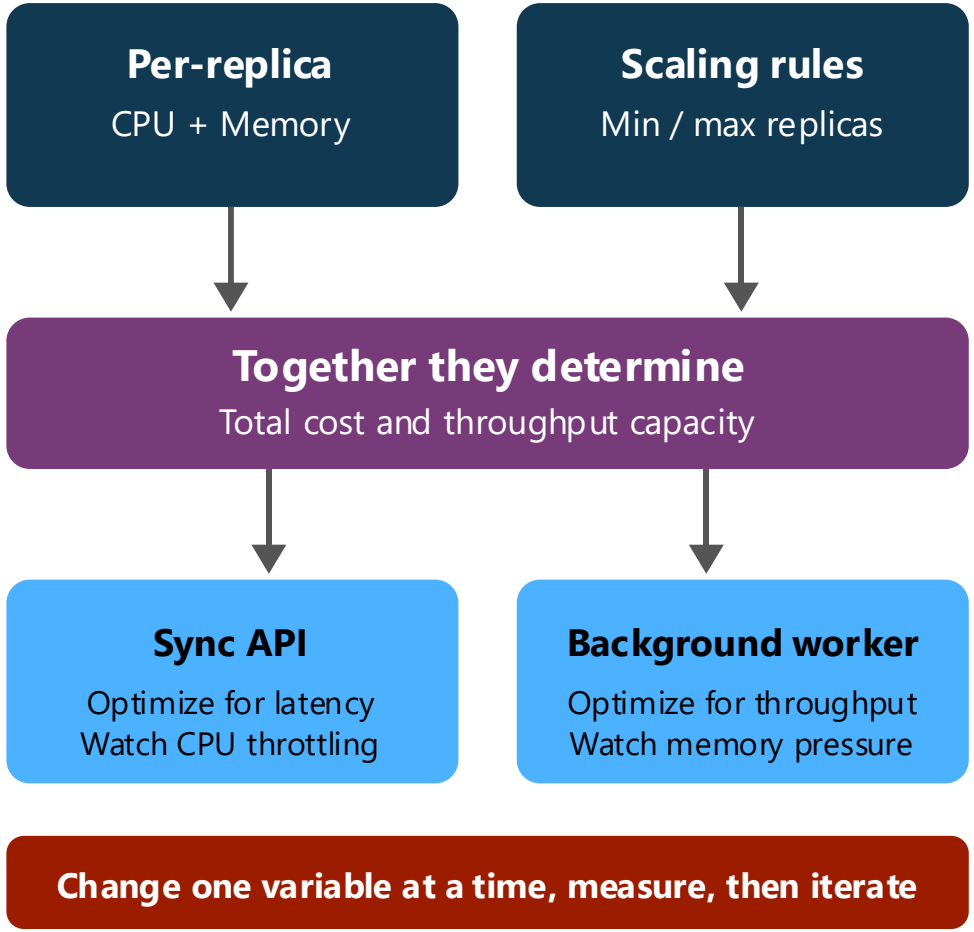
- CPU throttling = need more CPU per replica
- OOM restarts = need more memory per replica
- Total cost = per-replica size x replica count

Scaling strategies differ by workload

- **Sync API:** Minimum replicas to reduce cold starts. Optimize for latency.
- **Background worker:** Scale-to-zero if event-driven. Optimize for throughput.

Optimization approach

- Measure before changing (logs + metrics)
- Change one variable at a time
- Reassess after model version changes



Scaling

- Automatic horizontal scaling based on demand
- Powered by KEDA (Kubernetes Event-Driven Autoscaling)
- Scale to zero for cost savings during idle periods
- Built-in HTTP, TCP, CPU, and memory triggers
- Event-driven scaling for queues, streams, and custom metrics
- Traffic splitting across revisions for safe rollouts

Configure scale rules

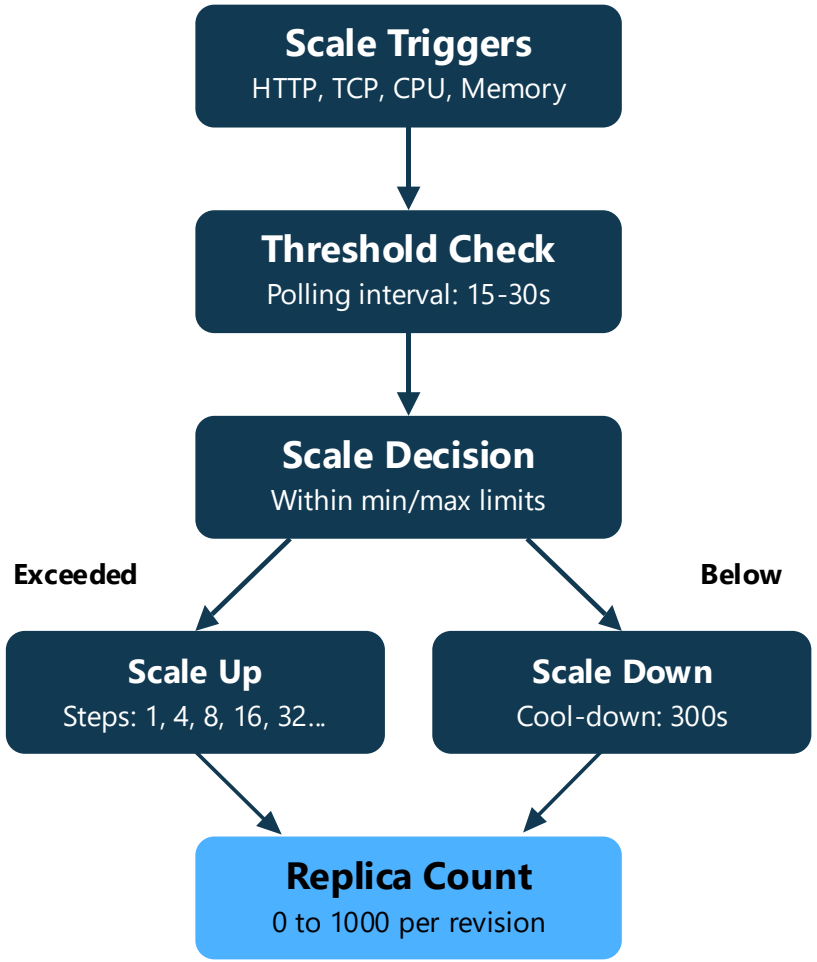
Scale definition components

- Limits: Min and max replica count (0 to 1000)
- Rules: Triggers that drive scaling decisions
- Behavior: Timing for polling, cool-down, stabilization

Built-in rule types

- HTTP: Concurrent requests per replica (default: 10)
- TCP: Concurrent connections for persistent protocols
- CPU: Utilization percentage (cannot scale to zero)
- Memory: Consumption percentage (cannot scale to zero)

Key behavior: Multiple rules use highest replica count



Implement event-driven scaling with KEDA

Azure-native scalers (first-party)

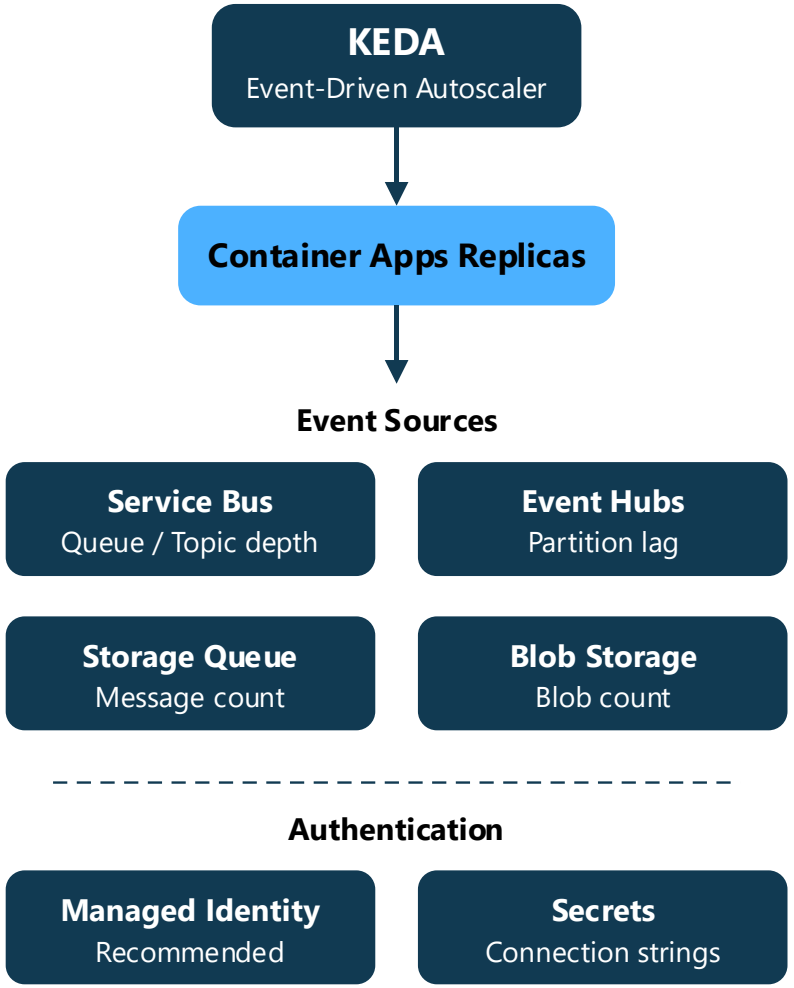
- Service Bus: Scale on queue/topic message count
- Event Hubs: Scale on partition consumer lag
- Storage Queue: Scale on approximate message count
- Blob Storage: Scale on blob count

How it works

- KEDA polls event source every 30 seconds
- Divides current metric by threshold to get replica count
- Supports scale-to-zero for idle queue processors

Authentication

- Managed identity (recommended for production)
- Secrets-based connection strings



Apply KEDA scalers for custom workloads

Beyond Azure-native scalers

- Kafka: Consumer group lag per partition
- Redis: List length or stream pending entries
- Cron: Time-based pre-scaling for known peaks
- Prometheus: Custom PromQL business metrics

Combining scalers

- Cron establishes baseline capacity on schedule
- HTTP or event scalers handle real-time demand
- Platform uses highest replica count across all rules

Migrating from Kubernetes KEDA

- Map ScaledObject type to Container Apps rule type
- Extract metadata and auth parameters
- Replace TriggerAuthentication with secrets or identity

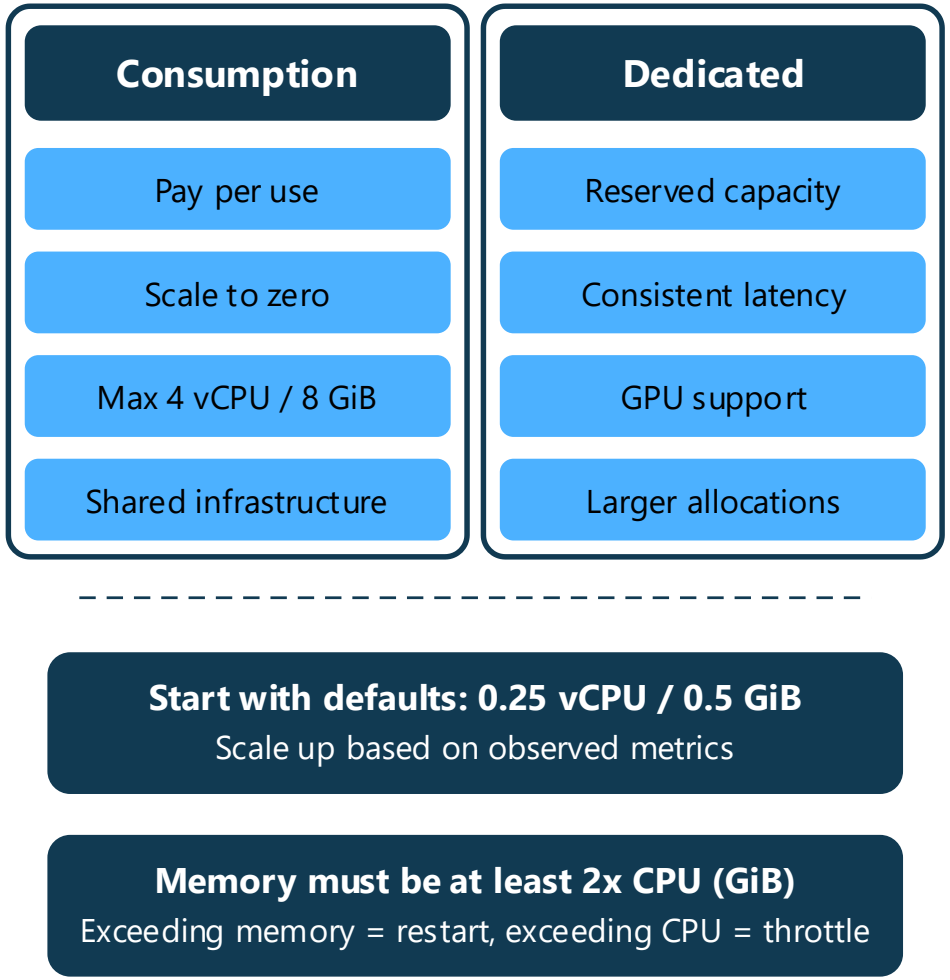
Select compute resources for performance and cost

Resource allocation

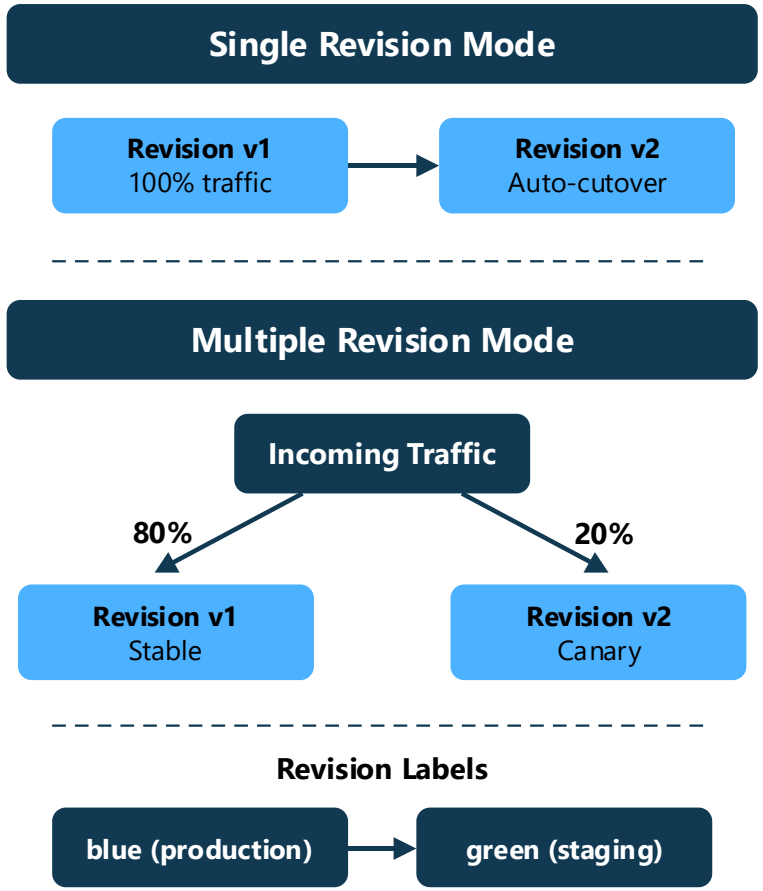
- CPU in cores, memory in GiB per container
- Memory must be at least 2x CPU value
- Exceeding memory limit restarts the replica
- Exceeding CPU limit throttles (no restart)

Cost levers

- Scale-to-zero eliminates idle compute charges
- Right-size based on actual utilization, not estimates
- Total capacity = per-replica resources x max replicas
- Smaller replicas scale more precisely but more frequently



Choose and apply revision modes



Single revision mode (default)

- One active revision at a time
- Zero-downtime deployment is automatic
- New revision replaces old after health checks pass
- No traffic splitting capability

Multiple revision mode

- Multiple revisions active simultaneously
- Traffic splitting by percentage weights
- Canary releases, blue-green deployments, A/B testing
- Must deactivate old revisions manually

Revision labels

- Named endpoints for direct revision access
- Test specific revisions without affecting production traffic
- Enable blue-green switching via label moves

Introduction to Azure Kubernetes Service

- Managed Kubernetes control plane on Azure
- Runs containerized APIs, workers, and AI backends at scale
- Deployments maintain replica count and restart failed Pods
- Services provide stable connectivity despite Pod churn
- Azure load balancers expose public endpoints when needed
- kubectl is the primary tool for deploy, inspect, and repair operations

Create Kubernetes deployment manifests

Desired state in YAML

- Define image, replica count, labels, and container port
- Keep Pod labels aligned with selector labels
- Treat the manifest as the source of truth for runtime intent

Runtime sizing

- Requests reserve schedulable CPU and memory
- Limits cap runaway usage and prevent noisy neighbors
- Replica count trades cost for availability and throughput

Configuration flow

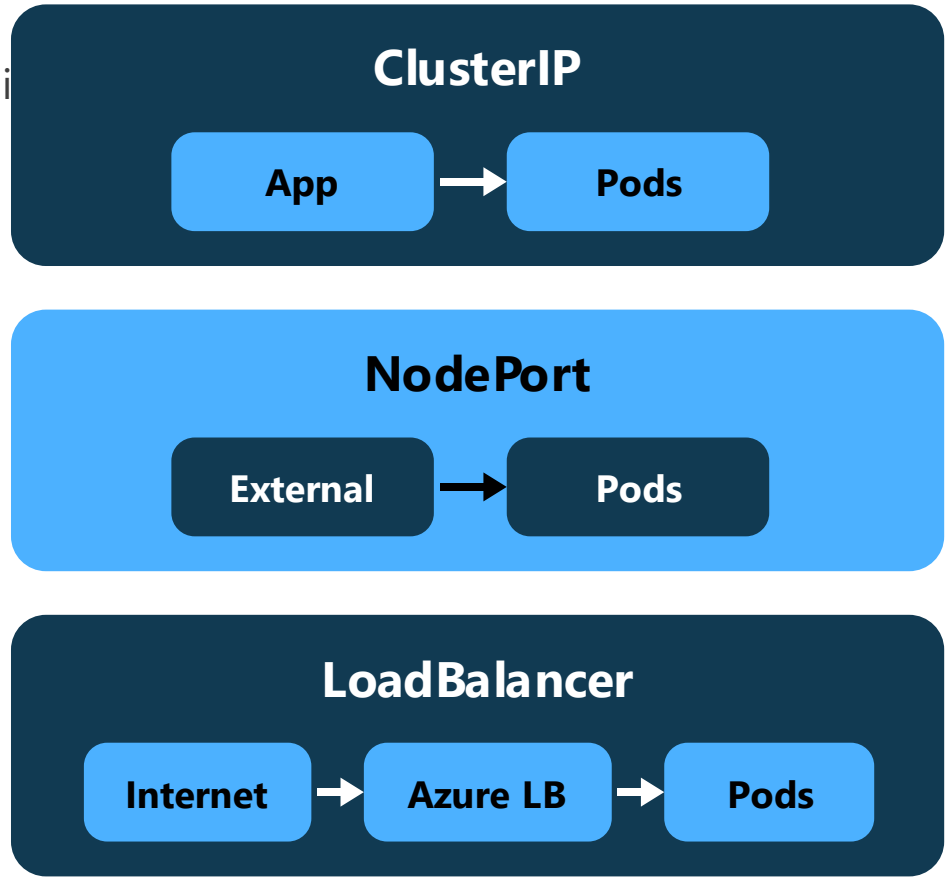
- Use env vars for non-sensitive settings
- Reference Secrets for keys and credentials
- Avoid embedding secret values in the manifest

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ai-api    # must match
  template:
    metadata:
      labels:
        app: ai-api  # must match
    spec:
      containers:
      - name: ai-api
        image: myacr.azurecr.io/ai-api:v1
        # ports, resources, env, secrets
```

Expose applications in Azure Kubernetes Services

- **ClusterIP:** Internal-only endpoint for app-to-app traffic
  - Best fit for back-end services inside the cluster
- **NodePort:** Opens a port on every node for basic external reachabili
  - Useful for development and simple validation
- **LoadBalancer:** Provisions an Azure-managed public entry point
  - Best fit for production APIs serving external clients

- Service selector must match Pod labels
- `port` is the client-facing port
- `targetPort` is the container-facing port



Deploy applications to Azure Kubernetes Services

Deployment flow

- Apply manifests and let the cluster reconcile state
- Wait for Pods to reach Running and replicas to become Ready
- Confirm the Service has endpoints and an address

Verification order

- Start with Pod and Deployment status
- Check Service exposure and endpoint assignment
- Read logs before changing configuration

Common failure signals

- ImagePullBackOff: Registry path or pull access issue
- CrashLoopBackOff: App starts, then exits or fails health checks
- Pending: Resource requests exceed available node capacity
- No endpoints: Service selector does not match Pod labels

Azure Integrations

- Externalize settings so containers stay portable across environments
- ConfigMaps store feature flags, endpoints, and nonsensitive values
- Secrets protect API keys, connection strings, and credentials
- PVCs provide durable storage that survives Pod restarts
- Azure integrations add Key Vault, App Configuration, and managed disks
- kubectl verifies that Pods receive the correct configuration

Define ConfigMaps for application settings

Key-value configuration

- Nonsensitive settings stored as string pairs, up to 1 MiB
- Update without rebuilding containers

Consumption patterns

- Environment variables via configMapKeyRef
- Volume mounts for file-based config, auto-update on sync

Immutable option

- Prevents changes, reduces API server load
- Requires delete and recreate to modify

```yaml
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-settings
data:
  FEATURE_X_ENABLED: "true"
  SERVICE_ENDPOINT:
"https://api.example.com"
  app.config: |
    log_level=info
    timeout_seconds=30
```

Implement secrets for sensitive data

Kubernetes Secrets

- Store credentials, API keys, and connection strings
- Inject via secretKeyRef or mount as files

Azure Key Vault integration

- CSI driver mounts secrets directly from Key Vault into Pods
- Automated rotation and audit logging

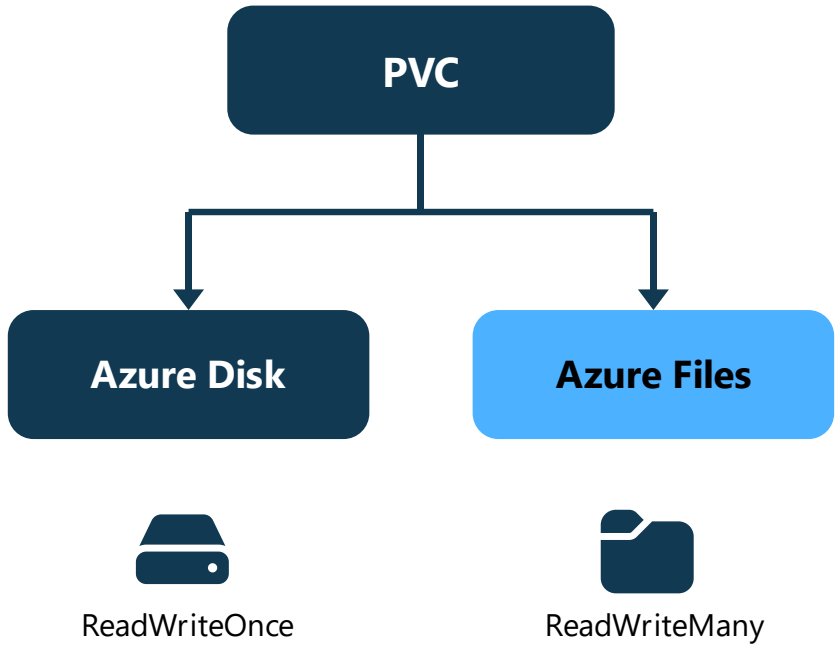
Best practices

- Restrict access with RBAC, never commit literal values
- Rotate credentials regularly and trigger rollouts





Attach persistent storage to an app



Storage model

- PVC requests storage with capacity and access mode
- StorageClass dynamically provisions Azure Disk or Azure Files

Access modes

- ReadWriteOnce: single-node block storage (Azure Disk)
- ReadWriteMany: shared file access across Pods (Azure Files)

Verification

- Confirm PVC status is Bound after applying
- Write a test file, delete the Pod, verify data persists

https://github.com/ronaldharmsen/azure-4-devs

Application Configuration

Azure KeyVault

- Centralized, cloud-hosted storage for secrets, keys, and certificates
- Entra ID authentication with fine-grained RBAC authorization
- Automatic versioning with Event Grid integration for rotation automation
- SDK support for Python, .NET, Java, JavaScript, and Go

Store and organize secrets, keys, and certificates

Three object types:

- Secrets: arbitrary strings like API keys, connection strings, and passwords (up to 25 KB)
- Keys: server-side cryptography operations, key material never leaves the vault
- Certificates: X.509 lifecycle management with certificate authority integration

Vault organization and access control:

- Separate vault per application and environment limits blast radius
- Secrets User role for app identities (read-only), Secrets Officer for operators
- Soft delete enabled by default, purge protection recommended for production

Retrieve secrets using Azure SDK client libraries

SecretClient operations:

- Initialized with vault URL and credential object
- `get_secret()` returns value plus metadata: version, created date, content type
- `list_properties_of_secrets()` discovers secrets without reading values

Error handling decisions:

- ResourceNotFoundError: configuration error, don’t retry
- HttpResponseError: auth or server failure, check RBAC assignments
- ServiceRequestError: transient network issue, safe to retry

```
# Retrieve a secret and access metadata
secret = client.get_secret("openai-api-key")

api_key = secret.value
version = secret.properties.version
created = secret.properties.created_on
```

Handle secret versioning and rotation

Versioning and rotation:

- Each `set_secret()` creates a new version with a unique identifier
- `get_secret()` returns the latest enabled version by default
- Three strategies: manual, automated (Event Grid), dual-credential

Zero-downtime credential updates:

- App catches auth failure, refetches secret from vault (gets latest version)
- Retries with new credential, no restart needed
- Set `expires_on` plus Event Grid to automate rotation triggers

```
# Versioning: each set creates a new version
v1 = client.set_secret("db-key", "old-value")
v2 = client.set_secret("db-key", "new-value")

# Returns latest version (v2)
latest = client.get_secret("db-key")

# Retrieve a specific version
old = client.get_secret(
    "db-key", version=v1.properties.version
)
```

Implement caching strategies to reduce Key Vault calls

Why cache:

- Key Vault throttles at 4,000 GET per vault per 10-second window
- Time-based cache stores secret with TTL, refreshes periodically
- Event Grid invalidation provides near-real-time freshness

Cache scope and TTL guidance:

- Per-process cache: simplest approach, recommended starting point
- Shared distributed cache: reduces total calls but adds complexity
- Match TTL to rotation frequency (90-day rotation = 1-hour TTL)

```python
# Time-based cache with configurable TTL
_cache = {}
_ttl = 900  # 15 minutes

def get_cached_secret(client, name):
    cached = _cache.get(name)
    now = time.monotonic()
    if cached and (now - cached["ts"]) < _ttl:
        return cached["value"]
    secret = client.get_secret(name)
    _cache[name] = {
        "value": secret.value, "ts": now
    }
    return secret.value
```

Use Key Vault secrets in App Service environment variables

How it works:

- The app reads a normal environment variable; the secret itself stays in Key Vault
- App Service resolves the reference and injects the value at startup
- No SDK or code changes needed

Setup:

1. Turn on a managed identity for the App Service
2. Grant it the Key Vault Secrets User role on the vault
3. Add an app setting with a Key Vault reference as its value

App Service › Settings › Environment variables

```
Name:  ConnectionStrings__Db
Value:
@Microsoft.KeyVault(VaultName=kv-training;
  SecretName=db-connection)

# Or point at a full secret URI
@Microsoft.KeyVault(SecretUri=
  https://kv-training.vault.azure.net/
  secrets/db-connection/)
# Enter the reference as one line
```

Same Environment variables blade we used for app settings and slot settings earlier in the course.

Azure App Configuration

- Managed key-value store for application settings separate from code
- Supports hierarchical keys, labels for environment variants, and feature flags
- Single retrieval path unifies nonsensitive settings and Key Vault secret references
- Dynamic refresh enables configuration updates without application restart
- Python provider library returns settings as a dictionary-like object

Connect to App Configuration from application code

Data model and connection:

- Settings stored as key-value pairs with optional labels and content types
- Provider `load()` returns dictionary-like object for standard access

Filtering and refresh:

- `SettingSelector` filters keys by prefix pattern and label
- Sentinel key pattern triggers configuration reload without restart

```
# Filter settings by key prefix and label
selects = [
    SettingSelector(
        key_filter="Pipeline:*",
        label_filter="Production"
    )
]

config = load(
    endpoint=endpoint,
    credential=credential,
    selects=selects
)
batch_size = config["Pipeline:BatchSize"]
```

Organize settings with labels and feature flags

Labels and composition:

- Labels create environment-specific variants of the same key
- Defaults load first, environment labels override matching keys

Feature flags:

- `FeatureManager.is_enabled()` evaluates flags at runtime
- Operators toggle features without redeployment

```python
# Evaluate feature flags at runtime
config = load(
    endpoint=endpoint,
    credential=credential,
    feature_flags_enabled=True
)

feature_manager = FeatureManager(config)
if feature_manager.is_enabled("UseNewModel"):
    process_with_new_model(document)
else:
    process_with_current_model(document)
```

Use App Configuration in App Service environment variables

How it works:

- Settings stay centralised in App Configuration; the app reads an environment variable
- Values are fetched when the app starts or restarts, with no live refresh
- Mark them as deployment slot settings so each slot can use its own label

Setup:

1. Turn on a managed identity for the App Service
2. Grant it the App Configuration Data Reader role
3. Add an app setting with an App Configuration reference as its value

App Service › Settings › Environment variables

```
Name:  Pipeline__BatchSize
Value:
@Microsoft.AppConfiguration(
  Endpoint=https://appcs-training.azconfig.io;
  Key=Pipeline:BatchSize;
  Label=Production)
# Enter the reference as one line

# Need live refresh or feature flags?
# Use the App Configuration provider in code
```

Key Vault references stored in App Configuration also resolve this way, so one setting can bring in a secret too.

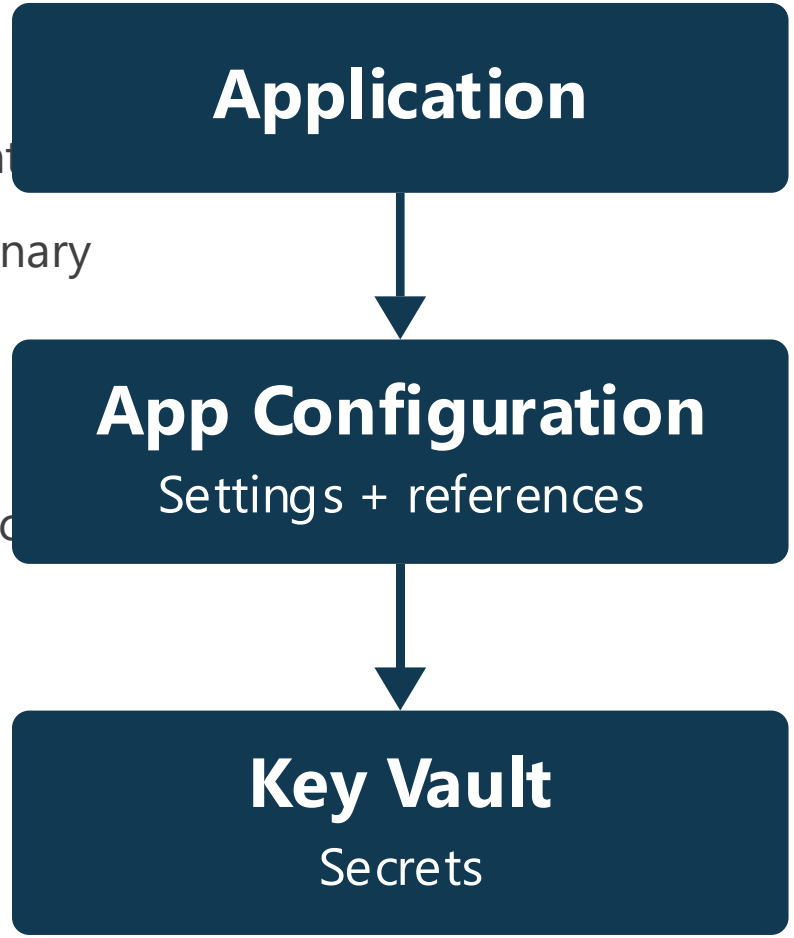
Reference Key Vault secrets from App Configuration

How references work:

- App Configuration stores a URI pointer, not the secret value
- Provider resolves secrets automatically using the reference content
- Application accesses secrets and settings through the same dictionary

Authentication and refresh:

- Identity needs Data Reader and Key Vault Secrets User roles
- `secret_refresh_interval` controls independent secret refresh c



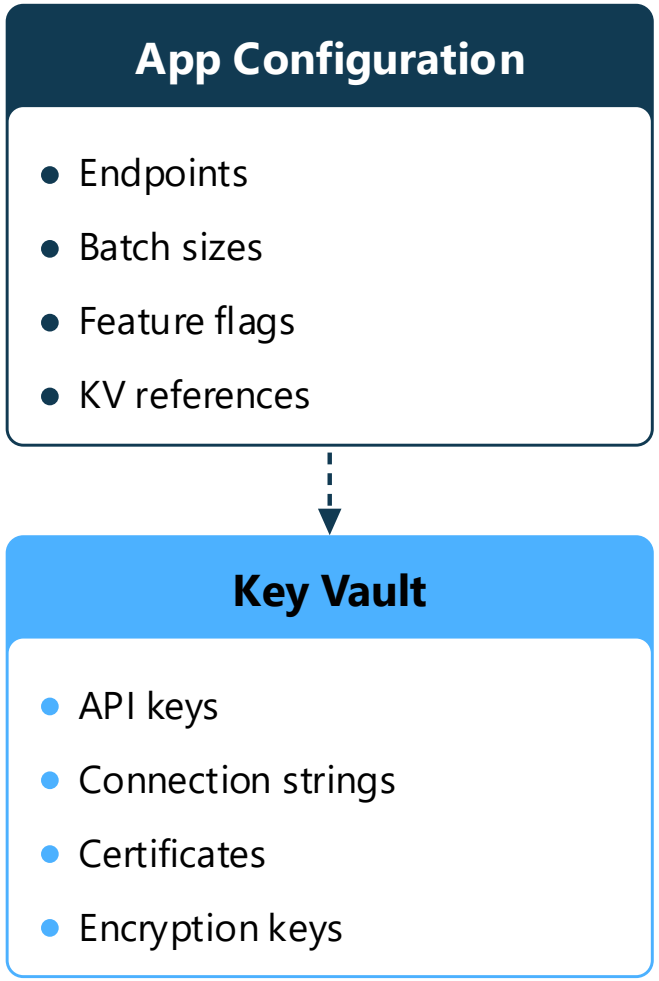
Decide what to store in App Configuration vs Key Vault

Classification criteria:

- Sensitivity: does the value grant access to a resource?
- Rotation: does it need audit trails and expiration?
- Access control: does it require per-object tracking?

Complementary architecture:

- App Configuration as single entry point for all settings
- Key Vault references for sensitive values behind the scenes



Azure Storage

Storage in the cloud

01

Storing files in Azure Blob & File storage

02

Tabular data: Azure Tables

03

Introduction to Azure SQL & Data Storage

04

Introduction to NoSQL on Azure: Azure Cosmos DB

05

Overview of moving data around: Data Factory

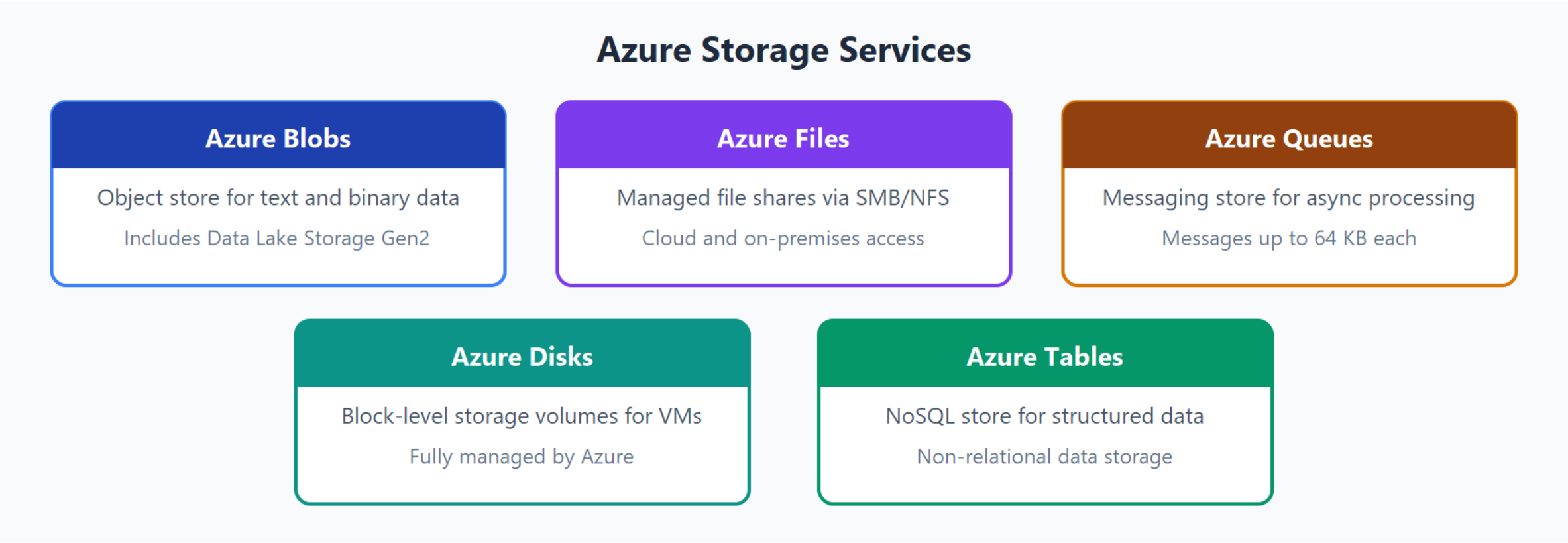
1. Overview of Azure Storage

- What is Azure Storage?
- Overview of blob storage
- Overview of table storage
- Overview of queue storage

What is Azure Storage?

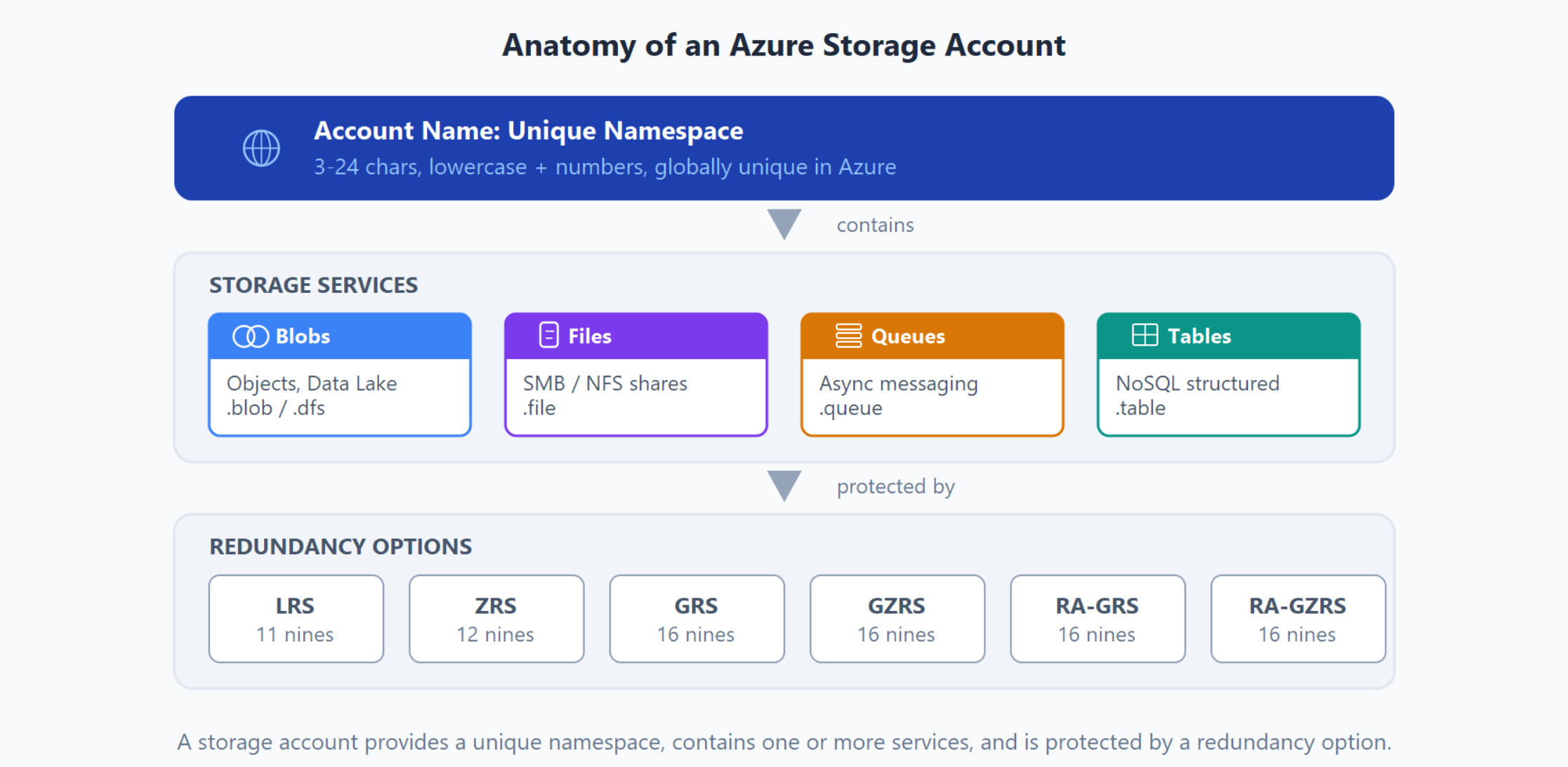
- Azure Storage is a cloud-based, large-scale data store for persisting data in the cloud
- You can't just store data on the VM file system…
  - Due to the transient nature of VMs being created and destroyed to meet elastic demands
- Azure Storage offers several data services…
  - Blob storage (incl. Data Lake Storage)
  - Azure Files, Queue storage and Table storage
  - Managed Disks for virtual machines
- All of these storage options support replication (LRS, ZRS, GRS, GZRS)
  - For availability and disaster recovery

Azure storage services



© Copyright Microsoft Corporation. All rights reserved.

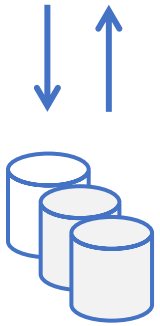
Azure storage accounts



© Copyright Microsoft Corporation. All rights reserved.

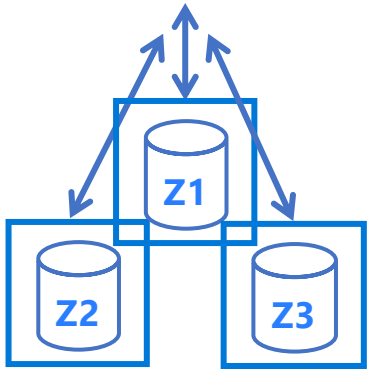
Evaluate Azure Storage redundancy options (1 / 2)

Single region



LRS

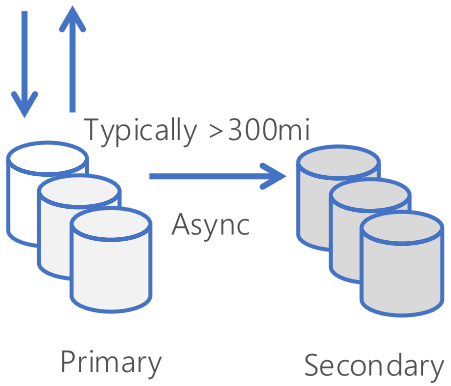
- Three replicas, one region
- Protects against disk, node, rack failures
- Write is acknowledged when all replicas are committed
- Superior to dual-parity RAID



ZRS

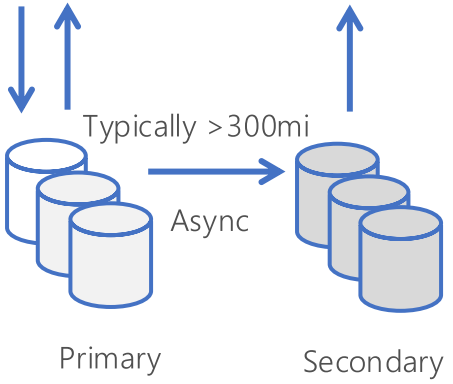
- Three replicas, three zones, one region
- Protects against disk, node, rack, and zone failures
- Synchronous writes to all three zones

Multiple regions



GRS

- Six replicas, two regions (three per region)
- Protects against major regional disasters
- Asynchronous copy to secondary

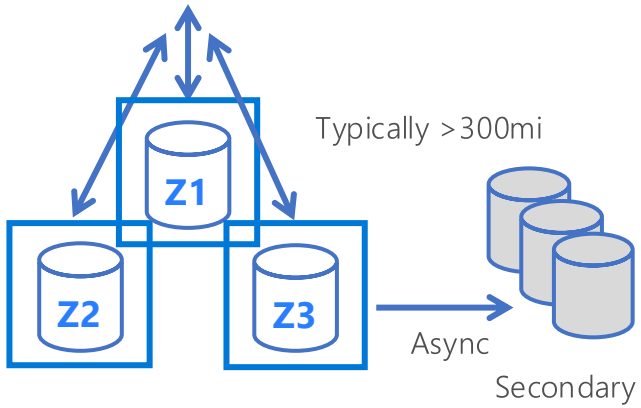


RA-GRS

- GRS + read access to secondary
- Separate secondary endpoint
- Recovery point objective (RPO) delay to secondary can be queried

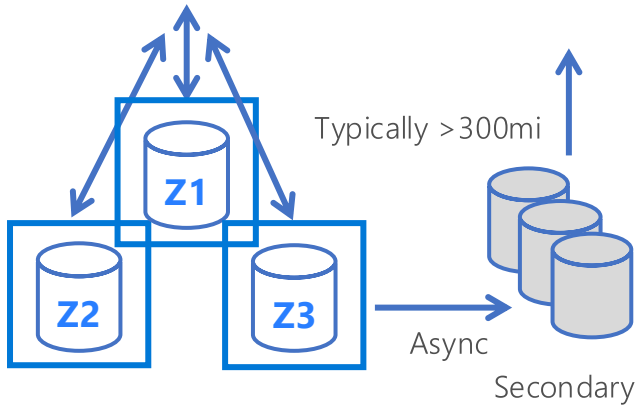
Evaluate Azure Storage redundancy options (2 / 2)

Multiple regions



GZRS

- Six replicas, 3+1 zones, two regions
- Protects against disk, node, rack, zone, and region failures
- Synchronous writes to all three zones and asynchronous copy to secondary



RA-GZRS

- GZRS + read access to secondary
- Separate secondary endpoint
- RPO delay to secondary can be queried

Overview of Blob Storage

- Blob storage is an unstructured collection of data objects
  - Accessed via a resource identifier
- Useful for storing durable unstructured files, such as:
  - Video and audio files for streaming
  - Large text documents
  - Backups and disaster recovery data
- Maximum size:
  - ~190.7 TiB per block blob
  - 8 TiB per page blob
  - ~195 GiB per append blob
  - (We'll discuss the blob types later)

Overview of Table Storage

- Table Storage is a semi-structured collection of objects
  - Objects can have fields, but not relationships between objects
- The fields in an object aren't bound to a schema structure
  - Different objects can have different fields
  - You use a special query API to find and access objects
- Maximum size:
  - Up to the storage account limit (5 PiB by default)
  - Same API available in Azure Cosmos DB for Table

Overview of Queue Storage

- Queue Storage provides a persistent messaging queue
  - Stores messages, useful for communicating between layers in a cloud solution

- Example scenario:
  - A web app receives requests in real time from a user, and sends messages to Queue Storage
  - A background worker (Function, WebJob or container) picks off messages from Queue Storage when it's ready

- Maximum size:
  - 64 KiB per message

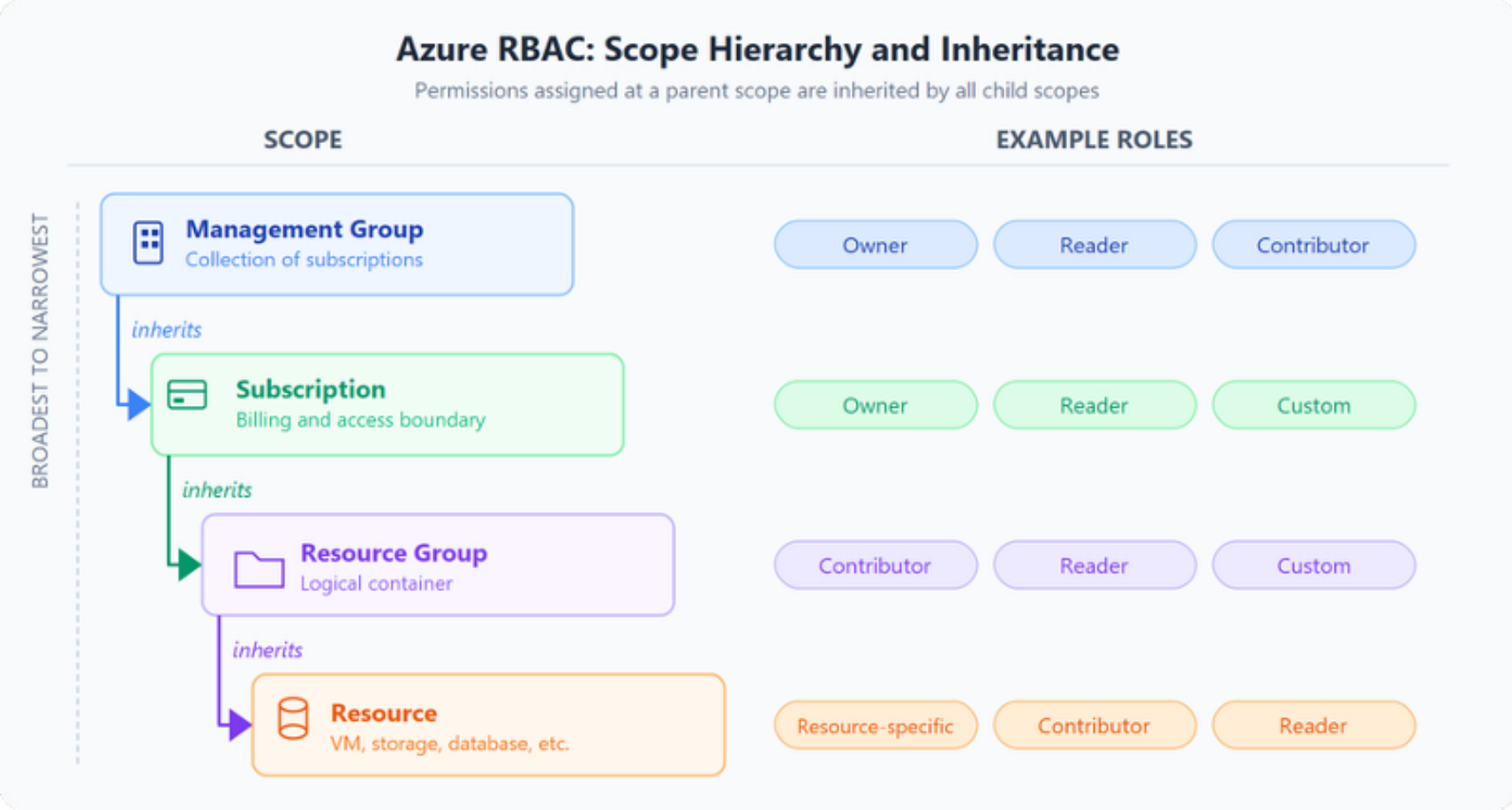
2. Getting Started Using Azure Storage

- Azure storage accounts
- Creating a storage account
- Configuring a storage account
- Using the Azurite storage emulator
- Authenticating: Microsoft Entra ID and connection strings
- Accessing a Storage Account from .NET

Azure Storage Accounts

- To use Azure Storage, you must first create an Azure Storage account
  - The storage account will store your blobs, tables, and queues
- A storage account provides:
  - A DNS prefix for URLs for your blobs, tables, and queues
  - Microsoft Entra ID (RBAC) authorization, plus 2 access keys
  - Integration with Azure Front Door / CDN for caching content close to users
- E.g. if you create a storage account named `myaccount`, use the following URLs to access blobs, tables, and queues
  - `https://myaccount.blob.core.windows.net/`
  - `https://myaccount.table.core.windows.net/`
  - `https://myaccount.queue.core.windows.net/`

Role-based access control (RBAC)



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Configuring a Storage Account (1 of 2)

- Wait until the deployment completes, then select Go to resource
- Use the left-hand menu of the storage account to configure:
  - Redundancy (geo-replication)
  - Data protection (soft delete, versioning)
  - Networking (public access, private endpoints)
  - Configuration (e.g. disable anonymous blob access)
  - Monitoring (Insights, diagnostic settings)

Configuring a Storage Account (2 of 2)

- Every storage account has 2 access keys associated with it
  - To view them, open Security + networking > Access keys in the Azure portal
  - You can view and rotate the access keys
- Prefer Microsoft Entra ID or SAS tokens over account keys
  - You can disable shared key access on the account entirely

Using the Azurite Storage Emulator

- Azurite is the local emulator for Azure Storage (it replaces the retired Storage Emulator)
  - Provides local blob/queue/table storage
  - Runs anywhere: built into Visual Studio, npm or Docker
  - Ideal for testing applications locally
  - No need for an Azure subscription!
- Some features differ from the cloud service; see the docs:
  - learn.microsoft.com/azure/storage/common/storage-use-azurite

Defining a Storage Account Connection String

- If you use account keys, keep the connection string out of your code
  - In a .NET app, put it in appsettings.json locally (or user secrets)
  - In Azure, use App Service settings or Azure Key Vault
- Examples
  - Connection string for a real Azure storage account

```
"ConnectionStrings": {
  "Storage": "DefaultEndpointsProtocol=https;AccountName=your-account;
     AccountKey=your-account-key;EndpointSuffix=core.windows.net"
}
```

  - Connection string for the Azurite emulator

```
"ConnectionStrings": {
  "Storage": "UseDevelopmentStorage=true"
}
```

Accessing a Storage Account from .NET

- The Azure SDK for .NET ships one NuGet package per service
  - E.g. `Azure.Storage.Blobs`, `Azure.Data.Tables`, `Azure.Identity`

- You create one client per service, e.g. `BlobServiceClient`
  - Recommended: Entra ID via `DefaultAzureCredential`

```
var credential = new DefaultAzureCredential();
var blobService = new BlobServiceClient(
    new Uri("https://myaccount.blob.core.windows.net"), credential);
```

  - Alternatively you can create via a connection string

```
string cnStr = builder.Configuration.GetConnectionString("Storage");
var blobService = new BlobServiceClient(cnStr);
```

Accessing a Storage Account from .NET

- Each service client exposes its endpoint
  - `BlobServiceClient.Uri`
  - `TableServiceClient.Uri`
  - `QueueServiceClient.Uri`

```
private static void DisplayStorageAccountUrls(
    BlobServiceClient blobs, TableServiceClient tables, QueueServiceClient queues)
{
  Console.WriteLine("Blob endpoint:  {0}", blobs.Uri);
  Console.WriteLine("Table endpoint: {0}", tables.Uri);
  Console.WriteLine("Queue endpoint: {0}", queues.Uri);
}
```

- Service clients create child clients for containers, tables and queues
  - `GetBlobContainerClient()`
  - `GetTableClient()`
  - `GetQueueClient()`

3. Using Blob Storage

- Essential concepts
- Creating a blob container
- Working with block blobs
- Working with page blobs
- Asynchronous methods
- Organizing blobs in directories
- Configuring client access to blobs

Essential Concepts

- Blob storage is useful for storing unstructured files in the cloud
  - Video / audio files, documents, etc.

- Blobs are stored in a container
  - You can have many containers in your storage account
  - Each container is like a flat folder, and can hold any number of blobs

- There are three types of blob:
  - Block blobs: files and streams, uploaded in blocks
  - Append blobs: optimised for appending, e.g. logs
  - Page blobs: 512-byte pages for random access (VM disks)

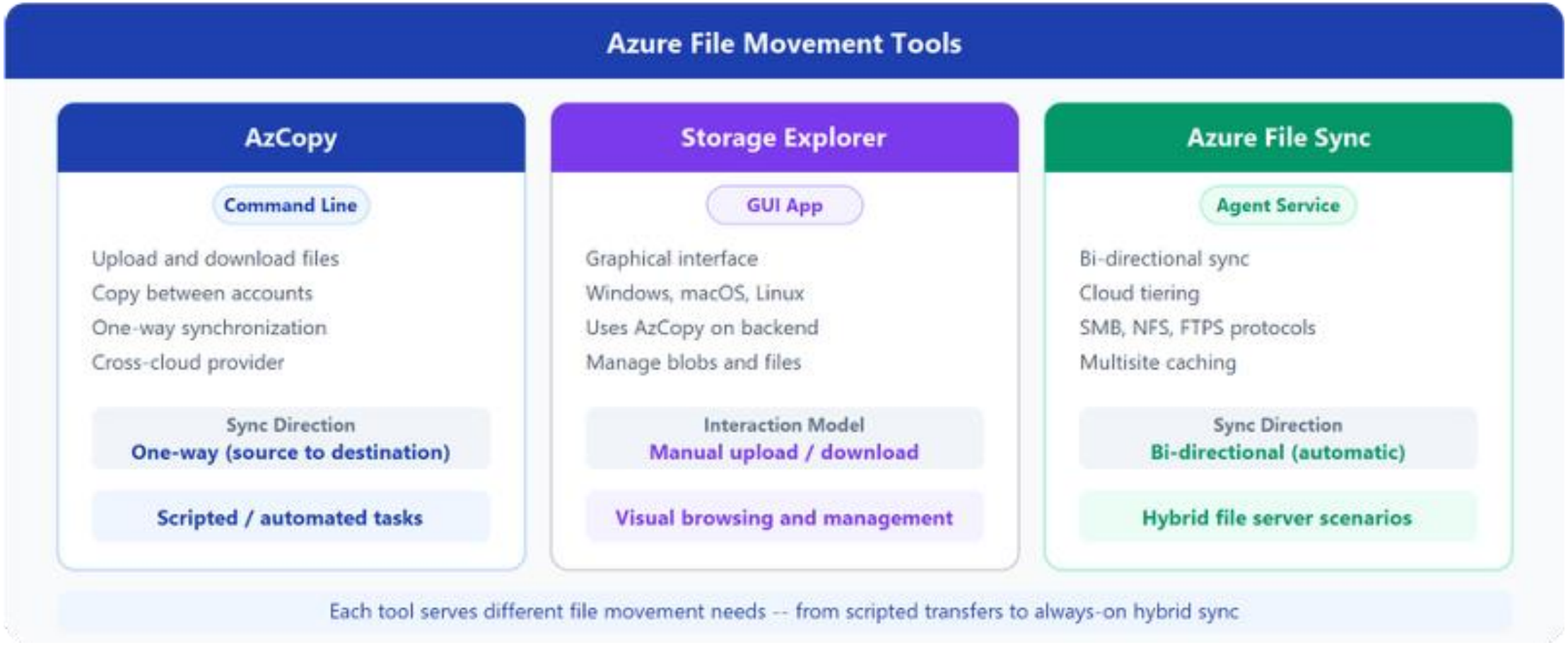
Creating a Blob Container (1 of 4)

- Before you can create any blobs, you must first create a blob container
  - Some names are reserved for special containers ($root, $web, $logs)
- There are several ways to create a container…
  - The easiest way is via the Azure portal: Storage account > Data storage > Containers > + Container

Creating a Blob Container (2 of 4)

- If you're using Azurite locally…
  - You can create a blob container with Azure Storage Explorer or the Azure Storage extension for VS Code

Azure file movement options



Source: Microsoft AZ-900 Azure Fundamentals course (AZ-900T00)

Creating a Blob Container (3 of 4)

- You can also create a blob container programmatically
  - Start from a `BlobServiceClient`; it's the entry point for all blob operations
  - Call GetBlobContainerClient() to get a reference to a container
  - Call CreateIfNotExistsAsync() to create the container

```
BlobContainerClient container =
    blobService.GetBlobContainerClient(containerName);
await container.CreateIfNotExistsAsync();
```

- You can set the public access level (blocked by default on new accounts)
  - Call `SetAccessPolicyAsync()` with appropriate parameters

```
await container.SetAccessPolicyAsync(
    PublicAccessType.Blob);
// Fails unless anonymous blob access is enabled on the account
```

- You can also delete a container if you want to

```
await container.DeleteAsync();
```

Creating a Blob Container (4 of 4)

- Azure Storage has a REST API for all data operations
  - Reference: learn.microsoft.com/rest/api/storageservices
- Example:
  - REST request to create a container:

```
PUT https://myaccount.blob.core.windows.net/mycontainer?restype=container
```

  - REST request to delete a container:

```
DELETE https://myaccount.blob.core.windows.net/mycontainer?restype=container
```

  - In both cases, pass the following HTTP headers into the request:

```
x-ms-version: 2026-04-06
x-ms-date: Thu, 15 Oct 2026 16:52:32 GMT
x-ms-meta-Name: MyContainerExample
Authorization: Bearer <Microsoft Entra ID access token>
```

Working with Block Blobs (1 of 2)

- Block blobs represent a contiguous file in Azure Storage
  - You can get a reference to a blob from a container like so:

```
BlobClient blob = container.GetBlobClient(blobName);
```

  - You can upload, download, delete, etc. a blob in a container

```
await blob.UploadAsync(aFileStream, overwrite: true);
```

```
await blob.DownloadToAsync(aFileStream);
```

```
await blob.DeleteAsync();  // Also DeleteIfExistsAsync()
```

  - You can get a list of all the blobs in a container

```
await foreach (BlobItem item in container.GetBlobsAsync())
{
    Console.WriteLine("\t" + item.Name);
}
```

Working with Block Blobs (2 of 2)

- It's also possible to upload a blob in chunks (i.e. blocks!)
  - Each block has its own ID, and can also have an optional MD5 hash to ensure reliable delivery
  - BlockBlobClient.StageBlockAsync() uploads a block; multiple threads can upload in parallel
  - Blocks are not committed until the end of the upload
  - So you can replace uncommitted blocks
  - At the end, CommitBlockListAsync() commits the block IDs you specify
- The SDK does this for you on large uploads
  - Tune it with StorageTransferOptions
  - See the BlockBlobChunking.sln demo project

Working with Page Blobs (1 of 2)

- Page blobs represent files split into 512-byte pages
  - You can get a reference to a page blob from a container like so:

```
PageBlobClient pageBlob = container.GetPageBlobClient(blobName);
```

  - You can create and delete a page blob:

```
await pageBlob.CreateAsync(sizeOfBlobInBytes);
```

```
await pageBlob.DeleteAsync();  // Also DeleteIfExistsAsync()
```

Working with Page Blobs (2 of 2)

- You can upload data to a specific page
  - If the data is too large for 1 page, it's automatically split into multiple pages

```
await pageBlob.UploadPagesAsync(streamOfData, offset512ByteBoundary);
```

- You can get a list of all the pages that are actually populated with data in the blob
  - You can then then download the data for each page

```
using Stream blobStream = await pageBlob.OpenReadAsync();

await foreach (PageRangeItem item in pageBlob.GetAllPageRangesAsync())
{
    if (item.IsClear) continue;
    blobStream.Seek(item.Range.Offset, SeekOrigin.Begin);

    byte[] buffer = new byte[item.Range.Length.Value];
    await blobStream.ReadExactlyAsync(buffer);
    …
}
```

Asynchronous Methods

- Every SDK method has an async version (…Async) for use with async/await
  - Highly recommended: storage calls are network I/O
- E.g. asynchronous methods in BlobClient:
  - UploadAsync(), DownloadToAsync()
  - DeleteAsync(), DeleteIfExistsAsync()
  - Etc…
- E.g. asynchronous methods in PageBlobClient:
  - UploadPagesAsync(), OpenReadAsync()
  - GetAllPageRangesAsync()
  - Etc…
- Listings return AsyncPageable<T>: iterate with await foreach
- The old Begin…/End… methods no longer exist

Organizing Blobs in Directories

- Blob names can include / characters, to simulate the idea of directories
  - E.g. a blob named "dir1/blob1" is notionally in the "dir1" directory

- List by prefix and delimiter to walk virtual directories
  - To list the contents of a virtual directory:

```
await foreach (BlobHierarchyItem item in
    container.GetBlobsByHierarchyAsync(
        new GetBlobsByHierarchyOptions { Prefix = "dir1/", Delimiter = "/" }))
```

  - To tell sub-directories from blobs:

```
if (item.IsPrefix) Console.WriteLine("Dir: " + item.Prefix);
else Console.WriteLine(item.Blob.Name);
```

Configuring Client Access to Blobs

- You can configure how the client application accesses blobs in a storage account
  - Pass BlobClientOptions when you create the BlobServiceClient

```
var options = new BlobClientOptions();
options.Retry.Mode = RetryMode.Exponential; options.Retry.MaxRetries = 5;
var blobService = new BlobServiceClient(uri, credential, options);
```

  - Pass StorageTransferOptions per upload to tune chunking and parallelism

```
var transfer = new StorageTransferOptions { InitialTransferSize = 50 * 1024 * 1024 };
```

```
transfer.MaximumConcurrency = 5;  // pass via BlobUploadOptions.TransferOptions
```

4. Using Table Storage

- Essential concepts
- Creating a table
- Defining entity types
- Overview of table APIs
- Working with entities
- Querying entities

Essential Concepts

- Table storage is a key-value storage area, an alternative to using Azure SQL Database
  - You create tables in your storage account
  - A table contains a collection of key-value pairs
  - Each entity is identified by a PartitionKey + RowKey (RowKey is often a GUID)
  - The value is called an "entity" (nothing to do with EF!) and is a set of properties
- The same table can contain any types of values
  - There's no schema for the table – i.e. no column types
  - You can't define relationships and joins
- Hence table storage is more like NoSQL than SQL Server
  - The purpose of table storage is to be more scalable than a RDBMS

Creating a Table

- There are several ways to create a table…
  - Use the Azure portal
  - Use Azurite locally (with Storage Explorer)
  - Create programmatically in code:

```
TableServiceClient tableService = new TableServiceClient(tableUri, credential);
TableClient table = tableService.GetTableClient(tableName);

await table.CreateIfNotExistsAsync();    // Plus various other methods
```

  - Create via an HTTP REST request:

```
POST  https://myaccount.table.core.windows.net/Tables HTTP/1.1
```

Defining Entity Types (1 of 2)

- You can store any type of value in a table
  - Define a class that implements `ITableEntity`
  - … or use `TableEntity` (a property dictionary) for untyped access

- Additional rules for defining entity types
  - Define a no-arg constructor
  - Implement the PartitionKey property (entities are grouped into partitions for efficiency)
  - Implement the RowKey property (effectively the primary key, e.g. a GUID)
  - Implement Timestamp and ETag (set by the service)

- Allowed property types in entities:
  - int, long, double, bool, byte[], string, DateTime, DateTimeOffset, Guid, BinaryData

Defining Entity Types (2 of 2)

- Here's an example entity for table storage:

```
public class BankAccountEntity : ITableEntity
{
    public string PartitionKey { get; set; }   // e.g. branch
    public string RowKey { get; set; }         // e.g. account number

    public DateTimeOffset? Timestamp { get; set; }
    public ETag ETag { get; set; }

    public string AccountHolderName { get; set; }
    public double Balance { get; set; }

    public int NumDeposits { get; set; }
    public int NumWithdrawals { get; set; }
}
```

Overview of Table APIs

- The TableClient class has methods to execute CRUD operations on a table, and to manage the table itself
  - It's important to understand that we're not using SQL any more!!!
  - To create / delete a table:

```
await table.CreateAsync();
await table.CreateIfNotExistsAsync();
await table.DeleteAsync();
```

  - To create, read, update, or delete entities:

```
await table.AddEntityAsync(entity);
Response<BankAccountEntity> r = await table.GetEntityAsync<BankAccountEntity>(pk, rk);
await table.SubmitTransactionAsync(batchActions);
AsyncPageable<BankAccountEntity> q = table.QueryAsync<BankAccountEntity>(filter);
```

- Every method has a synchronous and an async version

Working with Entities (1 of 2)

- TableClient has one method per entity operation
  - Each returns a Response with the result and the HTTP status

```
Response r = await table.AddEntityAsync(aBankAccountEntity);
int status = r.Status;   // 204 No Content
```

```
Response<BankAccountEntity> result =
    await table.GetEntityAsync<BankAccountEntity>(partitionKey, rowKey);
BankAccountEntity retrievedEntity = result.Value;
```

```
Response response = await table.DeleteEntityAsync(partitionKey, rowKey);
int httpStatusCode = response.Status;
```

Working with Entities (2 of 2)

- TableClient gives you several ways to insert or update entities
  - Insert this entity

```
await table.AddEntityAsync(anEntity);
```

  - Merge this entity with existing entity

```
await table.UpdateEntityAsync(anEntity,
anEntity.ETag, TableUpdateMode.Merge);
```

  - This entity replaces existing entity

```
await table.UpdateEntityAsync(anEntity,
anEntity.ETag, TableUpdateMode.Replace);
```

  - Insert this entity, or merge with existing

```
await table.UpsertEntityAsync(anEntity,
TableUpdateMode.Merge);
```

  - Insert this entity, or replace existing

```
await table.UpsertEntityAsync(anEntity,
TableUpdateMode.Replace);
```

Querying Entities (1 of 2)

- To query, call QueryAsync() with a LINQ expression or an OData filter

```
AsyncPageable<BankAccountEntity> all = table.QueryAsync<BankAccountEntity>();
```

```
var query = table.QueryAsync<BankAccountEntity>(
    e => e.PartitionKey == aPartnKey);
```

```
var query = table.QueryAsync<BankAccountEntity>(
    e => e.PartitionKey == aPartnKey && e.Balance < 1000);

// Or as an OData filter string:
string filter = TableClient.CreateQueryFilter(
    $"PartitionKey eq {aPartnKey} and Balance lt {1000.0}");

var query2 = table.QueryAsync<BankAccountEntity>(filter);
```

Querying Entities (2 of 2)

- To execute a query, iterate over QueryAsync() with await foreach
  - Results are fetched page by page as you iterate
  - Returns a collection of entities

```
await foreach (BankAccountEntity account in table.QueryAsync<BankAccountEntity>(
    e => e.PartitionKey == aPartnKey))
{
    // Process each BankAccountEntity object here…
}
```

5. Using Queue Storage

- Essential concepts
- Creating a queue
- Adding a message to a queue
- Peeking messages
- Two-phase Dequeuing messages

Essential Concepts

- Queue storage is an efficient way for web apps to delegate computational work to background workers
  - Designed for scalability in cloud solutions
  - Messages are limited to 64 KiB each
  - Total queue capacity is limited only by the storage account (5 PiB by default)

- Note:
  - Azure also provides queuing via the Service Bus
  - The Service Bus is a more general mechanism for communication between large-scale applications

Creating a Queue

- There are several ways to create a queue…
  - Use the Azure portal
  - Use Azurite locally (with Storage Explorer)
  - Create programmatically in code:

```
QueueServiceClient queueService = new QueueServiceClient(queueUri, credential);
QueueClient queue = queueService.GetQueueClient(queueName);

await queue.CreateIfNotExistsAsync();    // Plus methods to delete, etc.
```

  - Create via an HTTP REST request:

```
PUT  https://myaccount.queue.core.windows.net/myqueue HTTP/1.1
```

Adding a Message to a Queue

- To add a message to a queue:
  - Call SendMessageAsync(), specifying the text content for the message
  - Optional parameters for visibility delay and time-to-live

```
string content = … ;

await queue.SendMessageAsync(content,
    visibilityTimeout: optionalVisibilityDelay,
    timeToLive: optionalTimeToLive);
```

Peeking Messages

- You can peek messages in a queue
  - Allows you to see messages without removing from the queue
  - To peek the message at the front of the queue:

```
PeekedMessage message = (await queue.PeekMessageAsync()).Value;
```

  - To peek multiple messages in the queue:

```
int count = … ;

PeekedMessage[] messages = (await queue.PeekMessagesAsync(maxMessages: count)).Value;
foreach (PeekedMessage message in messages)
{
    // Do something with the message…
}
```

- Useful properties in PeekedMessage:
  - MessageId, MessageText, InsertedOn, ExpiresOn

Two-Phase Dequeuing Messages

- Queue storage supports the notion of 2-phase dequeuing
  - Receive a message (it's hidden for the visibility timeout)
  - Process the message
  - Delete the message from the queue, using its pop receipt

```
QueueMessage message;
while ((message = (await queue.ReceiveMessageAsync()).Value) != null)
{
    // Do something with the message…

    // Then delete the message from the queue.
    await queue.DeleteMessageAsync(message.MessageId, message.PopReceipt);
}
```

CosmosDB

Identify key benefits of Azure Cosmos DB



Global replication: Automatic and synchronous multi-region replication, supports automatic and manual failover



Varied consistency levels: Offers five consistency models. Provides control over performance-consistency tradeoffs, backed by comprehensive SLAs



Low latency: Serve <10 ms read and <10 ms write requests at the 99th percentile



Elastic scale-out: Elastically scale throughput from 10 to 100s of millions of requests/sec across multiple regions

Explore the resource hierarchy (1 / 2)



Elements in an Azure Cosmos account

An Azure Cosmos container is the fundamental unit of scalability.

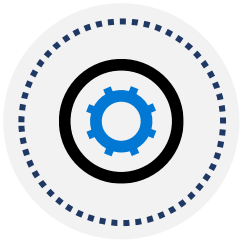
Azure Cosmos DB transparently partitions your container using the logical partition key that you specify in order to elastically scale your provisioned throughput and storage.



Azure Cosmos databases

You can create one or multiple Azure Cosmos databases under your account. A database is analogous to a namespace.

A database is the unit of management for a set of Azure Cosmos containers.



Azure Cosmos containers

An Azure Cosmos container is the unit of scalability both for provisioned throughput and storage.

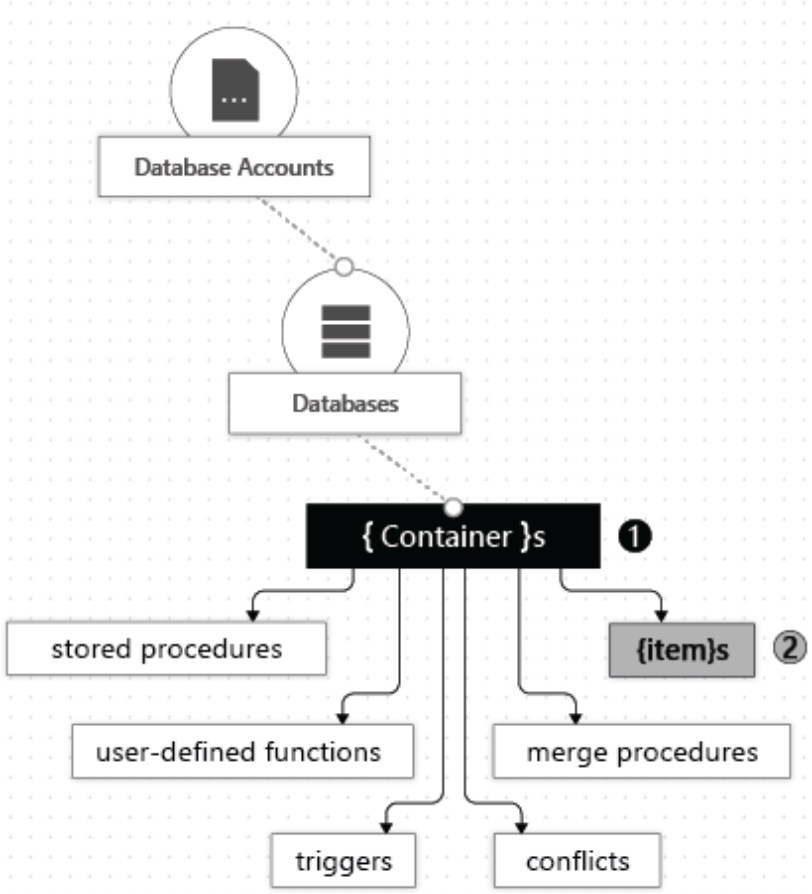
A container is horizontally partitioned and then replicated across multiple regions.

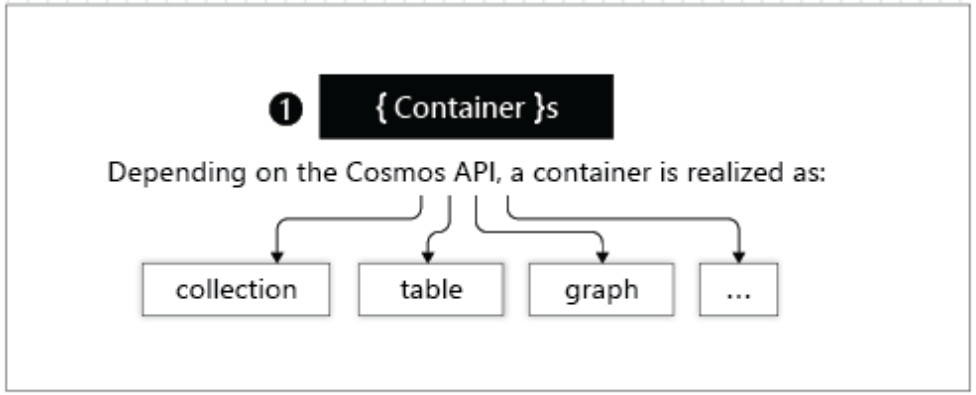


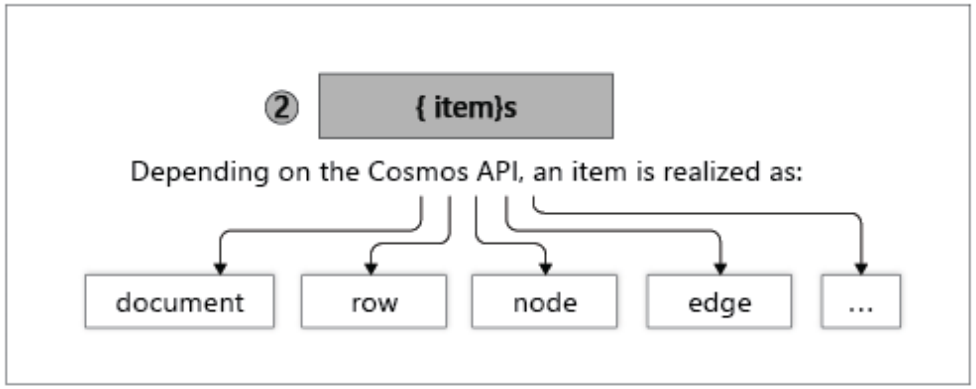
Azure Cosmos items

Depending on which API you use, an Azure Cosmos item can represent either a document in a collection, a row in a table, or a node or edge in a graph.

Explore the resource hierarchy (2 / 2)

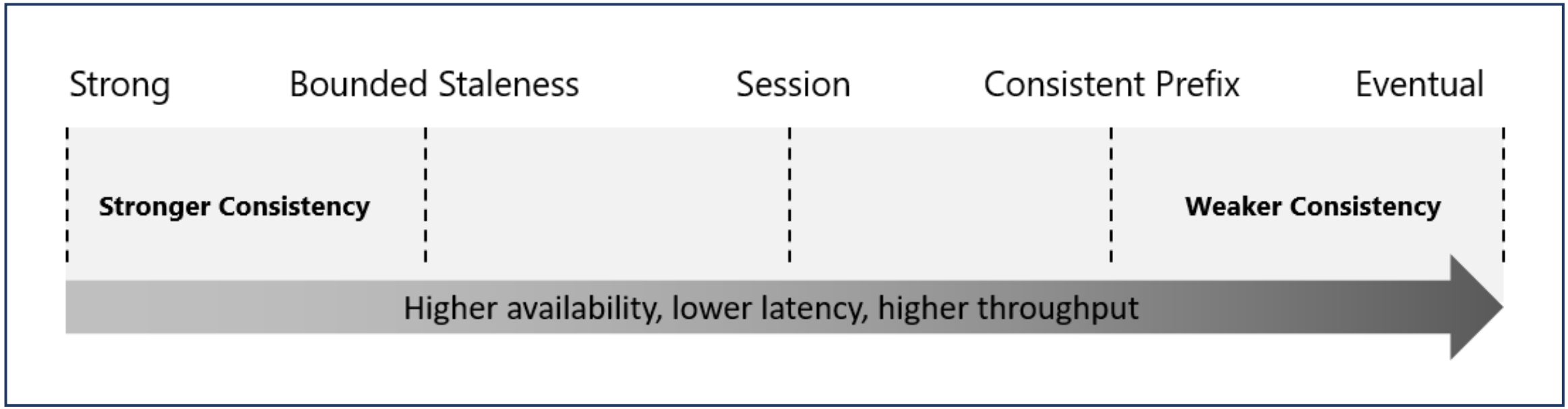






Explore consistency levels (1 / 2)

- Azure Cosmos DB approaches data consistency as a spectrum of choices instead of two extremes.



Explore consistency levels (2 /2)

Consistency Level	Description
Strong	When a write operation is performed on your primary database, the write operation is replicated to the replica instances. The write operation is committed (and visible) on the primary only after it has been committed and confirmed by all replicas.
Bounded Staleness	This level is similar to the Strong level with the major difference that you can configure how stale documents can be within replicas. Staleness refers to the quantity of time (or the version count) a replica document can be behind the primary document.
Session	This level guarantees that all read and write operations are consistent within a user session. Within the user session, all reads and writes are monotonic and guaranteed to be consistent across primary and replica instances.
Consistent Prefix	This level has loose consistency but guarantees that when updates show up in replicas, they will show up in the correct order (that is, as prefixes of other updates) without any gaps.
Eventual	This level has the loosest consistency and essentially commits any write operation against the primary immediately. Replica transactions are asynchronously handled and will eventually (over time) be consistent with the primary. This tier has the best performance, because the primary database does not need to wait for replicas to commit to finalize its transactions.

Choose the right consistency level

SQL API and Table API

For many real-world scenarios, session consistency is optimal and it's the recommended option.

If your application requires strong consistency, it is recommended that you use bounded staleness consistency level.

If you need the highest availability and the lowest latency, then use eventual consistency level.

Cassandra, MongoDB, and Gremlin APIs

- Azure Cosmos DB provides native support for wire protocol-compatible APIs for popular databases.
- These include MongoDB, Apache Cassandra, and Gremlin.
- When using Gremlin API the default consistency level configured on the Azure Cosmos account is used.

Consistency guarantees in practice

- Consistency guarantees for a read operation correspond to the freshness and ordering of the database state that you request.
- Read-consistency is tied to the ordering and propagation of the write/update operations.
- The Probabilistically Bounded Staleness metric provides an insight into how often you can get a stronger consistency than the level that you have currently configured on your Azure Cosmos account.

Explore supported APIs



Core(SQL) API: This API stores data in document format. It offers the best end-to-end experience as we have full control over the interface, service, and the SDK client libraries.



API for MongoDB: This API stores data in a document structure, via BSON format. It is compatible with MongoDB wire protocol.



Cassandra API: This API stores data in column-oriented schema and is wire protocol compatible with Apache Cassandra.

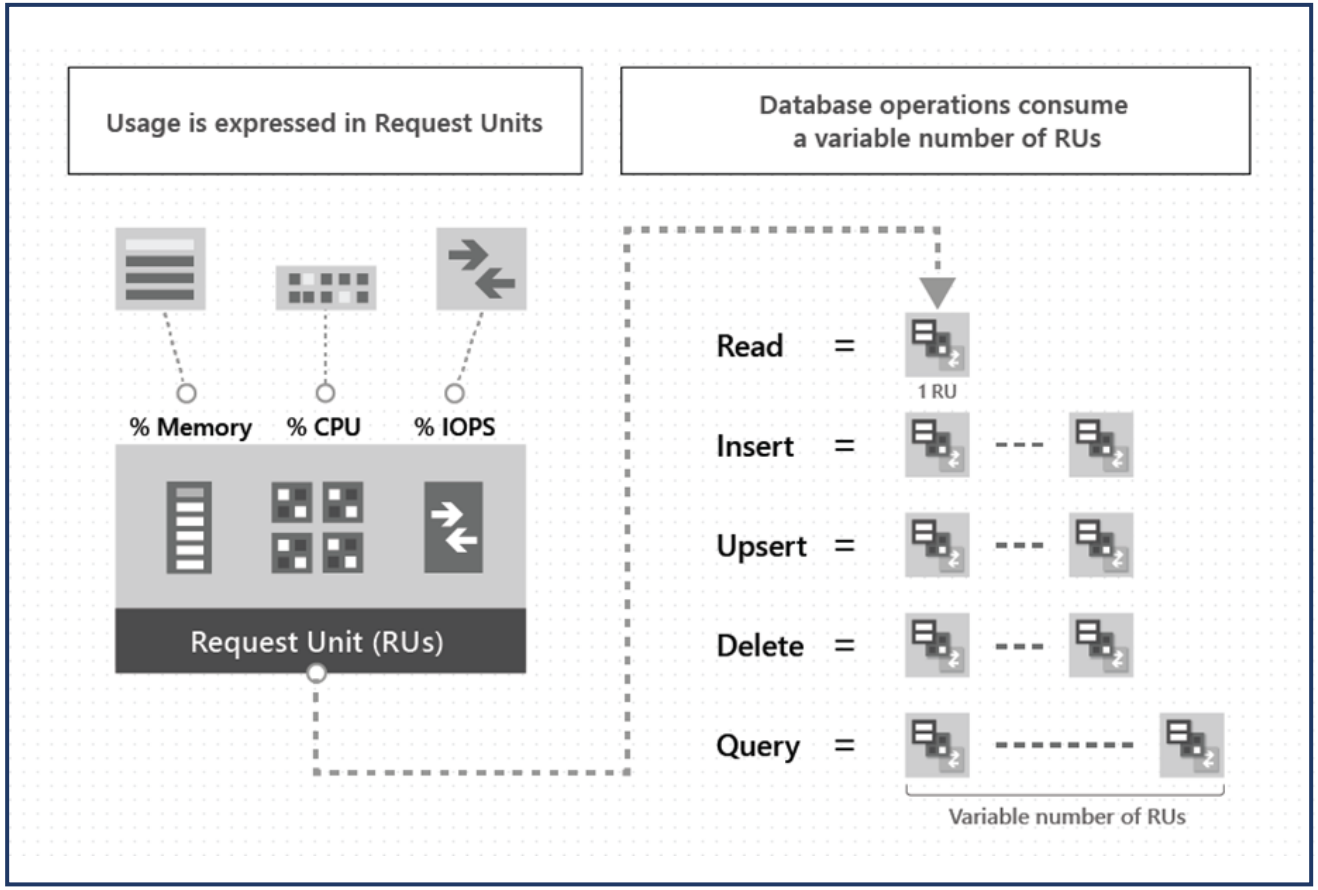


Table API: This API stores data in key/value format.



Gremlin API: This API allows users to make graph queries and stores data as edges and vertices.

Discover request units



Explore Microsoft .NET SDK v3 for Azure Cosmos DB (1 / 3)

- Creating items

```
// Get container reference (Microsoft Entra ID auth)
CosmosClient client = new CosmosClient(endpoint, new DefaultAzureCredential());
Container container = client.GetContainer(databaseName, containerName);

// Create a Product instance
Product orangeSoda = new Product {
    id = "7cc3212d-0e2c-4a13-b348-f2d879c43342",
    name = "Orange Soda", group = "Beverages",
    diet = false, price = 1.50m, quantity = 2000
};

// Upload item (pass the partition key value)
Product item = await container.CreateItemAsync(orangeSoda, new PartitionKey(orangeSoda.group));
item = await container.UpsertItemAsync(orangeSoda, new PartitionKey(orangeSoda.group)); // or upsert
```

Explore Microsoft .NET SDK v3 for Azure Cosmos DB (2 / 3)

- Reading items

```
// Get container reference (Microsoft Entra ID auth)
CosmosClient client = new CosmosClient(endpoint, new DefaultAzureCredential());
Container container = client.GetContainer(databaseName, containerName);

// Get unique fields
string id = "7cc3212d-0e2c-4a13-b348-f2d879c43342";
PartitionKey partitionKey = new PartitionKey("Beverages");

// Read item using unique id
ItemResponse<Product> response = await container.ReadItemAsync<Product>(
    id,
    partitionKey);

// Serialize response
Product item = response.Resource;
```

Explore Microsoft .NET SDK v3 for Azure Cosmos DB (3 / 3)

- Query documents

```
// Get container reference (Microsoft Entra ID auth)
CosmosClient client = new CosmosClient(endpoint, new DefaultAzureCredential());
Container container = client.GetContainer(databaseName, containerName);

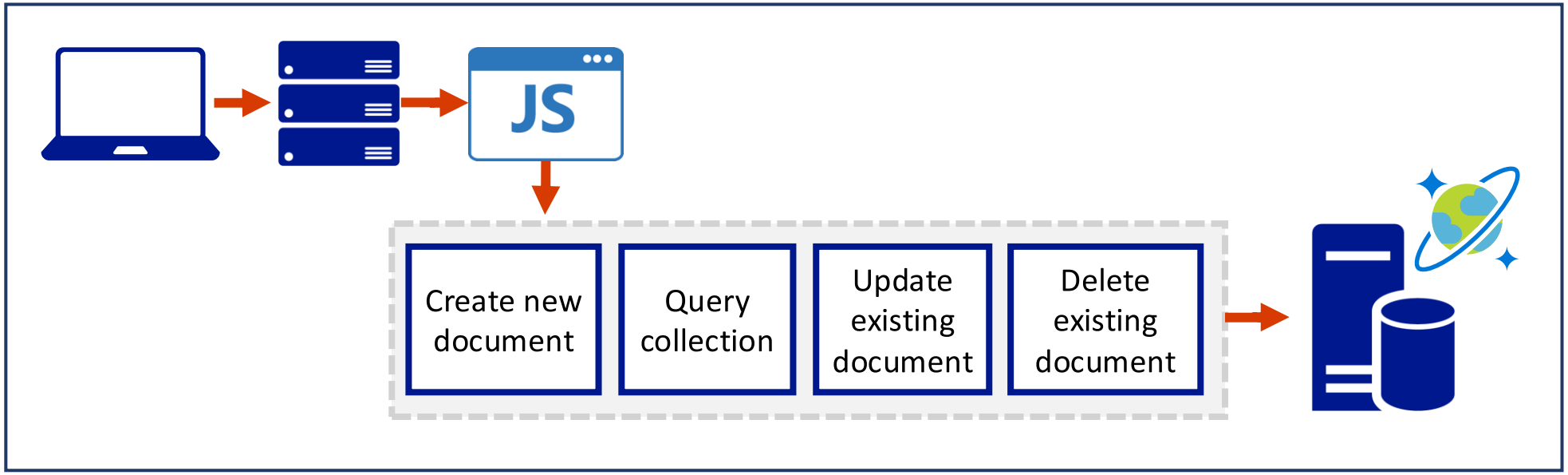
// Use a parameterized SQL query
FeedIterator<Product> iterator = container.GetItemQueryIterator<Product>(
    new QueryDefinition("SELECT * FROM products p WHERE p.diet = @diet")
        .WithParameter("@diet", false));

// Iterate over results
while (iterator.HasMoreResults)
{
    FeedResponse<Product> batch = await iterator.ReadNextAsync();
    foreach (Product item in batch) { }
}
```

Create stored procedures (1 / 4)

In Azure Cosmos DB, JavaScript is hosted in the same memory space as the database

Requests made within stored procedures and triggers run in the same scope of a database session



Create stored procedures (2 / 4)

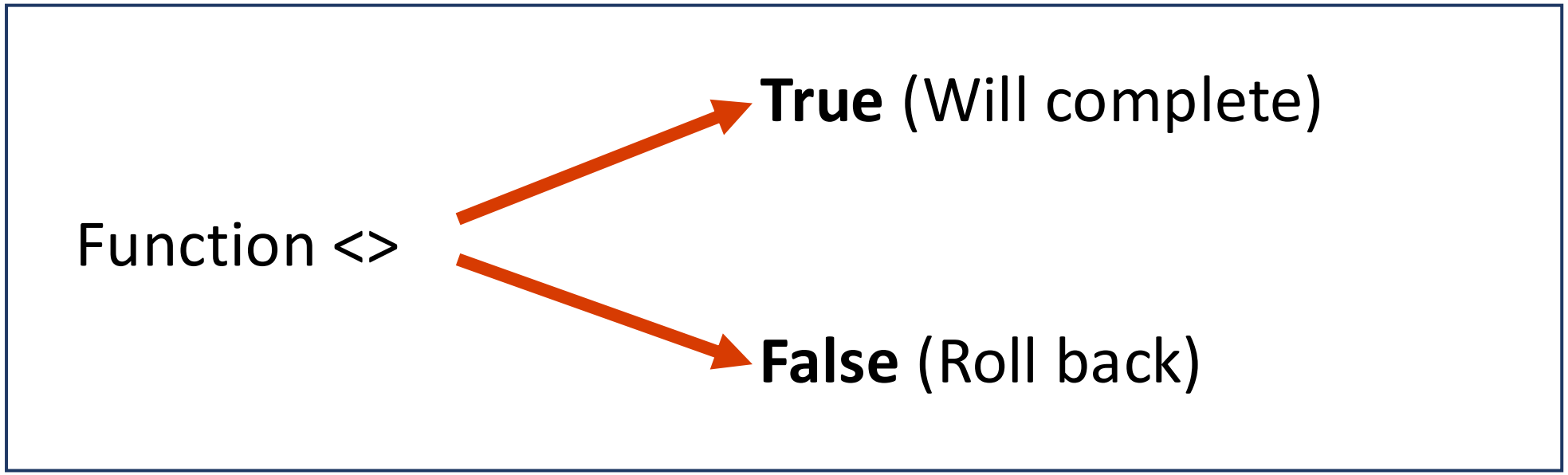
- Example code

```
function createSampleDocument(documentToCreate) {
    var context = getContext();
    var collection = context.getCollection();
    var accepted = collection.createDocument(
        collection.getSelfLink(),
        documentToCreate,
        function (error, documentCreated) {
            if (error) throw error;
            context.getResponse().setBody(documentCreated.id)
        }
    );
    if (!accepted) return;
}
```

Create stored procedures (3 / 4)

- Bounded execution

All Azure Cosmos DB operations must complete within a limited amount of time

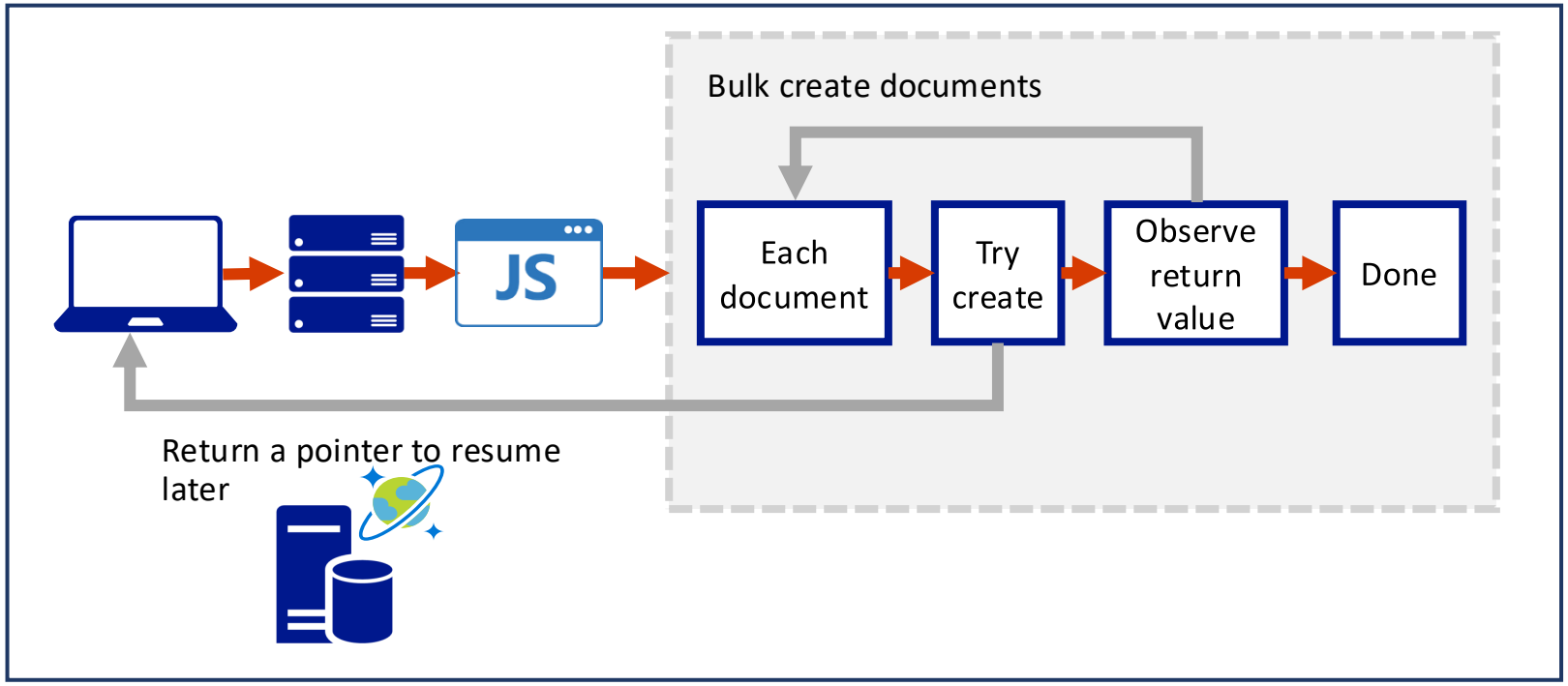


Create stored procedures (4 / 4)

- Transaction continuation

JavaScript functions can implement a continuation-based model to batch or resume execution

- The continuation value can be any value of your choice
- Your applications can then use this value to resume a transaction from a new starting point



Create triggers and user-defined functions (1 / 2)



Pre-triggers: Executed before modifying, or creating, a database item.



Post-triggers: Executed after modifying a database item



User-defined function: User-defined functions extend the Azure Cosmos DB SQL API's query language grammar and implement custom business logic.

Create triggers and user-defined functions (2 / 2)

- Pre-trigger example code

```
function validateToDoItemTimestamp() {
    var context = getContext();
    var request = context.getRequest();

    // item to be created in the current operation
    var itemToCreate = request.getBody();

    // validate properties
    if (!("timestamp" in itemToCreate)) {
        var ts = new Date();
        itemToCreate["timestamp"] = ts.getTime();
    }

    // update the item that will be created
    request.setBody(itemToCreate);
}
```

Building a communication strategy for your applications

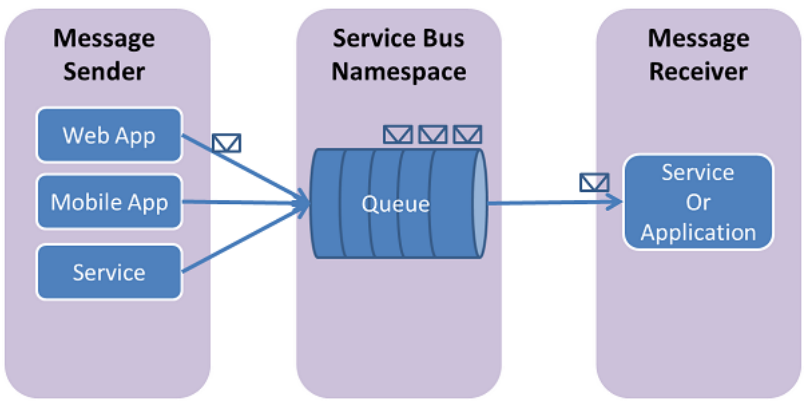
01
Storage Queues

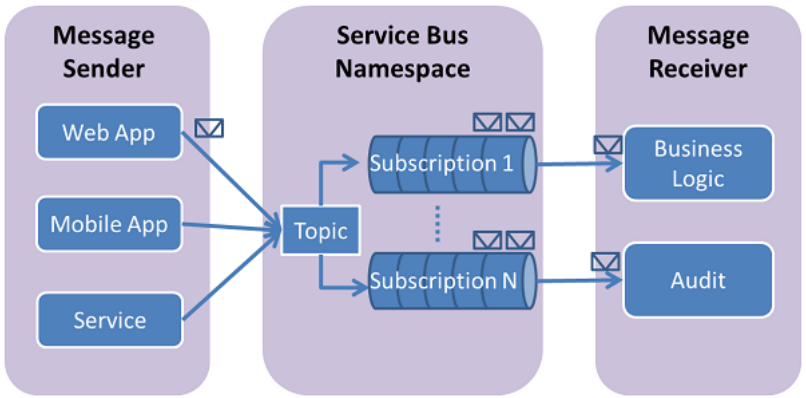
02
Service Bus Queues & Topics

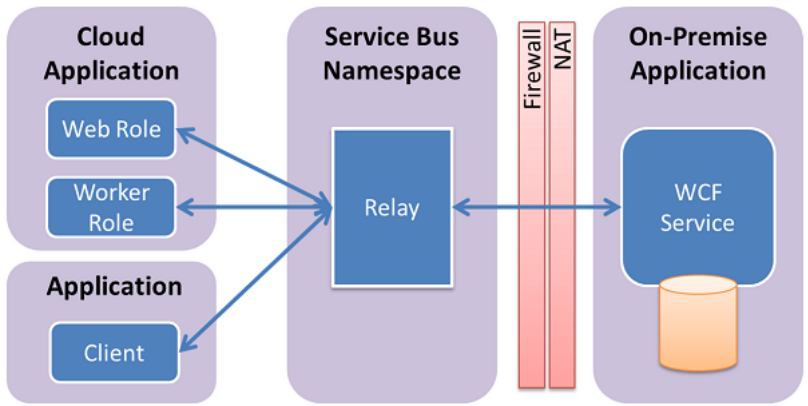
03
Service Bus Relay

04
Event Hubs

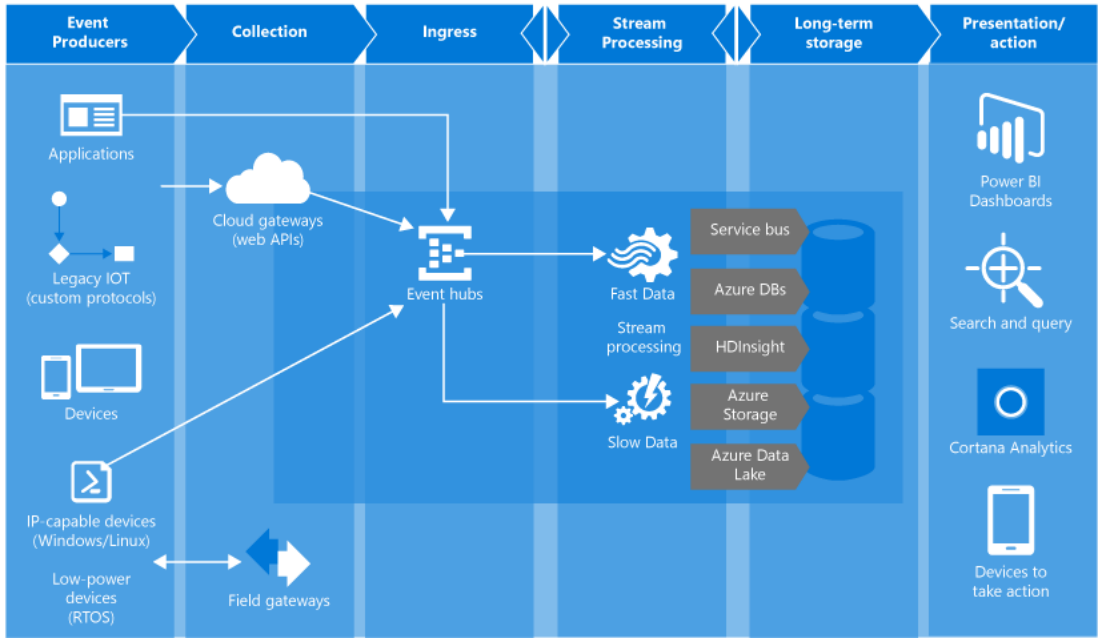
05
Working with Data Streams


Message Sender
Web App
Mobile App
Service
Service Bus Namespace
Queue
Message Receiver
Service Or Application


Message Sender
Web App
Mobile App
Service
Service Bus Namespace
Topic
Subscription 1
Subscription N
Message Receiver
Business Logic
Audit


Cloud Application
Web Role
Worker Role
Application
Client
Service Bus Namespace
Relay
Firewall
NAT
On-Premise Application
WCF Service

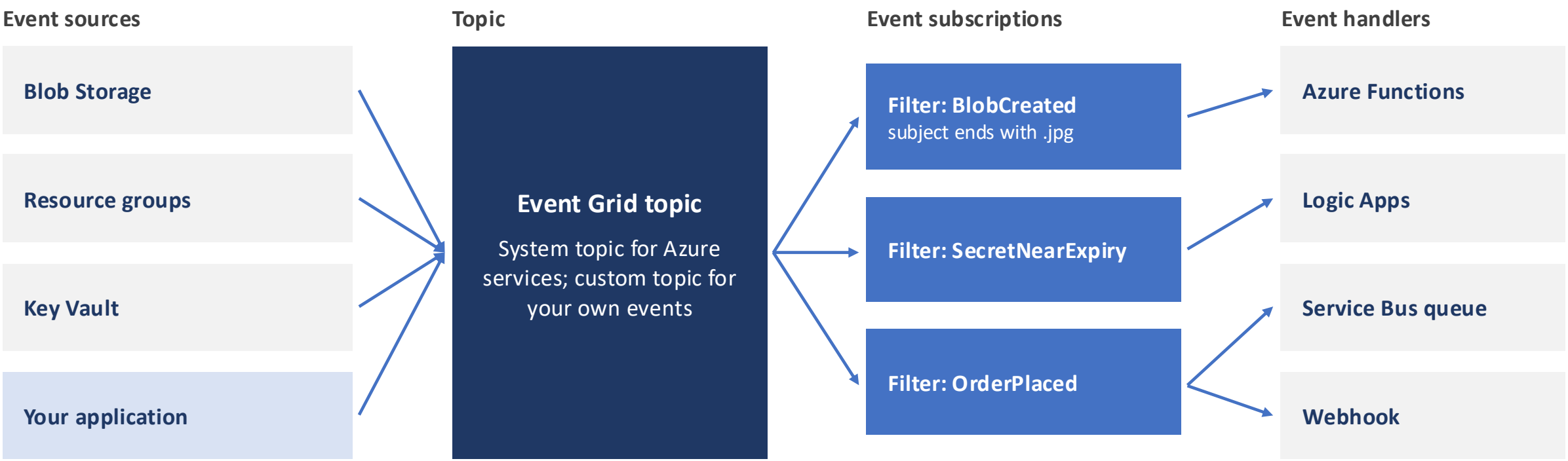
Event Hubs


Event Producers
Collection
Ingress
Stream Processing
Long-term storage
Presentation/ action
Applications
Legacy IOT (custom protocols)
Devices
IP-capable devices (Windows/Linux)
Low-power devices (RTOS)
Cloud gateways (web APIs)
Field gateways
Event hubs
Fast Data
Stream processing
Slow Data
Service bus
Azure DBs
HDInsight
Azure Storage
Azure Data Lake
Power BI Dashboards
Search and query
Cortana Analytics
Devices to take action

Event Hubs

- Events published by producers over AMQP, Apache Kafka protocol or HTTPS
- Consumers use AMQP or Kafka clients
- Capture streaming data and store it in Azure Storage / Data Lake
- Partitions: consumers can process subsets
- Microsoft Entra ID (recommended) or SAS tokens

Event Grid



Push delivery over HTTP · CloudEvents 1.0 schema · Retries for up to 24 hours, then dead-letter to Blob Storage · Pay per operation

Event Grid

- Reacts to discrete events: "something happened", not "do this"
- System topics publish events from Azure services such as Blob Storage and Key Vault
- Subscriptions filter on event type and subject, so each handler gets only what it needs
- At-least-once delivery: make handlers idempotent
- Namespaces add pull delivery and an MQTT broker for devices
- Microsoft Entra ID (recommended) or access keys

Event Grid, Service Bus or Event Hubs?

	Event Grid	Service Bus	Event Hubs
Use it for	Reacting to state changes	Business transactions and commands	High-volume streams of telemetry
Delivery	Pushed to handlers	Pulled from queues and topics	Pulled from partitions
Ordering	Not guaranteed	FIFO with sessions; transactions; duplicate detection	Ordered within a partition
Retention	Retries up to 24 h, then dead-letter	Until consumed or expired; dead-letter queue	Time-based; consumers can replay
Example	Blob uploaded: create a thumbnail	Order placed: charge the payment	Device telemetry: live analytics

Combine them: Event Grid can forward events into a Service Bus queue when a handler needs ordering or transactions

Monitoring & Tooling

Monitoring & Tooling

01

Logging options & Azure Monitor

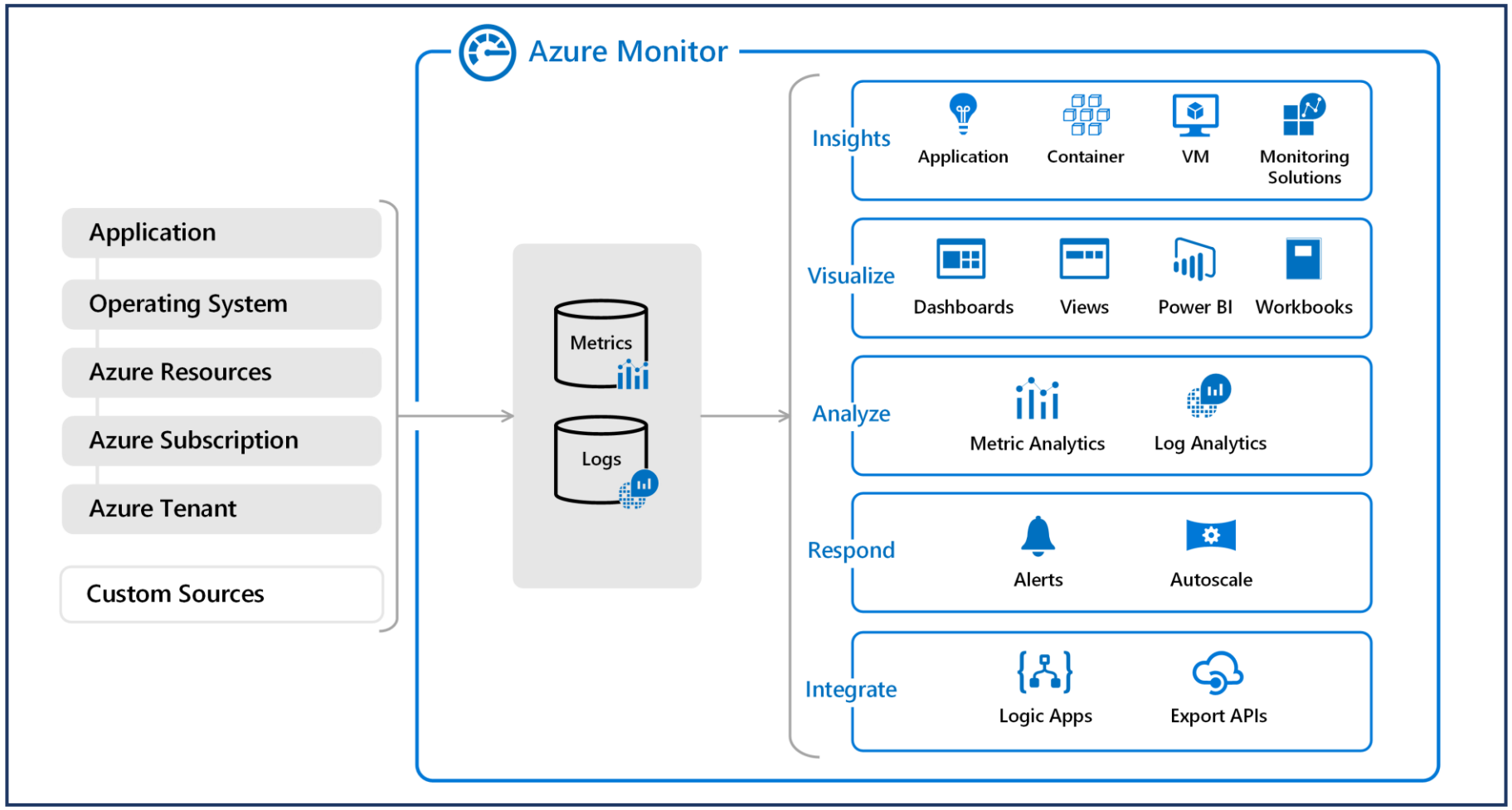
02

Application Insights

03

Azure Resource Manager & Bicep

Explore Azure Monitor (1 / 3)



© Copyright Microsoft Corporation. All rights reserved.

Explore Azure Monitor (2 / 3)

- Azure Monitor collects data from each of the following tiers:
- Application monitoring data
- Guest OS monitoring data
- Azure resource monitoring data
- Azure subscription monitoring data
- Azure tenant monitoring data

- Insights and curated visualizations
- Insights and curated visualizations
- Container Insights
- VM Insights

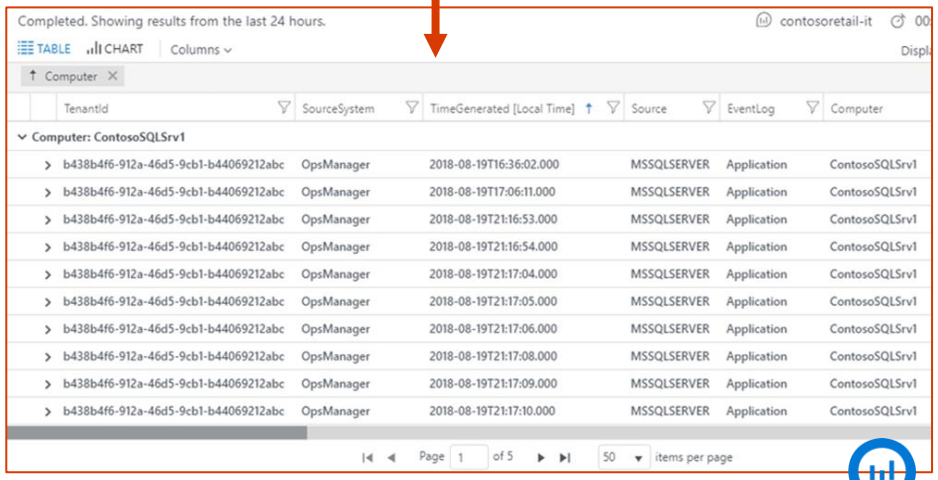
© Copyright Microsoft Corporation. All rights reserved.

Explore Azure Monitor (3 / 3)

Monitoring data platform



Logs



Log Analytics





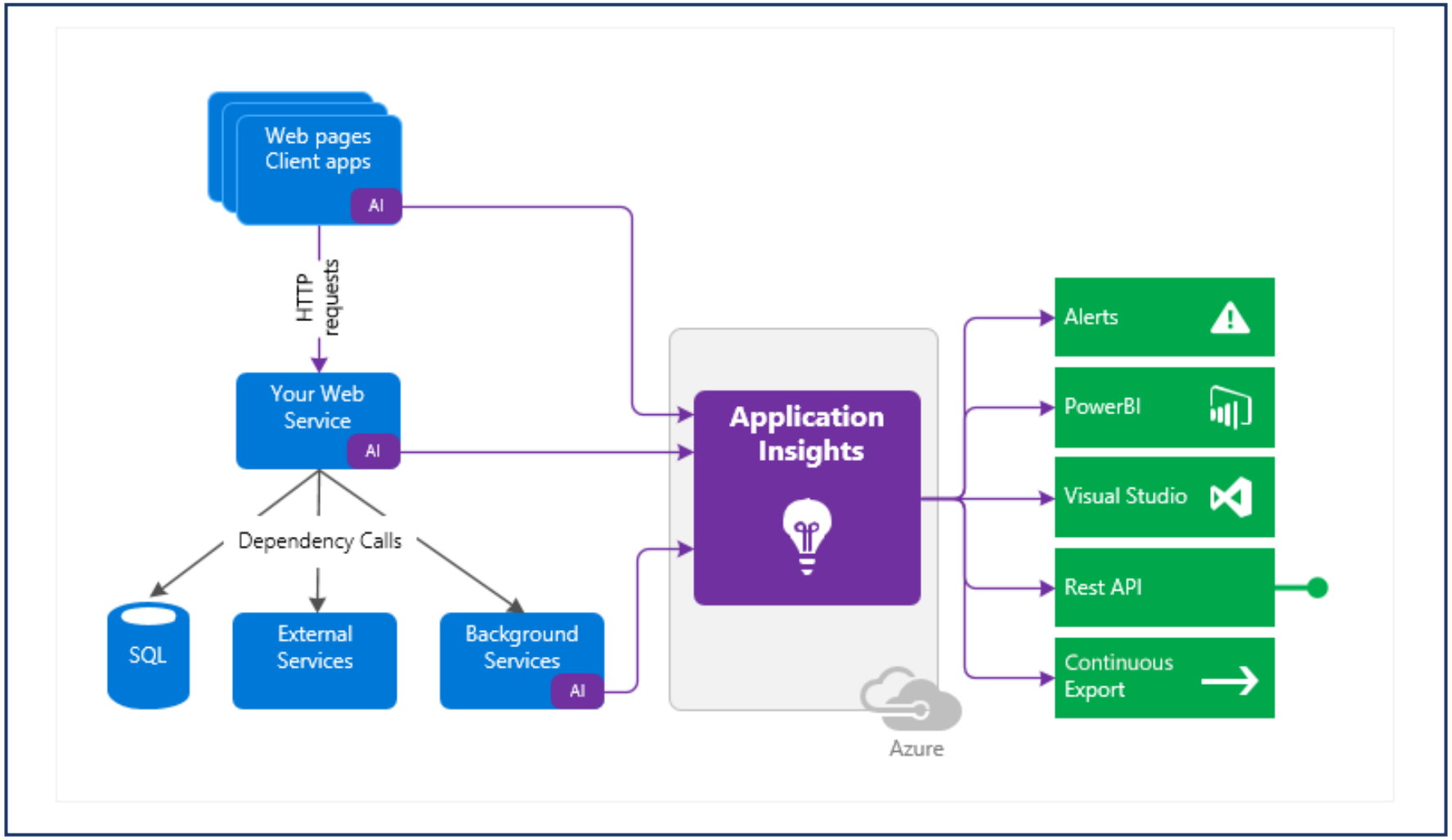
Metrics



Metrics Explorer

© Copyright Microsoft Corporation. All rights reserved.

Explore Application Insights (1 / 2)



© Copyright Microsoft Corporation. All rights reserved.

Explore Application Insights (2 / 2)

- Application Insights helps you understand how your app is performing and how it's being used. It monitors:
  - Request rates, response times, and failure rates
  - Dependency rates, response times, and failure rates
  - Exceptions
  - Page views and load performance
  - AJAX calls from web pages
  - User and session counts.
  - Performance counters
  - Host diagnostics
  - Diagnostic trace logs
  - Custom events and metrics

- There are several ways to get started monitoring and analyzing app performance
  - At run time
  - At development time
  - Instrument your web pages
  - Analyze mobile app usage
  - Availability tests

© Copyright Microsoft Corporation. All rights reserved.

Discover log-based metrics

- Log-based metrics
  - The Application Insights backend stores all collected events as logs, and the Application Insights blades in the Azure portal act as an analytical and diagnostic tool for visualizing event-based data from logs.
  - Using logs to retain a complete set of events can bring great analytical and diagnostic value.
  - Collecting a complete set of events may be impractical (or even impossible) for applications that generate a large volume of telemetry.

- Pre-aggregated metrics
  - Stored as pre-aggregated time series, and only with key dimensions
  - The newer SDKs (Application Insights 2.7 SDK or later for .NET) pre-aggregate metrics during collection
  - For the SDKs that don't implement pre-aggregation, the Application Insights backend still populates the new metrics by aggregating the events received by the Application Insights event collection endpoint

© Copyright Microsoft Corporation. All rights reserved.

Instrument an app for monitoring

- Auto-instrumentation
  - Auto-instrumentation allows you to enable application monitoring with Application Insights without changing your code.
  - All you have to do is enable and, in some cases, configure the agent, which will collect the telemetry automatically.
  - You'll see the metrics, requests, and dependencies in your Application Insights resource

- Instrumenting for distributed tracing
  - How to Enable Distributed Tracing
  - Enabling via Application Insights SDKs
  - Enable via OpenCensus

© Copyright Microsoft Corporation. All rights reserved.

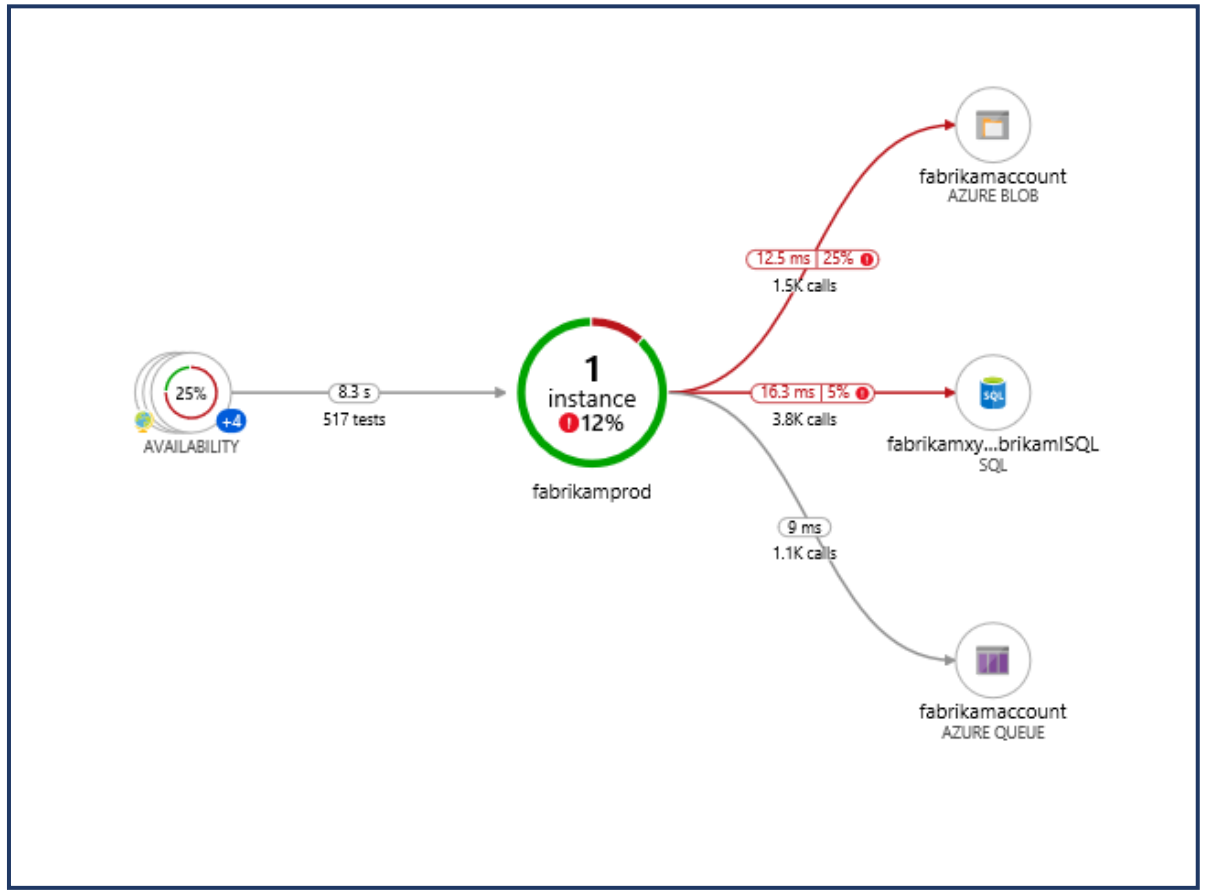
Select an availability test

- You can set up availability tests for any HTTP or HTTPS endpoint
- You don't have to make any changes to the website you're testing
- It doesn't even have to be a site that you own, you can test the availability of a REST API that your service depends on.
- You can create up to 100 availability tests per Application Insights resource, and there are three types of availability tests
  - URL ping test (classic)
  - Standard test (Preview)
  - Custom TrackAvailability

© Copyright Microsoft Corporation. All rights reserved.

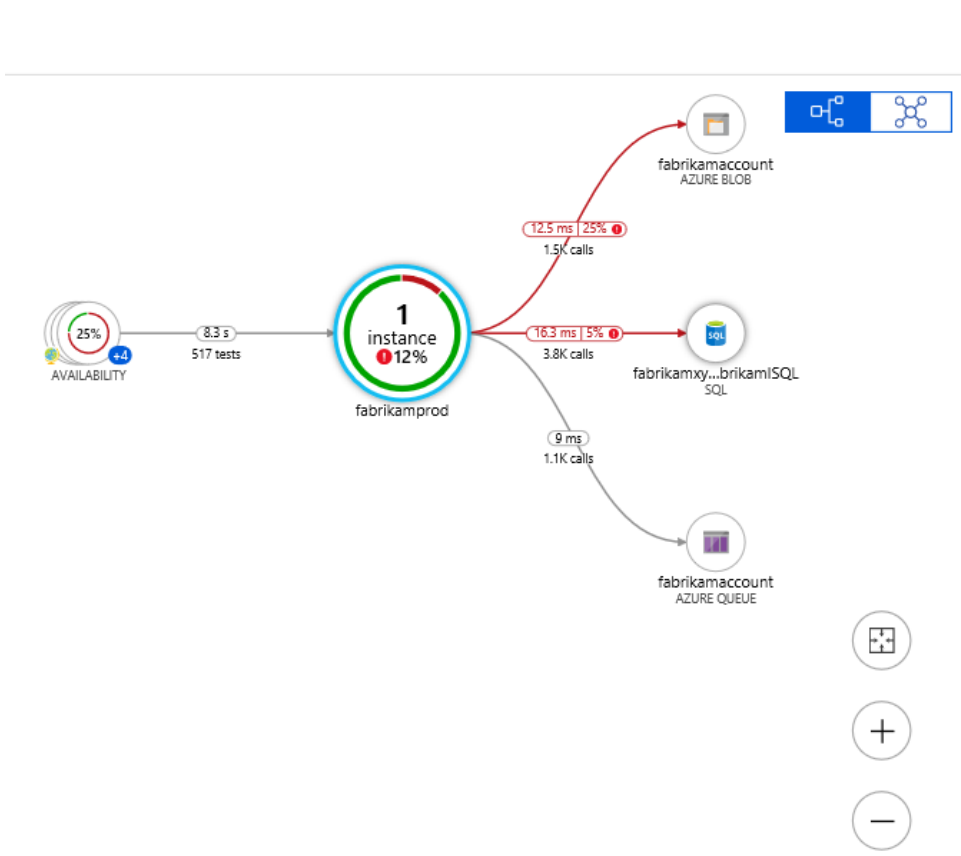
Troubleshoot app performance by using Application Map (1 / 2)

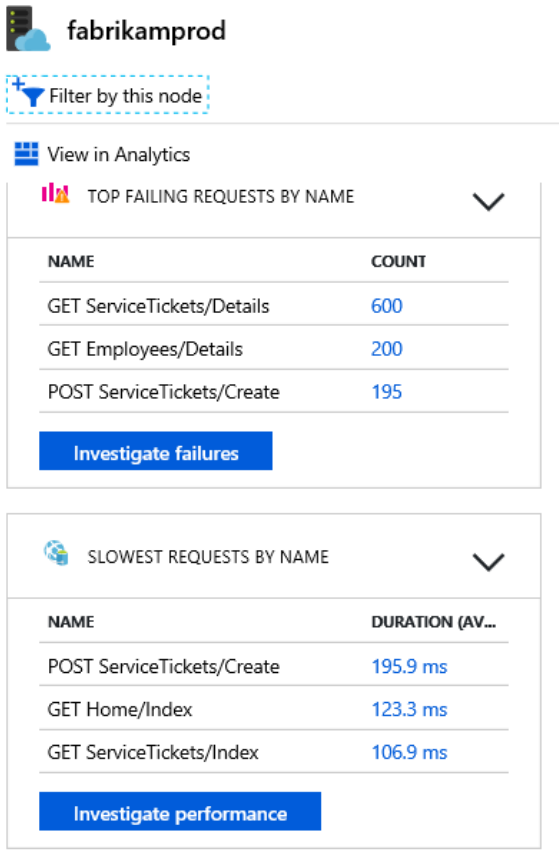
- Application Map helps you spot performance bottlenecks or failure hotspots across all components of your distributed application.
- Each node on the map represents an application component or its dependencies; and has health KPI and alerts status.
- You can click through from any component to more detailed diagnostics
- Components are independently deployable parts of your distributed/microservices application



© Copyright Microsoft Corporation. All rights reserved.

Troubleshoot app performance by using Application Map (2 / 2)



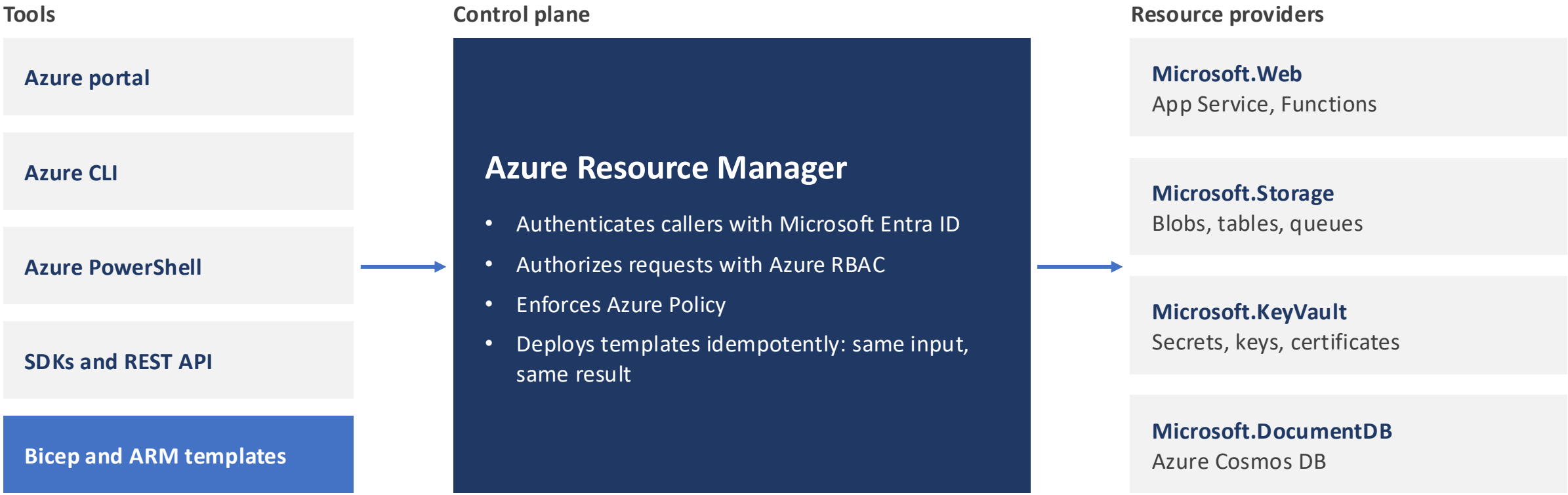


© Copyright Microsoft Corporation. All rights reserved.

Azure Resource Manager & Bicep

What is Azure Resource Manager?

- Every Azure tool goes through the same control plane



Deployment scopes: Tenant → Management group → Subscription → Resource group

Why infrastructure as code?

- Describe the environment in a file and let Azure Resource Manager build it



Repeatable

The same template builds dev, test and production, so environments stop drifting apart



Reviewable

Infrastructure changes go through pull requests and source control like application code



Idempotent

Redeploying an unchanged template changes nothing; Resource Manager converges to the declared state



Predictable

what-if shows exactly what a deployment will create, change or delete before it runs

ARM templates vs Bicep

- Bicep compiles to the same ARM JSON in less than half the lines

ARM template (JSON)

```
{
  "$schema": "…/deploymentTemplate.json#",
  "contentVersion": "1.0.0.0",
  "parameters": {
    "storageName": { "type": "string" },
    "location": {
      "type": "string",
      "defaultValue": "[resourceGroup().location]"
    }
  },
  "resources": [{
    "type": "Microsoft.Storage/storageAccounts",
    "apiVersion": "2026-04-01",
    "name": "[parameters('storageName')]",
    "location": "[parameters('location')]",
    "sku": { "name": "Standard_LRS" },
    "kind": "StorageV2"
  }]
}
```

Bicep

```
param storageName string
param location string = resourceGroup().location

resource storage 'Microsoft.Storage/storageAccounts@2026-04-01' = {
  name: storageName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}
```

Anatomy of a Bicep file

- Four building blocks: parameters, variables, resources and outputs

```
// Parameters: values supplied at deployment time
param location string = resourceGroup().location
param environment string = 'dev'

// Variables: values computed inside the template
var storageName = 'st${uniqueString(resourceGroup().id)}'

// Resources: what Resource Manager should create
resource storage 'Microsoft.Storage/storageAccounts@2026-04-01' = {
  name: storageName
  location: location
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
  tags: { environment: environment }
}

// Outputs: values returned after the deployment
output blobEndpoint string = storage.properties.primaryEndpoints.blob
```

Parameters and parameter files

- One template, plus one parameter file per environment

main.bicep

```
@description('Deployment environment')
@allowed([ 'dev', 'test', 'prod' ])
param environment string = 'dev'

@minLength(3)
@maxLength(24)
param storageName string

@secure()
param apiKey string
```

main.prod.bicepparam

```
using './main.bicep'

param environment = 'prod'
param storageName = 'stcontosoprod01'
param apiKey = readEnvironmentVariable('API_KEY')
```

- Decorators validate input before Resource Manager deploys anything
- `@secure()` keeps the value out of deployment history and logs
- Keep secrets out of parameter files: read them from the environment or from Key Vault

Resource references and dependencies

- Reference a resource by its symbolic name and Bicep works out the deployment order

```
resource plan 'Microsoft.Web/serverfarms@2024-04-01' = {
  name: 'plan-${appName}'
  location: location
  sku: { name: 'B1' }
}

resource app 'Microsoft.Web/sites@2024-04-01' = {
  name: appName
  location: location
  properties: {
    serverFarmId: plan.id      // implicit dependency: the plan deploys first
    httpsOnly: true
  }
}

// Reference a resource that already exists, without redeploying it
resource vault 'Microsoft.KeyVault/vaults@2023-07-01' existing = {
  name: vaultName
}
```

Modules

- Split a deployment into reusable modules, and reuse Azure Verified Modules

```
// Local module: a Bicep file in your repository
module storage 'modules/storage.bicep' = {
  name: 'storage'
  params: { storageName: storageName, location: location }
}

module web 'modules/webapp.bicep' = {
  name: 'web'
  params: {
    appName: appName
    blobEndpoint: storage.outputs.blobEndpoint   // output of another module
  }
}

// Azure Verified Module from the public Bicep registry (pin a published version)
module vault 'br/public:avm/res/key-vault/vault:0.11.0' = {
  name: 'vault'
  params: { name: vaultName }
}
```

Loops and conditions

- Create resources from an array, and deploy them only when a condition holds

```
param locations array = [ 'westeurope', 'northeurope' ]
param deployInsights bool = true

// Loop: one storage account per location
resource stores 'Microsoft.Storage/storageAccounts@2026-04-01' = [for loc in locations: {
  name: 'st${uniqueString(resourceGroup().id, loc)}'
  location: loc
  sku: { name: 'Standard_LRS' }
  kind: 'StorageV2'
}]

// Condition: deploy only when the parameter is true
resource insights 'Microsoft.Insights/components@2020-02-02' = if (deployInsights) {
  name: 'appi-${appName}'
  location: location
  kind: 'web'
  properties: { Application_Type: 'web', WorkspaceResourceId: workspaceId }
}
```

Managed identity and role assignments

- Grant the web app access to Key Vault without a secret in the template

```
resource app 'Microsoft.Web/sites@2024-04-01' = {
  name: appName
  location: location
  identity: { type: 'SystemAssigned' }      // managed identity
  properties: {
    serverFarmId: plan.id
    siteConfig: { appSettings: [ { name: 'KeyVaultUri', value: vault.properties.vaultUri } ] }
  }
}

var secretsUser = '4633458b-17de-408a-b874-0445c86b69e6'   // Key Vault Secrets User role

resource kvAccess 'Microsoft.Authorization/roleAssignments@2022-04-01' = {
  name: guid(vault.id, app.id, secretsUser)
  scope: vault
  properties: {
    roleDefinitionId: subscriptionResourceId('Microsoft.Authorization/roleDefinitions', secretsUser)
    principalId: app.identity.principalId
    principalType: 'ServicePrincipal'
  }
}
```

Deploying Bicep files

- Preview the changes with what-if, then deploy

```
# Preview what Resource Manager will create, change or delete
az deployment group what-if --resource-group rg-azure4devs --parameters main.prod.bicepparam

# Deploy the template (incremental mode by default)
az deployment group create --resource-group rg-azure4devs --parameters main.prod.bicepparam

# Deployment stack: manages the resources as one unit and cleans up removed ones
az stack group create --name app-stack --resource-group rg-azure4devs \
  --parameters main.prod.bicepparam --action-on-unmanage deleteResources --deny-settings-mode none
```

Incremental (default)

Adds and updates the resources in the template; leaves other resources in the group untouched

Complete

Also deletes resources in the group that are not in the template

Deployment stack

Tracks the resources it manages; can delete or lock them. Use it instead of complete mode

Bicep tooling

- Write, check and reuse templates with the free Bicep tools



VS Code extension

IntelliSense for every resource type, validation as you type and a resource visualizer



Linter

Rules in `bicepconfig.json` flag problems such as unused parameters before you deploy



Decompile

`az bicep decompile` turns existing ARM JSON, including portal exports, into Bicep



Azure Verified Modules

Microsoft-maintained modules in the public Bicep registry for common resources

Alternative: Pulumi

- Infrastructure as code in C#, TypeScript, Python, Go or Java

Program.cs

```
using Pulumi;
using Pulumi.AzureNative.Resources;
using Pulumi.AzureNative.Storage;
using Pulumi.AzureNative.Storage.Inputs;

return await Deployment.RunAsync(() =>
{
    var rg = new ResourceGroup("rg-azure4devs");

    var storage = new StorageAccount("st", new StorageAccountArgs
    {
        ResourceGroupName = rg.Name,
        Sku = new SkuArgs { Name = SkuName.Standard_LRS },
        Kind = Kind.StorageV2,
    });

    return new Dictionary<string, object?>
    {
        ["blobEndpoint"] = storage.PrimaryEndpoints.Apply(e => e.Blob)
    };
});
```

Workflow

```
pulumi new azure-csharp
pulumi preview   # like what-if
pulumi up        # deploy
pulumi destroy
```

- Loops, functions, NuGet packages and unit tests come from the language you already use
- Azure Native provider is generated from the same Resource Manager APIs as Bicep
- Pulumi keeps a state file in Pulumi Cloud or a backend you choose, such as Blob Storage

Bicep, Terraform or Pulumi?

- All three deploy the same Azure resources; they differ in language, state and reach

	Bicep	Terraform	Pulumi
Language	Bicep (domain-specific)	HCL (domain-specific)	C#, TypeScript, Python, Go, Java
State	Kept by Azure Resource Manager	State file you store and lock	State in Pulumi Cloud or your own backend
Clouds	Azure only	Azure, AWS, Google Cloud and more	Azure, AWS, Google Cloud and more
Preview	what-if	terraform plan	pulumi preview
New Azure APIs	Available as soon as Resource Manager ships them	azurerm provider; azapi for any API version	Azure Native provider generated from Resource Manager APIs
License	Free, MIT	Business Source License (OpenTofu is an open-source fork)	Free engine, Apache 2.0